



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Synchronous Sequential Logic

Team DDCO

Department of Computer Science and Engineering

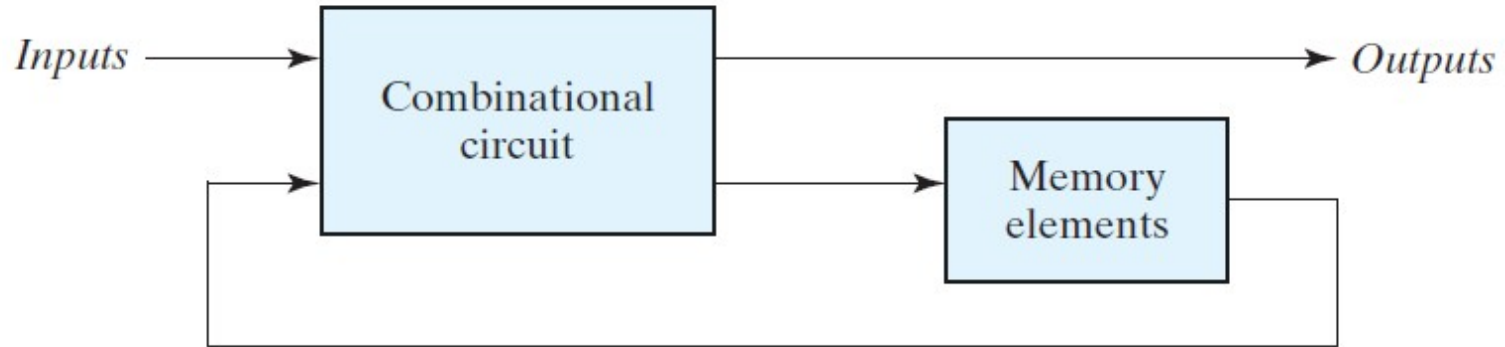
Introduction- T1- section 5.1



- Hand-held devices, cell phones, navigation receivers, personal computers, digital cameras, personal media players, and virtually all electronic consumer products have the ability to send, receive, store, retrieve, and process information represented in a binary format.
- The technology enabling and supporting these devices is critically dependent on electronic components that can store information, i.e., have memory.
- This chapter examines the operation and control of these devices and their use in circuits.

- The digital circuits considered thus far have been **combinational**.
- **Their output depends only and immediately on their inputs—they have no memory, i.e., dependence on past values of their inputs.**
- Sequential circuits, however, act as **storage elements and have memory**.
- They can store, retain, and then retrieve information when needed at a later time.

Sequential Circuits (T1- section 5.2)



Sequential Circuits



- It consists of a ***combinational circuit*** to which ***storage elements*** are connected to form a ***feedback path***.
- The storage elements are devices capable of storing ***binary information***.
- The binary information stored in these elements at any given time *defines the **state** of the sequential circuit* at that time.
- The sequential circuit receives binary information from external inputs that, together with the present state of the storage elements, determine the binary value of the outputs.
- These **external inputs** also determine the condition for changing the state in the storage elements.
- The ***next state*** of the *storage elements* is also a function of ***external inputs*** and the ***present state***.

Sequential Circuits: Key Points

- Depends on current and prior input
- Memory element
- Feedback/cycles
- Remember previous input
- Next state= external input + present state
- State of system-binary information stored in system
- State variables

Thus, a sequential circuit is specified by a time sequence of inputs, outputs, and internal states.

Types of Sequential Circuits



There are two main types of sequential circuits, and their classification is a function of the timing of their signals.

1. A **synchronous sequential circuit** is a system whose behavior can be defined from the knowledge of its signals at discrete instants of time.
2. The behavior of an **asynchronous sequential circuit** depends upon the **input signals** at any instant of time and the order in which the inputs change.

Synchronous Sequential Circuit

- A synchronous sequential circuit employs signals that affect the storage elements at only discrete instants of time.
- Synchronization is achieved by a timing device called a **clock generator**, which provides a clock signal having the form of a periodic train of clock pulses.
- The clock signal is commonly denoted by the identifiers ***clock*** and ***clk***.
- The **clock pulses are distributed throughout the system in such a way that storage elements are affected only with the arrival of each pulse.**

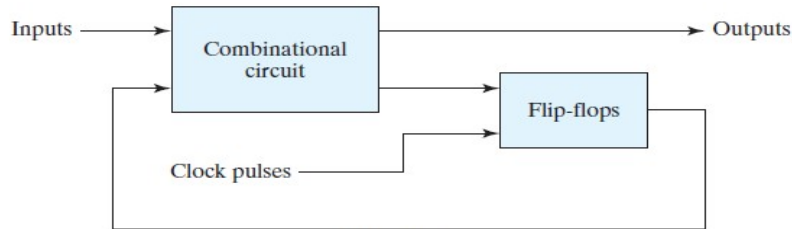
Synchronous Sequential Circuit



- In practice, the clock pulses determine when computational activity will occur within the circuit, and other signals (external inputs and otherwise) determine what changes will take place affecting the storage elements and the outputs.
- For example, a **circuit that is to add and store two binary numbers would compute their sum from the values of the numbers and store the sum at the occurrence of a clock pulse.**
- Synchronous sequential circuits that use clock pulses to control storage elements are called **clocked sequential circuits**

Synchronous Sequential Circuit

- The storage elements (memory) used in clocked sequential circuits are **called flip flops**. A flip-flop is a binary storage device capable of storing one bit of information. In a stable state, the output of a flip-flop is either **0 or 1**



(a) Block diagram



(b) Timing diagram of clock pulses

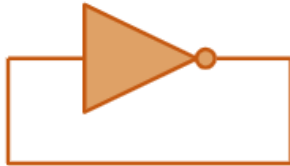
- The new value is stored (i.e., the flip-flop is updated) when a pulse of the clock signal occurs

Synchronous Sequential Circuit

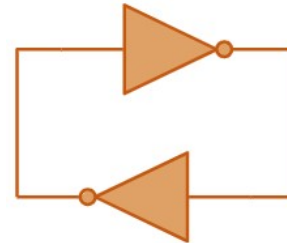
- Propagation delays play an important role in determining the minimum interval between clock pulses that will allow the circuit to operate correctly. **A change in state of the flip-flops is initiated only by a clock pulse transition—for example, when the value of the clock signals changes from 0 to 1**
- When a **clock pulse is not active**, the feedback loop between the value stored in the flip-flop and the value formed at the input to the flip-flop is effectively broken because the flip flop outputs cannot change even if the outputs of the combinational circuit driving their inputs change in value.
- Thus, **the transition** from one state to the next occurs only at predetermined intervals dictated by the **clock pulses**.

How to Implement Memory?

- The inverter is essentially the simplest logic gate
- What happens when we connect its output to input, forming a loop?
- Inverter loop is thus not in a **stable state**



- How about a loop of two inverters?
- A two-inverter loop can be in one of **two stable states**.
- A bit can have one of **two different values**.
Thus, a two-inverter loop can **store** (or “remember”) a single bit.
- But how can we **change the bit stored** or the **stable state**?



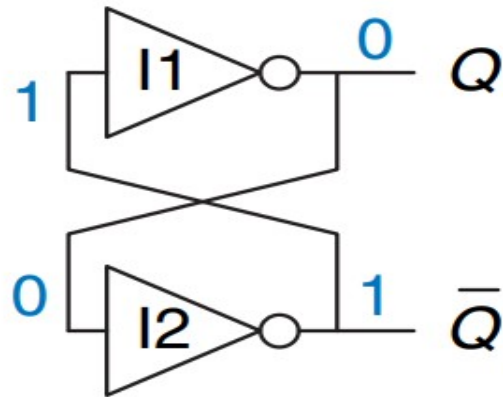
Storage Using Cross-Coupled Inverters



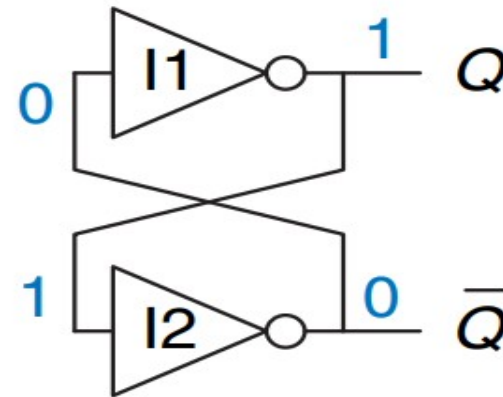
- **Cross-coupled inverters** form a basic memory element with **two stable states**.
- A system with **N stable states** can store up to **$\log_2(N)$ bits of information**.
- Therefore, a **bistable element** (with 2 stable states) can store **1 bit** of information.

Storage Using Cross-Coupled Inverters

Bistable Operation of Cross-Coupled Inverters



(a)



(b)

Storage Using Cross-Coupled Inverters



- When power is first applied to sequential circuit, initial state is unknown.
- It changes each time clock is on, it is unpredictable which is not desirable.
- **Cross coupled inverters are not practical because user has no input to control the state**
- **Hence we have flipflops as memory elements**

Asynchronous Sequential Circuit

- The storage elements commonly used in asynchronous sequential circuits are **time-delay devices**.
- The storage capability of a time-delay device varies with the time it takes for the signal to propagate through the device.
- An asynchronous sequential circuit is **one where the outputs (and next states) can change immediately when inputs change, without waiting for a clock pulse**.
- In gate-type asynchronous systems, **the storage elements consist of logic gates whose propagation delay provides the required storage**.
- Thus, an **asynchronous sequential circuit may be regarded as a combinational circuit with feedback**.

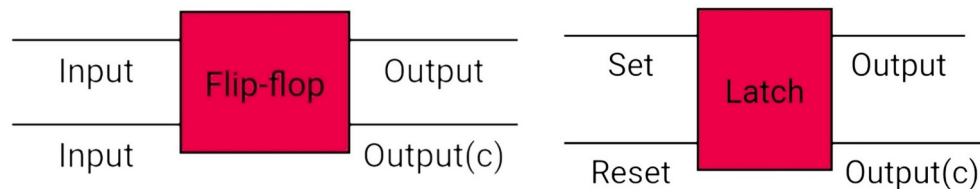
Flip-Flops-key points

- The storage elements (memory) used in clocked sequential circuits are called flip-flops.
- A flip-flop is a binary storage device capable of storing one bit of information.
- In a stable state, the output of a flip-flop is either 0 or 1
- The new value is stored (i.e., the flip-flop is updated) when a pulse of the clock signal occurs.
- A change in state of the flip-flops is initiated only by a clock pulse transition, eg: when the value of the clock signals changes from 0 to 1
- Thus, the transition from one state to the next occurs only at predetermined intervals dictated by the clock pulses.

- A storage element in a digital circuit can maintain a binary state **indefinitely** (as long as power is delivered to the circuit), until directed by an input signal to switch states.
- The major differences among various types of storage elements are in the number of inputs they possess and in the manner in which the inputs affect the binary state.
- Storage elements that operate with **signal levels** (rather than signal transitions) are referred to as latches ; those controlled by a **clock transition** are **flip-flops** .
- Latches are said to be **level sensitive devices**; flip-flops are edge-sensitive devices.

Latches

- The two types of storage elements are related because latches are the basic circuits from which all flip-flops are constructed.
- Although latches are useful for storing binary information and for the design of asynchronous sequential circuits, they are not practical for use as storage elements in synchronous sequential circuits.
- They are the building blocks of flip-flops.



Difference between flip flop and latch

Difference b/w Latches & Flip-Flops

Latches	Flip-Flops
Latches are building blocks of sequential circuits, these can be built from logic gates	Flip flops are also building blocks of sequential circuits. But, these can be built from the latches .
Latch continuously checks its inputs and changes its output correspondingly.	Flip flop continuously checks its inputs and changes its output correspondingly only at times determined by clocking signal
The latch is sensitive to the duration of the pulse and can send or receive the data when the switch is on .	Flipflop is sensitive to a signal change . They can transfer data only at the single instant and data cannot be changed until next signal change . Flip flops are used as a register.
It is based on the enable function input .	It works on the basis of clock pulses.
It is a level triggered , it means that the output of the present state and input of the next state depends on the level that is binary input 1 or 0. (asynchronous circuits)	It is an edge triggered , it means that the output and the next state input changes when there is a change in clock pulse whether it may a+ve or-ve clock pulse. (synchronous circuits)

Difference b/w Latches & Flip-Flops



Example: Traffic Light Controller

Scenario:

A city traffic light controller uses digital storage elements to decide when to change from Green → Yellow → Red.

Using a Latch (Level-triggered)

Suppose the system uses a **latch** to store the current state of the traffic light.

Since a latch is **level-sensitive**, whenever the clock/enable signal is **high**, the latch is “transparent” — it immediately passes input changes to the output.

Problem: If there is noise or a small glitch during the enable-high period, the latch will capture it. This could cause the traffic light to change state unexpectedly (e.g., Green jumps to Red without Yellow).

Application-level interpretation: **Not safe** for traffic control, because continuous input changes during the active level may corrupt the output.

Difference b/w Latches & Flip-Flops



Example: Traffic Light Controller

Scenario:

A city traffic light controller uses digital storage elements to decide when to change from Green → Yellow → Red.

Using a Flip-Flop (Edge-triggered)

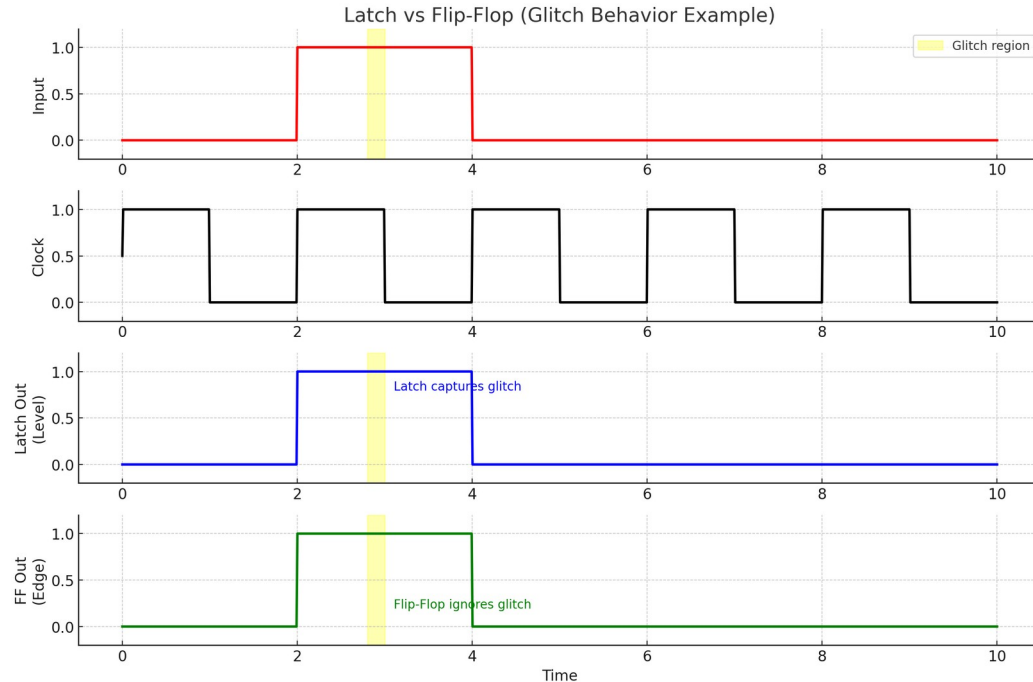
If the system uses a **Flip-Flop**, the light state is updated **only at the clock's rising (or falling) edge**.

Input changes happening in between edges are ignored until the next edge.

Advantage: The traffic light changes state in a controlled manner (e.g., every 30 seconds on the rising clock edge). Even if input glitches occur while the clock is low, they won't affect the state.

Application-level interpretation: **Much safer and reliable** for real-world sequential control (traffic lights, elevators, etc.).

Difference b/w Latches & Flip-Flops



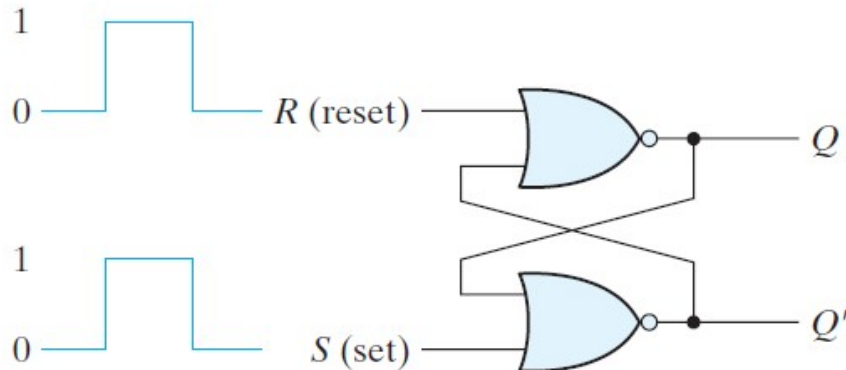
Yellow region → glitch in the input.

Latch Output (blue) → immediately captures the glitch while clock is HIGH.

Flip-Flop Output (green) → ignores the glitch, updates only at the clock edge.

SR Latch (Using NOR Gates) T1- section 5.3

- The SR latch is a circuit with two cross-coupled NOR gates, and two inputs labeled S for set and R for reset. The latch has two useful states.
- When output $Q = 1$ and $Q' = 0$, the latch is said to be in the **set state**.
- When $Q = 0$ and $Q' = 1$, it is in the **reset state**.
- Outputs Q and Q' are normally the complement of each other.



(a) Logic diagram

Truth table of NOR Gate

Input A	Input B	$0 = (A + B)'$
0	0	1
0	1	0
1	0	0
1	1	0

S	R	Q	Q'
1	0	1	0
0	0	1	0 (after $S = 1, R = 0$)
0	1	0	1
0	0	0	1 (after $S = 0, R = 1$)
1	1	0	0 (forbidden)

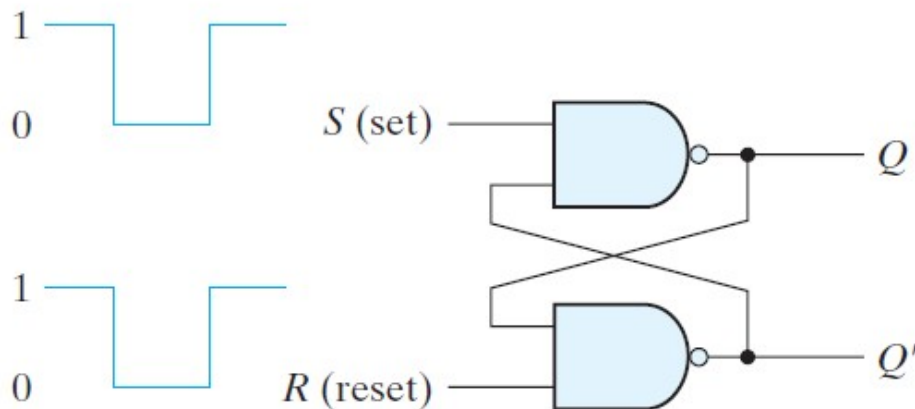
(b) Function table

SR Latch (Using NAND Gates)

- The SR latch is a circuit with two cross-coupled NAND gates and two inputs labeled S for set and R for reset.
- It operates with both inputs **normally at 1**, unless the state of the latch has to be changed.

Truth table of NAND Gate

A	B	Output
0	0	1
1	0	1
0	1	1
1	1	0



(a) Logic diagram

S	R	Q	Q'
1	0	0	1
1	1	0	1 (after $S = 1, R = 0$)
0	1	1	0
1	1	1	0 (after $S = 0, R = 1$)
0	0	1	1 (forbidden)

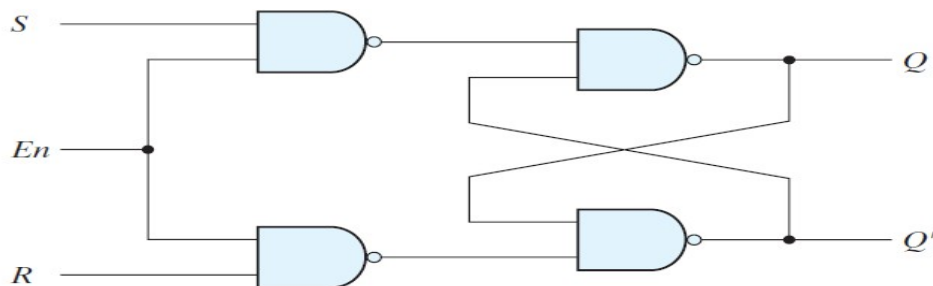
(b) Function table

SR Latch (Using NAND Gates)

- The application of **0 to the S input** causes output Q to go to 1, putting the latch in the **set state**.
- When the S input goes back to 1, the circuit remains in the set state.
- After both inputs go back to 1, we are allowed to change the state of the latch by placing a **0 in the R input**.
- This action causes the circuit to go to the **reset state** and stay there even after both inputs return to 1.
- The condition that is forbidden for the NAND latch is **both inputs being equal to 0 at the same time**, an input combination that should be **avoided**.

SR Latch with Control Input

- In comparing the NAND with the NOR latch, note that the input signals for the NAND require the complement of those values used for the NOR latch. Because the NAND latch requires a 0 signal to change its state, it is sometimes referred to as an $S'R'$ latch.
- The operation of the basic SR latch can be modified by providing an additional input signal that determines (controls) when the state of the latch can be changed by determining whether S and R (or S' and R') can affect the circuit.
- It consists of the basic SR latch and two additional NAND gates.



(a) Logic diagram

En	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$; reset state
1	1	0	$Q = 1$; set state
1	1	1	Indeterminate

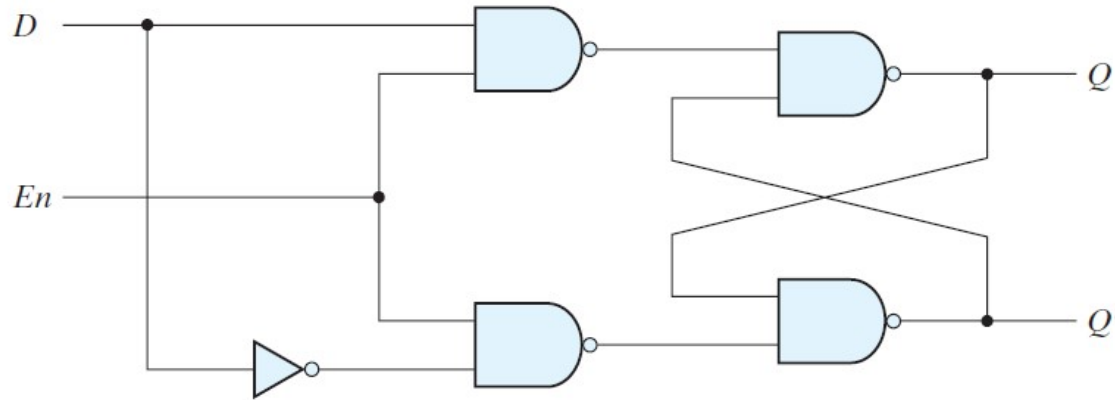
(b) Function table

SR Latch with Control Input

- The control input **En** acts as an **enable signal** for the other two inputs.
- The outputs of the NAND gates stay at the logic-1 level as long as the enable signal remains at 0.
- This is the quiescent condition for the SR latch.
- When the **enable input goes to 1, information from the S or R input is allowed to affect the latch.**
- The **set state** is reached with **$S = 1$, $R = 0$, and $En = 1$.**(active-high enabled).
- To change to the **reset state**, the inputs must be **$S = 0$, $R = 1$, and $En = 1$.**
- In either case, **when En returns to 0, the circuit remains in its current state.**
- The **control input disables the circuit by applying 0 to En , so that the state of the output does not change regardless of the values of S and R.**
- An **indeterminate condition occurs when all three inputs are equal to 1.**

D Latch(Transparent Latch)

- One way to eliminate the undesirable condition of the indeterminate state in the SR latch is to ensure that inputs S and R are never equal to 1 at the same time. This is done in the D latch.
- This latch has only two inputs: D (data) and En (enable).



(a) Logic diagram

<i>En</i>	<i>D</i>	Next state of <i>Q</i>
0	X	No change
1	0	$Q = 0$; reset state
1	1	$Q = 1$; set state

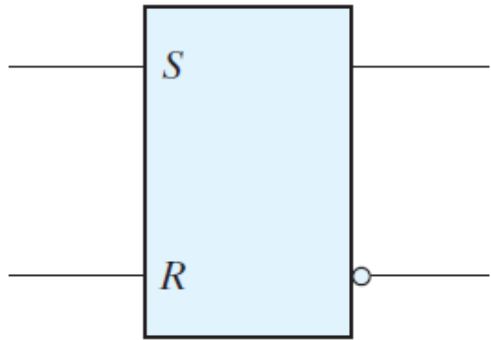
(b) Function table

D Latch

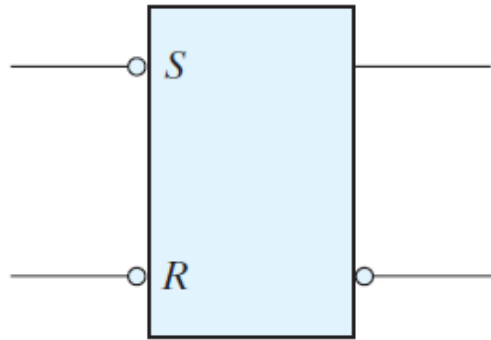
- As long as the **enable input is at 0**, the cross-coupled SR latch has both inputs at the 1 level and the **circuit cannot change state regardless of the value of D.**
- The binary information present at the data input of the D latch is transferred to the Q output when the enable input is asserted.
- **The output follows changes in the data input as long as the enable input is asserted.**
- This situation provides a **path from input D to the output**, and for this reason, the circuit is often called a **transparent latch.**
- When the **enable input signal is de-asserted**, the binary information that was present at the data input at the time the transition occurred is **retained** (i.e., stored) at the Q output until the enable input is asserted again.

Graphic Symbols for Latches

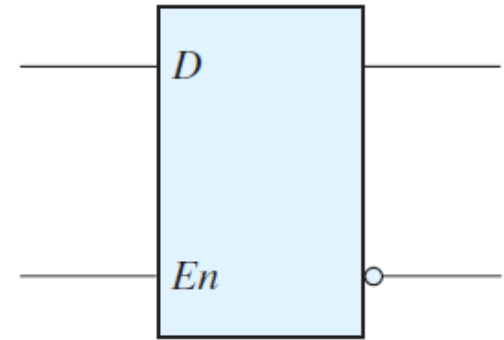
- A latch is designated by a rectangular block with inputs on the left and outputs on the right.
- One output designates the normal output, and the other (with the bubble designation) designates the complement output.



SR



$\bar{S}\bar{R}$



D

Flip-Flops T1- section 5.4

- When latches are used for the storage elements, a serious difficulty arises. The state transitions of the latches start as soon as the clock pulse changes to the logic-1 level. **The new state of a latch appears at the output while the pulse is still active.**
- **If the inputs applied to the latches change while the clock pulse is still at the logic-1 level,** the latches will respond to new values and a new output state may occur. The result is an unpredictable situation.
- **A flipflop triggers only during a signal transition (from 0 to 1 or from 1 to 0) of the synchronizing signal (clock) and is disabled during the rest of the clock pulse.**

Clock Response in Latch and Flip-Flop



(a) Response to positive level

The problem with the latch is that it responds to a change in the level of a clock pulse.



(b) Positive-edge response

The key to the proper operation of a flip-flop is to trigger it only during a signal transition



(c) Negative-edge response

A clock pulse goes through two transitions: from 0 to 1 and the return from 1 to 0.

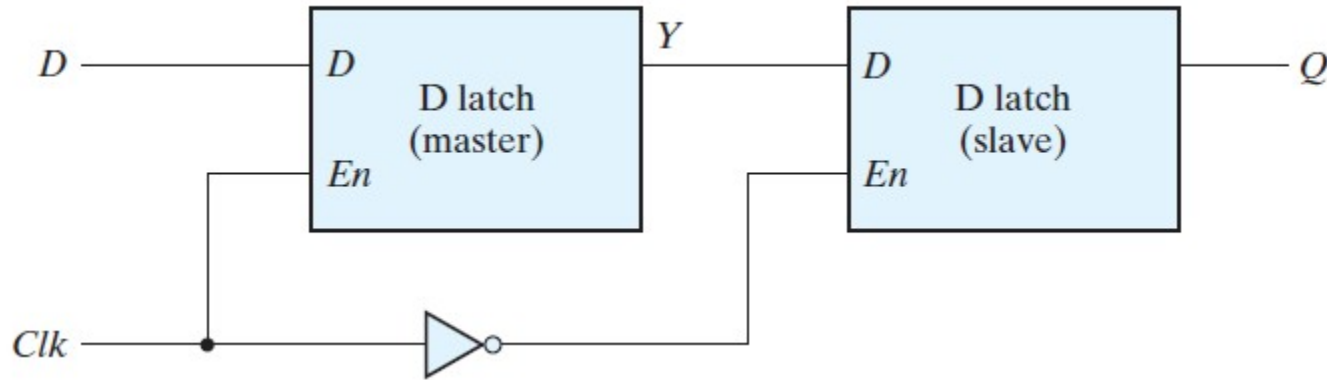
Flip-Flop

There are two ways that a latch can be modified to form a flip-flop

- One way is to employ two latches in a special configuration that isolates the output of the flip-flop and prevents it from being affected while the input to the flip-flop is changing.
- Another way is to produce a flip-flop that triggers only during a signal transition (from 0 to 1 or from 1 to 0) of the synchronizing signal (clock) and is disabled during the rest of the clock pulse.

Edge-Triggered D Flip-Flop

- The construction of a D flip-flop with two D latches and an inverter:



- Also called **Master Slave D Flip-Flop**.
- The first latch is called the master and the second the slave.
- The circuit samples the D input and changes its output Q only at the negative edge of the synchronizing or controlling clock (designated as Clk).

Edge-Triggered D Flip-Flop

- When $\text{clk}=1$ master is enabled and $\text{clk}=0$ slave is enabled.
- Any change in the input changes the master output at Y, but cannot affect the slave output.
- Thus, a change in the output of the flip-flop can be triggered only by and during the transition of the clock from 1 to 0.
- The behavior of the master–slave flip-flop just described dictates that
 - the output may change only once,
 - a change in the output is triggered by the negative edge of the clock,
 - the change may occur only during the clock's negative level.
- The value that is produced at the output of the flip-flop is the value that was stored in the master stage immediately before the negative edge occurred .

Edge-Triggered D Flip-Flop

- It is also possible to design the circuit so that the flip-flop output changes on the positive edge of the clock.
- In sum, when the input clock in the positive-edge-triggered flip-flop makes a positive transition, the value of D is transferred to Q . A negative transition of the clock (i.e., from 1 to 0) does not affect the output.
- There is a minimum time called **the setup time** during which the D input must be maintained at a constant value prior to the occurrence of the clock transition
- minimum time called the **hold time** during which the D input must not change after the application of the positive transition of the clock.
- The **propagation delay time** of the flip-flop is defined as the interval between the trigger edge and the stabilization of the output to a new state.

Edge-Triggered D Flip-Flop

Parameter	Definition	Example Value	Illustration
Setup Time	Minimum time D must be stable before the clock edge	5 ns	If clock ↑ at 20 ns → D must be stable from 15 ns onwards
Hold Time	Minimum time D must remain stable after the clock edge	2 ns	If clock ↑ at 20 ns → D must remain unchanged until 22 ns
Propagation Delay	Time taken for output Q to update after the clock edge	8 ns	If clock ↑ at 20 ns → output Q updates at ~28 ns

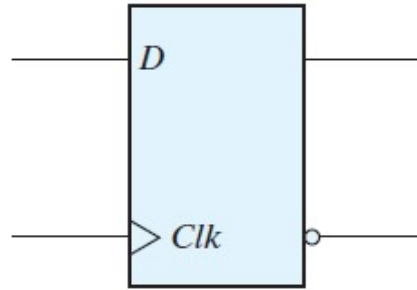
Real-Life Analogy

Think of it like catching a photo with a camera:

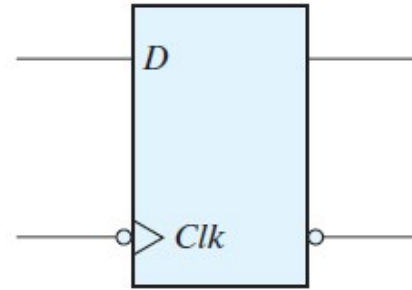
- **Setup time** → Subject must pose *before* the shutter clicks.
- **Hold time** → Subject must stay still *just after* the shutter clicks.
- **Propagation delay** → Time it takes for the photo to actually appear on screen after clicking.

Edge-Triggered D Flip-Flop

- Graphic symbol for edge-triggered D flip-flop:



(a) Positive-edge



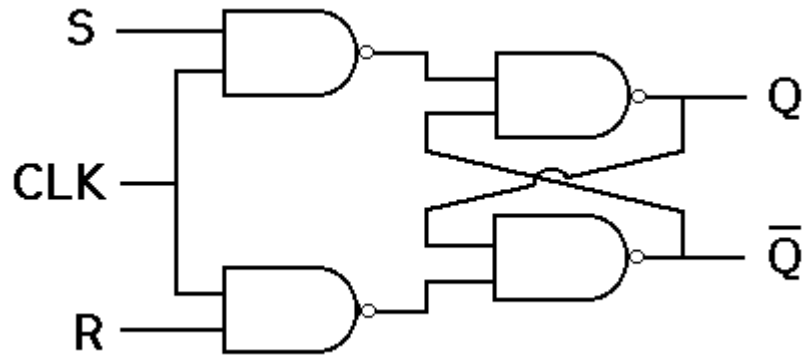
(a) Negative-edge

- The dynamic indicator ($>$) denotes the fact that the flip-flop responds to the edge transition of the clock.
- A bubble outside the block adjacent to the dynamic indicator designates a negative edge for triggering the circuit.
- The absence of a bubble designates a positive-edge response.

Flip-Flops have following representation:

- Function table- output Q and Q'
- Characteristic table- They define next state
- Characteristic equation-solve using K map- The logical properties of a flip-flop, as described in the characteristic table, can be expressed algebraically with a characteristic equation.
- Excitation table- predict the i/p based on current state and next state to build state diagram

SR Flip-Flops Characteristic table and Characteristic equation



<i>En</i>	<i>S</i>	<i>R</i>	Next state of <i>Q</i>
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$; reset state
1	1	0	$Q = 1$; set state
1	1	1	Indeterminate

(b) Function table

Characteristic table

<i>S</i>	<i>R</i>	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	-
1	1	1	-

Characteristic equation by simplifying using K maps
 $Q_{n+1} = S + R'Q_n$

Excitation Table

An excitation table shows the minimum inputs that are necessary to generate a particular next state when the current state is known.

The excitation tables are used to determine the inputs of the flip-flop when the present state and the next state to which the flip-flop goes after the occurrence of the clock pulse are known.

Excitation Table and State Diagram

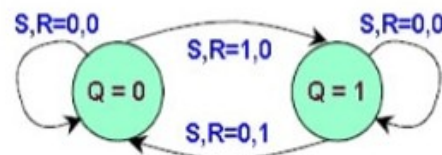
S	R	Present state Q_n	Next state Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

Truth table of SR flip flop

Invalid states

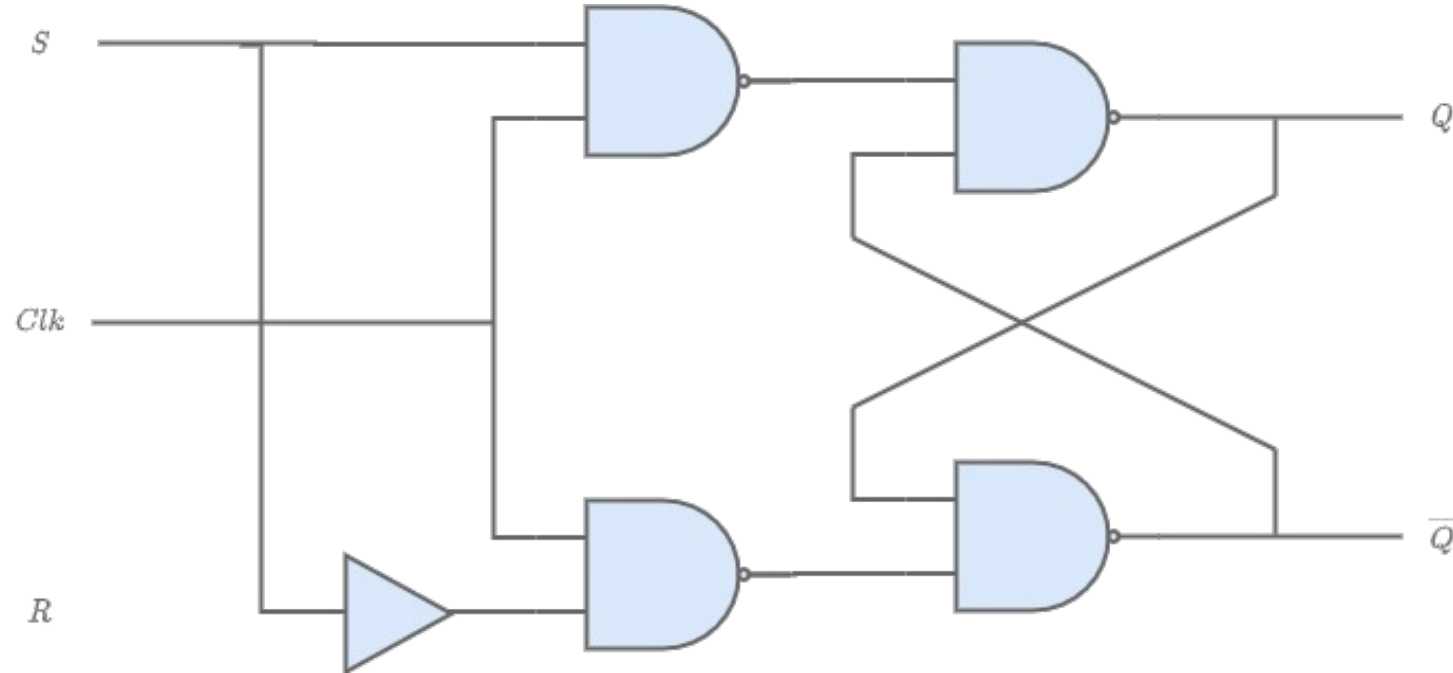
Q_n	Q_{n+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

Excitation table of SR flip flop

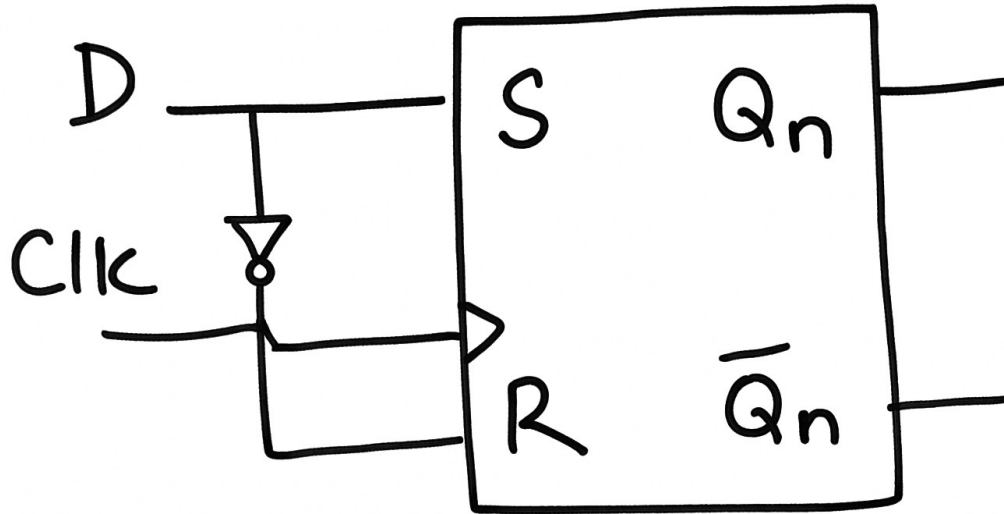


State Diagram

D Flip-Flop



D Flip-Flop



D Flip-Flop- Characterstic Table and Characterstic Equation



<i>En D</i>		Next state of <i>Q</i>
0	X	No change
1	0	$Q = 0$; reset state
1	1	$Q = 1$; set state

(b) Function table

D Flip-Flop

<i>D</i>	$Q(t + 1)$	
0	0	Reset
1	1	Set

D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

Simplifying using K maps- characteristic equation is

$$Q(t + 1) = D$$

D Flip-Flop- excitation table

Q_n	Q_(n+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

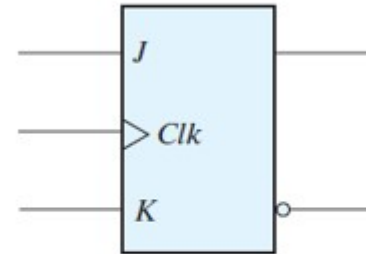
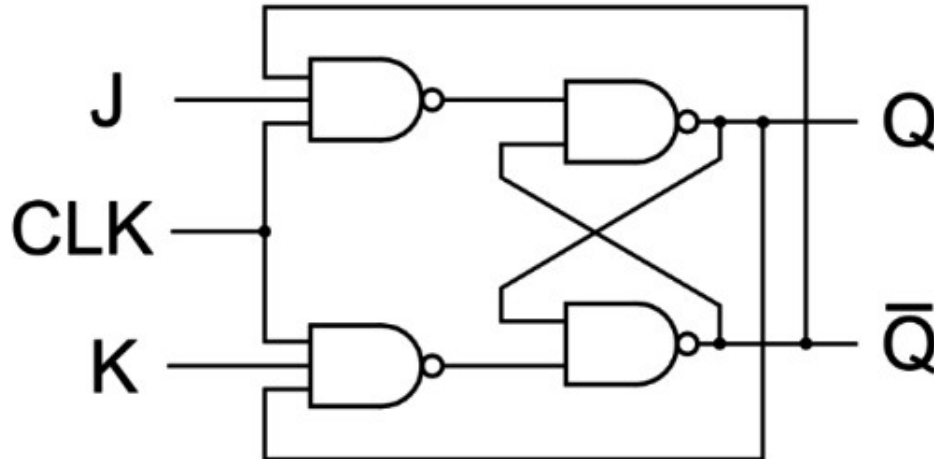
Excitation Table

State Diagram

JK Flip-Flop

A **JK flip-flop** is required mainly to overcome the limitations of the **SR flip-flop** and to provide a more versatile storage element in sequential circuits.

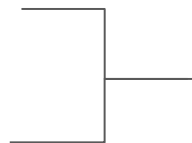
In an **SR flip-flop**, when **S=1** and **R=1** simultaneously, the output becomes indeterminate (invalid).



(b) Graphic symbol

JK Flip-Flop-function table

Clk	J	K	Q
0	x	x	Q _n
1	0	0	Q _n
1	0	1	0
1	1	0	1
1	1	1	Q_n



Memory

Toggle

race around condition

JK Flip-Flop

Characteristics table

Q(n)	J	K	Q(n+1)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Solve using K map to
get characteristic
equation:

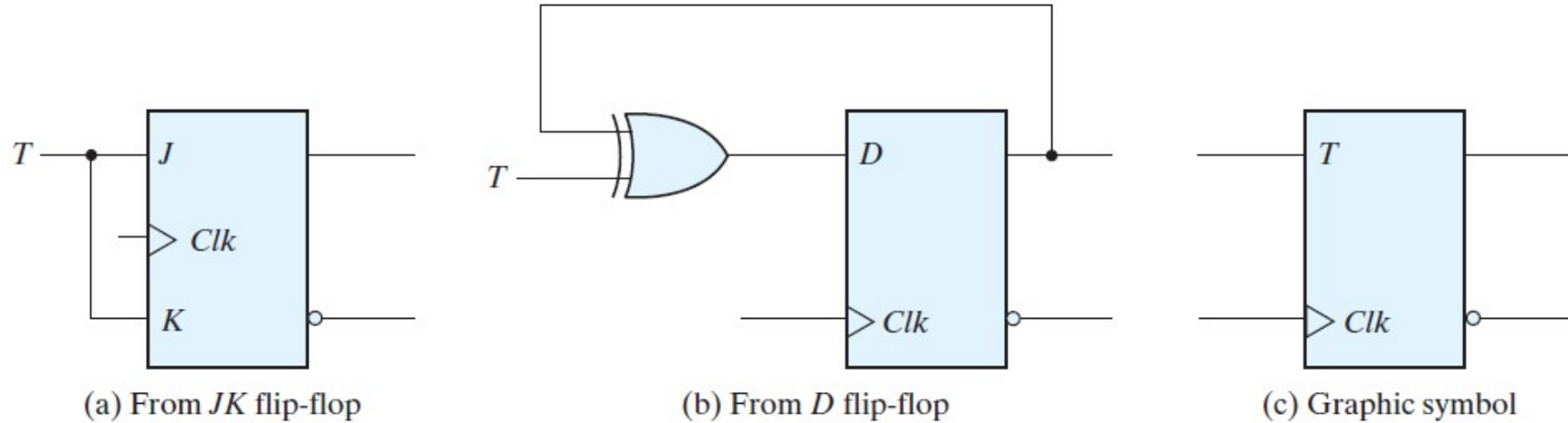
$$Q_{n+1} = Q_n K' + Q_n' J$$

JK Flip-Flop

Excitation table

Q(n)	Q(n+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

T Flip-Flop (Toggle Flip-Flop)



T Flip-Flop (Toggle Flip-Flop)

Truth table- if $\text{clk}=0$ then previous state

CLK	T	Q_{n+1}
↑	0	Q_n
↑	1	Q_n'

T Flip-Flop (Toggle Flip-Flop)

Characteristic table and equations:

T	Q _n	Q _{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

Excitation table

Q _n	Q _{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

$$Q(t + 1) = T \oplus Q = TQ' + T'Q$$

Flip-Flops

- JK flip-flop can function as:
- **SR flip-flop** ($J = S, K = R$)
- **T flip-flop** ($J=K=1 \rightarrow$ toggles)
- **D flip-flop** ($J=D, K=\neg D$)
- Hence, it's a **universal flip-flop** that can emulate other types.

Characteristics Table -summary

Flip-Flop Characteristic Tables

JK Flip-Flop

J	K	$Q(t + 1)$	
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$Q'(t)$	Complement

D Flip-Flop

D	$Q(t + 1)$	
0	0	Reset
1	1	Set

T Flip-Flop

T	$Q(t + 1)$	
0	$Q(t)$	No change
1	$Q'(t)$	Complement

Characteristics Equations-summary

The logical properties of a flip-flop, as described in the characteristic table, can be expressed algebraically with a characteristic equation. For the D flip-flop, we have the characteristic equation

$$Q(t + 1) = D$$

which states that the next state of the output will be equal to the value of input D in the present state. The characteristic equation for the JK flip-flop can be derived from the characteristic table or from the circuit of Fig. 5.12. We obtain

$$Q(t + 1) = JQ' + K'Q$$

where Q is the value of the flip-flop output prior to the application of a clock edge. The characteristic equation for the T flip-flop is obtained from the circuit of Fig. 5.13:

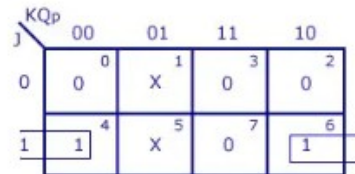
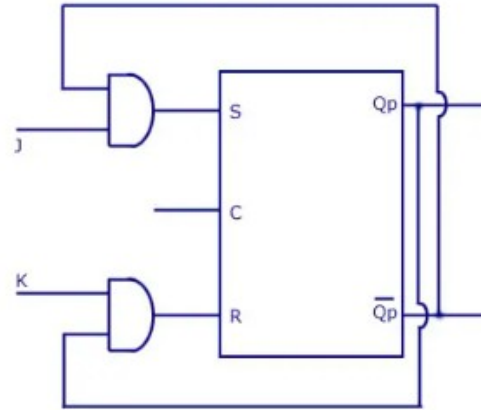
$$Q(t + 1) = T \oplus Q = TQ' + T'Q$$

SR Flip-Flop to JK Flip-Flop

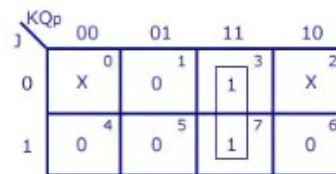
Conversion Table

J-K Inputs		Outputs		S-R Inputs	
J	K	Q_p	Q_{p+1}	S	R
0	0	0	0	0	X
0	0	1	1	X	0
0	1	0	0	0	X
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	X	0
1	1	0	1	1	0
1	1	1	0	0	1

Logic Diagram



$$S = \bar{J}Q_p$$



$$R = KQ_p$$

K-Map

<https://www.geeksforgeeks.org/digital-logic/flip-flop-types-their-conversion-and-applications/>

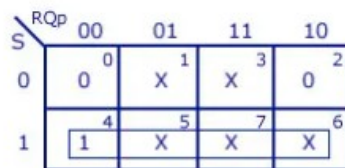
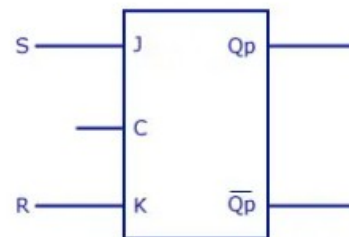
JK Flip-Flop to SR Flip-Flop

J-K Flip Flop to S-R Flip Flop

Conversion Table

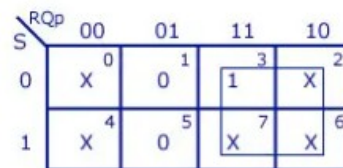
S-R Inputs		Outputs		J-K Inputs	
S	R	Q _p	Q _{p+1}	J	K
0	0	0	0	0	X
0	0	1	1	X	0
0	1	0	0	0	X
0	1	1	0	X	1
1	0	0	1	1	X
1	0	1	1	X	0
1	1	Invalid		Dont care	
1	1	Invalid		Dont care	

Logic Diagram



J=S

K-maps



K=R

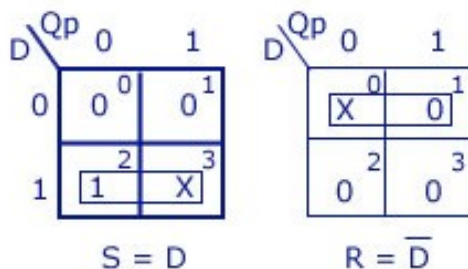
SR Flip-Flop to D Flip-Flop

S-R Flip Flop to D Flip Flop

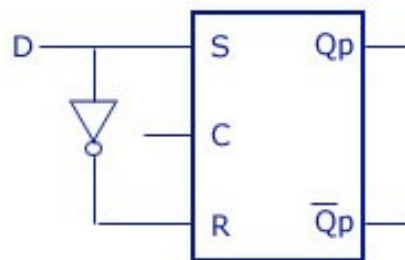
Conversion Table

D Input	Outputs		S-R Inputs	
	Q_p	Q_{p+1}	S	R
0	0	0	0	X
0	1	0	0	1
1	0	1	1	0
1	1	1	X	0

K-maps



Logic Diagram



D Flip-Flop to SR Flip-Flop

Conversion Table

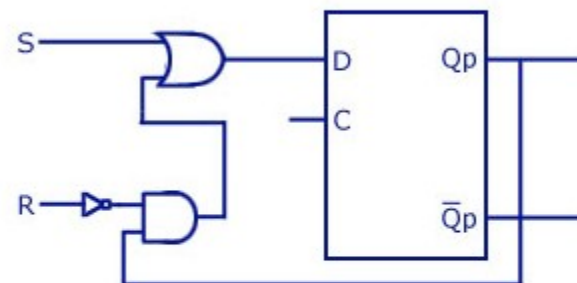
S-R Inputs		Outputs		D Input
S	R	Q _p	Q _{p+1}	
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	Invalid		Dont care
1	1	Invalid		Dont care

K-map

RQ _p		00	01	11	10
S	0	0	1	0	0
	1	1	1	X	X

$$D = S + \bar{R}Q_n$$

Logic Diagram



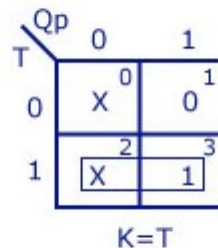
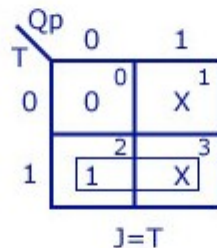
JK Flip-Flop to T Flip-Flop

J-K Flip Flop to T Flip Flop

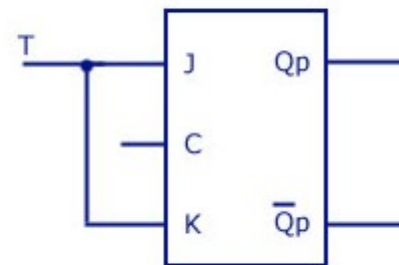
Conversion Table

T Input	Outputs		J-K Inputs	
	Q_p	Q_{p+1}	J	K
0	0	0	0	X
0	1	1	X	0
1	0	1	1	X
1	1	0	X	1

K-maps



Logic Diagram



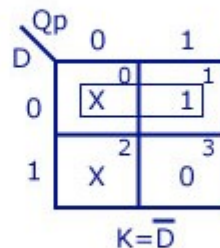
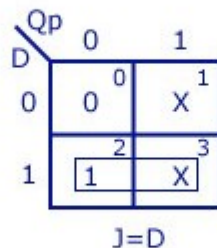
JK Flip-Flop to D Flip-Flop

J-K Flip Flop to D Flip Flop

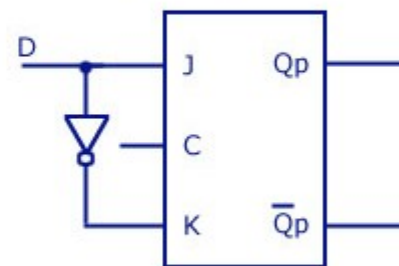
Conversion Table

D Input	Outputs		J-K Inputs	
	Q_p	Q_{p+1}	J	K
0	0	0	0	X
0	1	0	X	1
1	0	1	1	X
1	1	0	X	0

K-maps



Logic Diagram



D Flip-Flop to JK Flip-Flop

D Flip Flop to J-K Flip Flop

Conversion Table

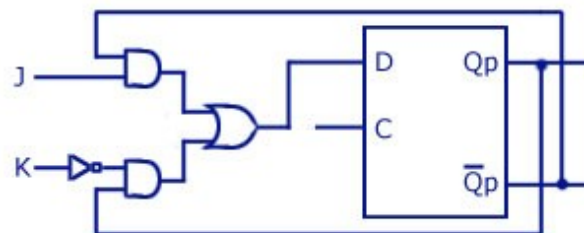
J-K Input		Outputs		D Input
J	K	Q _p	Q _{p+1}	
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

K-map

J \ KQ _p				
	00	01	11	10
1	0 ⁰	1 ¹	0 ³	0 ²
0	1 ⁴	1 ⁵	0 ⁷	1 ⁶

$$D = J\bar{Q}_p + \bar{K}Q_p$$

Logic Diagram



Applications of Flipflops and Latches



- **Electronic Voting Machine**
- **Scenario:** Registering and storing votes securely.
- **How flip-flops help:** Flip-flops store the vote data bit-by-bit, allowing fast, reliable vote counting.

Applications of Flipflops and Latches



Flip-Flop Type	Real-Time Example	How It Works in This Example	Why This Flip-Flop Is Suitable
JK Flip-Flop	Digital Watch Seconds Counter	Counts seconds by toggling state every clock pulse, progressing through time steps in binary.	JK flip-flop toggles easily on $J=K=1$, ideal for counters.
D Flip-Flop	Computer Register (CPU Storage)	Holds the current data/instruction being processed, updated only on clock edges for accuracy.	D flip-flop stores data bits reliably, synchronized to clock.
SR Flip-Flop	Elevator Door Control System	Sets door to open (set) or closed (reset) state based on user inputs or safety sensors.	Simple set/reset functionality matches door open/close signals.
T Flip-Flop	LED Blinking Circuit	Toggles LED ON and OFF on each clock pulse to create blinking effect.	T flip-flop toggles output state each pulse, perfect for blinking LEDs.

1. Which of the following statements about a latch is TRUE?

- A) A latch is edge-triggered.
- B) A latch is level-sensitive.
- C) A latch can only store analog values.
- D) A latch cannot hold its state without a clock signal.

2. In an SR latch constructed with NOR gates, what is the output when both S and R inputs are 0?

- A) Output is set ($Q=1$)
- B) Output is reset ($Q=0$)
- C) Output retains previous state
- D) Output is indeterminate/invalid

1. Which of the following statements about a latch is TRUE?

- A) A latch is edge-triggered.
- B) A latch is level-sensitive.
- C) A latch can only store analog values.
- D) A latch cannot hold its state without a clock signal.

Answer: B

2. In an SR latch constructed with NOR gates, what is the output when both S and R inputs are 0?

- A) Output is set ($Q=1$)
- B) Output is reset ($Q=0$)
- C) Output retains previous state
- D) Output is indeterminate/invalid

Answer: C

3. In a JK flip-flop, when both J and K inputs are 1 and a clock pulse occurs, the output:

- A) Is set to 1
- B) Is reset to 0
- C) Toggles from its previous state
- D) Remains unchanged

4. A D flip-flop can be constructed using:

- A) Two SR latches connected in master-slave configuration.
- B) Only combinational logic
- C) Only NOR gates without feedback.
- D) A JK flip-flop with both J and K tied to the D input.

3. In a JK flip-flop, when both J and K inputs are 1 and a clock pulse occurs, the output:

- A) Is set to 1
- B) Is reset to 0
- C) Toggles from its previous state
- D) Remains unchanged

Answer: C

4. A D flip-flop can be constructed using:

- A) Two SR latches connected in master-slave configuration.
- B) Only combinational logic
- C) Only NOR gates without feedback.
- D) A JK flip-flop with both J and K tied to the D input.

Answer: A



THANK YOU

Team DDCO

Department of Computer Science and Engineering



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Analysis of Clocked Seq. Circuits

Team DDCO

Department of Computer Science and Engineering

Introduction(section 5.5 T1)



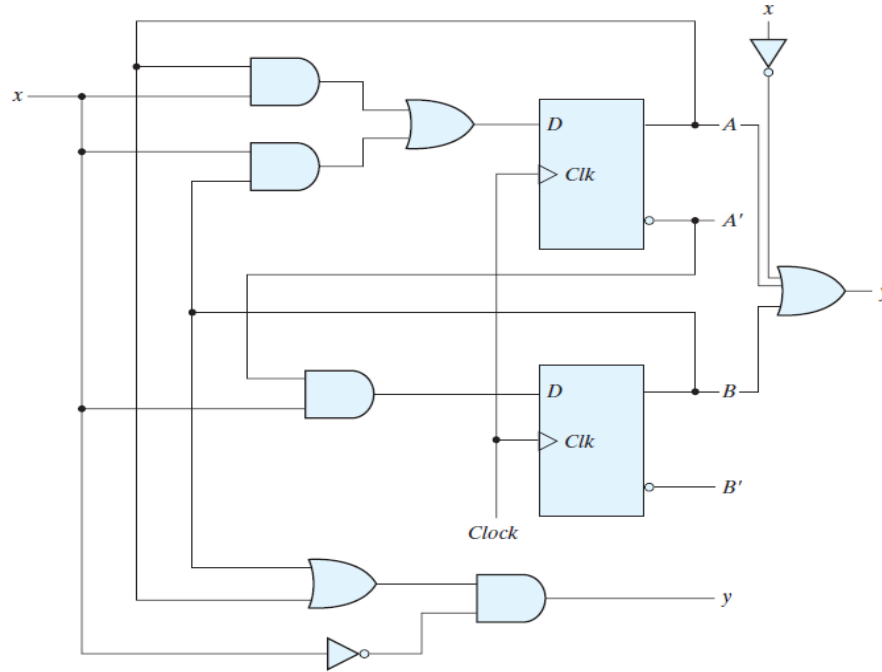
- Analysis describes what a given circuit will do under certain operating conditions.
- The **behavior** of a clocked sequential circuit is determined from **the inputs, the outputs, and the state of its flip-flops**.
- The outputs and the next state are both a function of the inputs and the present state.
- The **analysis of a sequential circuit** consists of obtaining **a table or a diagram** for the **time sequence of inputs, outputs, and internal states**.
- It is also possible to write Boolean expressions that describe the behavior of the sequential circuit.

- These expressions must include the necessary time sequence, either directly or indirectly.
- A logic diagram is recognized as a clocked sequential circuit if it includes flip-flops with clock inputs.
- The flip-flops may be of any type, and the logic diagram may or may not include combinational logic gates.
- **Algebraic representation for specifying the next-state condition in terms of the present state and inputs**

State Equations

- The **behavior** of a clocked sequential circuit can be described algebraically by means of **state equations**.
- A *state equation* (also called a *transition equation*) specifies the **next state as a function of the present state and inputs**.
- Consider the sequential circuit shown in the Figure.

State Equations



Circuit acts as a 0-detector by asserting its output when a 0 is detected in a stream of 1s

Behavior of circuit:

Imagine you're monitoring a signal line that should be continuously 1 (high).

- As long as the signal is 1, the detector stays quiet.
- When a glitch (a single 0) happens, the detector alerts with $y = 1$.
- But if the line stays 0, it doesn't keep alerting — only the first error is flagged.

State Equations

- It consists of two D flip-flops A and B, an input x and an output y .
- Since the D input of a flip-flop determines the value of the next state (i.e., the state reached after the clock transition), it is possible to write a set of state equations for the circuit:
 - $A(t + 1) = A(t)x(t) + B(t)x(t)$
 - $B(t + 1) = A'(t)x(t)$

State Equations

- The Boolean expressions for the state equations can be derived directly from the gates that form the combinational circuit part of the sequential circuit, since the D values of the combinational circuit determine the next state.
- Similarly, the present-state value of the output can be expressed algebraically as
 - $y(t) = [A(t) + B(t)]x'(t)$
- By removing the symbol (t) for the present state, we obtain the output Boolean equation:
 - $y = (A + B)x'$

State Table

- The time sequence of **inputs, outputs, and flip-flop states** can be enumerated in a *state table* (sometimes called a *transition table*).
- The state table for the previous circuit is shown in the table.
- The table consists of 4 sections: **present state, input, next state, and output** .
- The present-state section shows the states of flip-flops A and B at any given time t .
- The input section gives a value of x for each possible present state.
- The next-state section shows the states of the flip-flops one clock cycle later, at time $t + 1$.
- The output section gives the value of y at time t for each present state and input condition.

State Table- Type 1

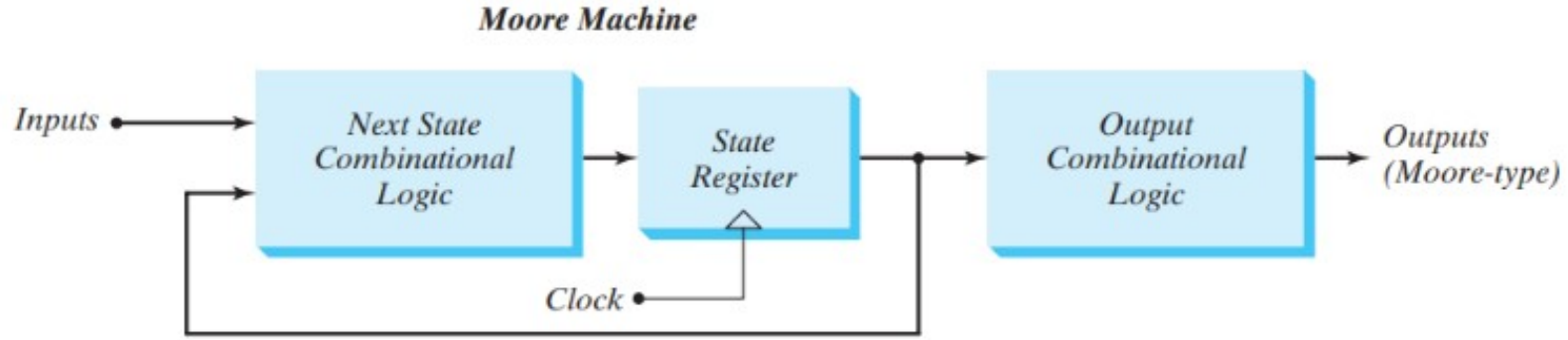
Present State		Input	Next State		Output
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

$$A(t + 1) = Ax + Bx$$

$$B(t + 1) = A'x$$

$$y = Ax' + Bx'$$

Moore Models of Finite State Machines



- In the Moore model, the output is a function of only the present state
- Next state is function of present state and input

State Table

- The state table of a sequential circuit with D -type flip-flops is obtained by the same procedure outlined in the previous example.
- In general, a sequential circuit with m flip-flops and n inputs needs 2^{m+n} rows in the state table.
- The binary numbers from 0 through $2^{m+n} - 1$ are listed under the present-state and input columns.
- The next-state section has m columns, one for each flip-flop.
- The binary values for the next state are derived directly from the state equations.
- The output section has as many columns as there are output variables.
- Its binary value is derived from the circuit or from the Boolean function in the same manner as in a truth table.

State Table

- It is sometimes convenient to express the state table in a slightly different form having only three sections: present state, next state, and output.
- The input conditions are enumerated under the next-state and output sections.
- The new state table is as shown in this second form.
- For each present state, there are two possible next states and outputs, depending on the value of the input.

State Table – Type 2

Second Form of the State Table

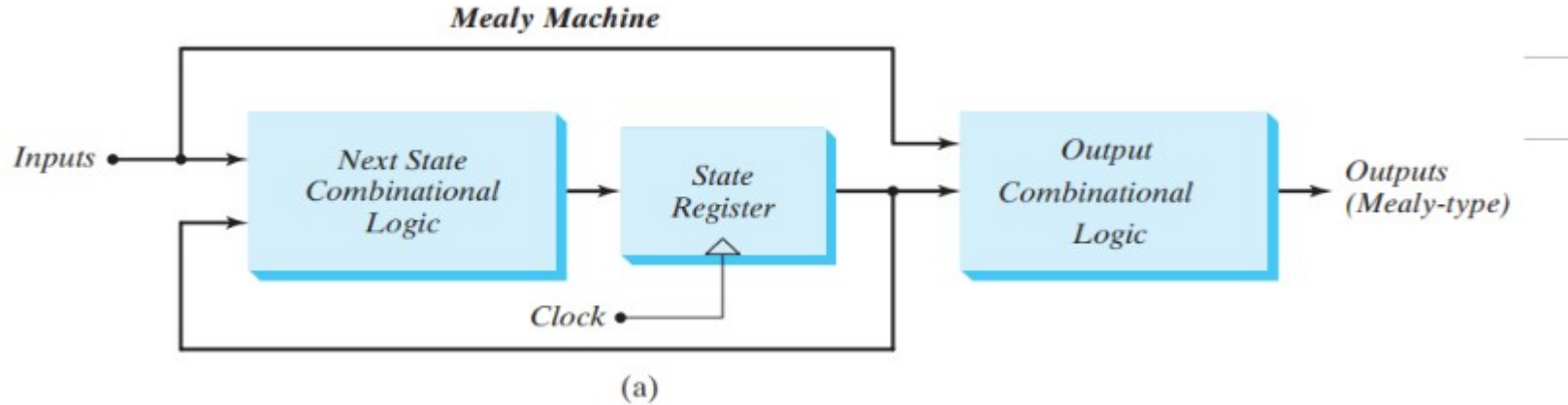
Present State		Next State				Output	
		$x = 0$		$x = 1$		$x = 0$	$x = 1$
A	B	A	B	A	B	y	y
0	0	0	0	0	1	0	0
0	1	0	0	1	1	1	0
1	0	0	0	1	0	1	0
1	1	0	0	1	0	1	0

$$A(t + 1) = Ax + Bx$$

$$B(t + 1) = A'x$$

$$y = Ax' + Bx'$$

Mealy Models of Finite State

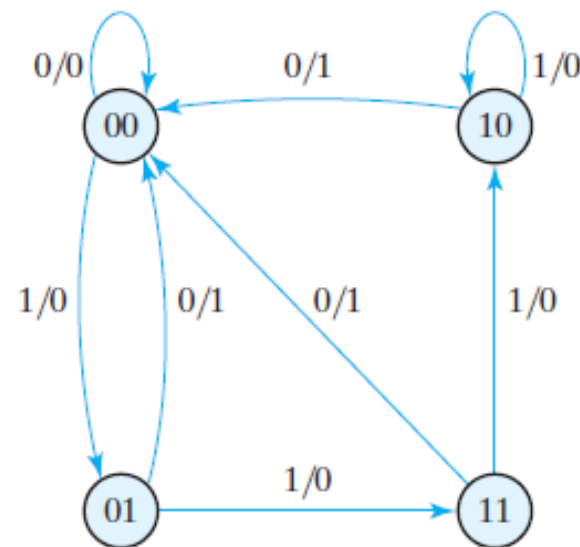


In the Mealy model, the output is a function of both the present state and the input.
Next state depends on present state and inputs

State Diagram

Second Form of the State Table

Present State		Next State				Output	
		$x = 0$		$x = 1$		$x = 0$	$x = 1$
A	B	A	B	A	B	y	y
0	0	0	0	0	1	0	0
0	1	0	0	1	1	1	0
1	0	0	0	1	0	1	0
1	1	0	0	1	0	1	0



Circuit diagram → Equations – State table → State diagram

State Diagram

- The state diagram provides the same information as the state table and is obtained directly from table. **The information available in a state table can be represented graphically in the form of a state diagram**
- The binary number inside each circle identifies the state of the flip-flops.
- The directed lines are labelled with two binary numbers separated by a slash.
- The input value during the present state is labelled first, and the number after the slash gives the output during the present state with the given input.
- The steps presented in this example are summarized below:

Circuit diagram → Equations – State table → State diagram

The state diagram gives a pictorial view of state transitions and is the form more suitable for human interpretation of the circuit's operation

Flip-Flop Input Equations

- The part of the combinational circuit that generates external outputs is described algebraically by a set of Boolean functions **called output equations**.
- The part of the circuit that generates the inputs to flip-flops is described algebraically by a set of Boolean functions called flip-flop **input equations** (or, sometimes, excitation equations).

$$D_A = Ax + Bx$$

$$D_B = A'x$$

$$y = (A + B)x'$$

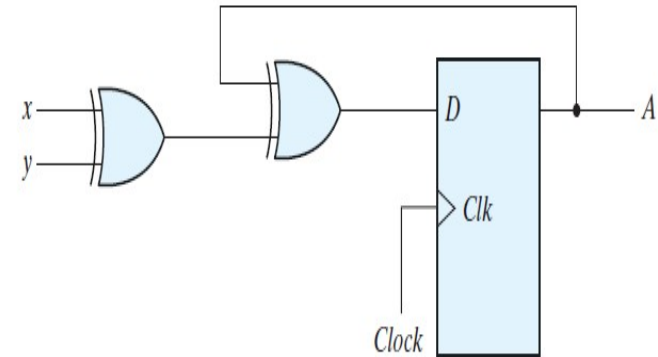
Note that the expression for the input equation for a D flip-flop is identical to the expression for the corresponding state equation. This is because of the characteristic equation that equates the next state to the value of the D input: $Q(t + 1) = D_Q$.

Analysis with D Flip-Flops

- The procedure for analyzing a clocked sequential circuit with D flip-flops by means of a simple example. The circuit we want to analyze is described by the input equation:

$$D_A = A \oplus x \oplus y$$

- The D_A symbol implies a D flip-flop with output A.
- The x and y variables are the inputs to the circuit.
- **No output equations are given, which implies that the output comes from the output of the flip-flop.**



(a) Circuit diagram

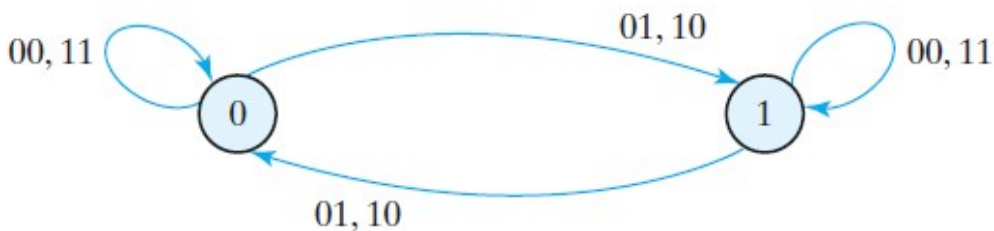
Analysis with D Flip-Flops

- The state table has one column for the present state of flip-flop A , two columns for the two inputs, and one column for the next state of A .
- The binary numbers under Axy are listed from 000 through 111 as shown in figure (b).
- The next-state values are obtained from the state equation
$$A(t + 1) = A \oplus x \oplus y$$
- The expression specifies an odd function and is equal to 1 when only one variable is 1 or when all three variables are 1.
- This is indicated in the column for the next state of A .

Analysis with D Flip-Flops

Present state	Inputs		Next state
A	x	y	A
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(b) State table



(c) State diagram

Analysis with D Flip-Flops

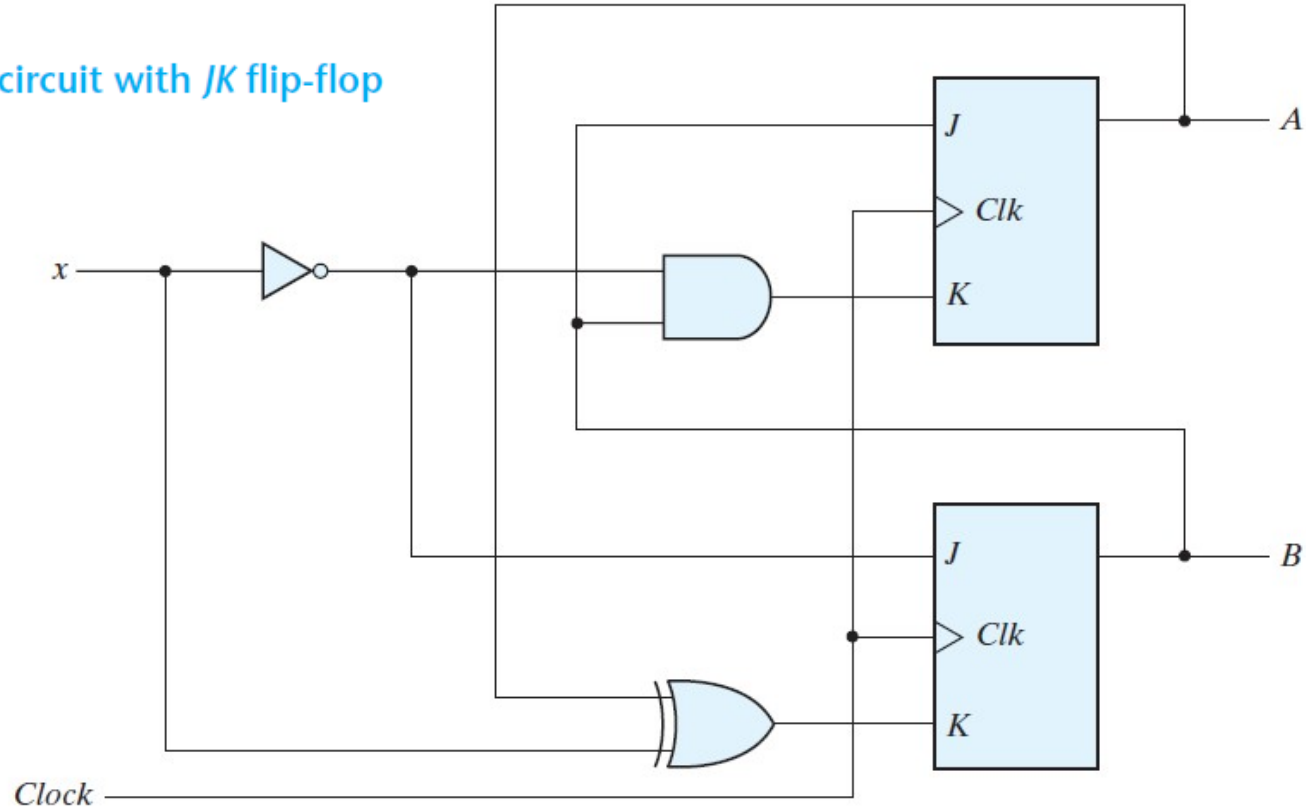
- **The circuit has one flip-flop and two states.**
- The state diagram consists of two circles, one for each state as shown in figure (c).
- **The present state and the output can be either 0 or 1, as indicated by the number inside the circles.(moore model)**
- **A slash on the directed lines is not needed, because there is no output from a combinational circuit.**
- The two inputs can have four possible combinations for each state.
- Two input combinations during each state transition are separated by a comma to simplify the notation.

Analysis of JK Flip-Flop

- When a flip–flop other than the D type is used, such as JK or T , it is necessary to refer to the corresponding characteristic table or characteristic equation to obtain the next–state values.
- The procedure is illustrated first by using the characteristic table and again by using the characteristic equation.
- The next–state values of a sequential circuit that uses JK – or T –type flip–flops can be derived as follows:
 - a. Determine the flip–flop input equations in terms of the present state and input variables.
 - b. List the binary values of each input equation.
 - c. Use the corresponding flip–flop characteristic table to determine the next–state values in the state table.

Analysis of JK Flip-Flop

Sequential circuit with JK flip-flop



Analysis of JK Flip-Flop

- As an example, consider the sequential circuit with two *JK* flip-flops *A* and *B* and one input *x*.
- The circuit has no outputs; therefore, the state table does not need an output column.
- The circuit can be specified by the flip-flop input equations

$$J_A = B \quad K_A = Bx'$$

$$J_B = x' \quad K_B = A'x + Ax' = A \oplus x$$

[illegible]

Present State		Input	Next State		Flip-Flop Inputs			
A	B	x	A	B	J_A	K_A	J_B	K_B
0	0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	1
0	1	0	1	1	1	1	1	0
0	1	1	1	0	1	0	0	1
1	0	0	1	1	0	0	1	1
1	0	1	1	0	0	0	0	0
1	1	0	0	0	1	1	1	1
1	1	1	1	1	1	0	0	0

Analysis of JK Flip-Flop

The next-state values can also be obtained by evaluating the state equations from the characteristic equation. This is done by using the following procedure:

1. Determine the flip-flop input equations in terms of the present state and input variables.
2. Substitute the input equations into the flip-flop characteristic equation to obtain the state equations.
3. Use the corresponding state equations to determine the next-state values in the state table.

Analysis of JK Flip-Flop

The input equations for the two JK flip-flops of Fig. 5.18 were listed a couple of paragraphs ago. The characteristic equations for the flip-flops are obtained by substituting A or B for the name of the flip-flop, instead of Q :

$$A(t + 1) = JA' + K'A$$

$$B(t + 1) = JB' + K'B$$

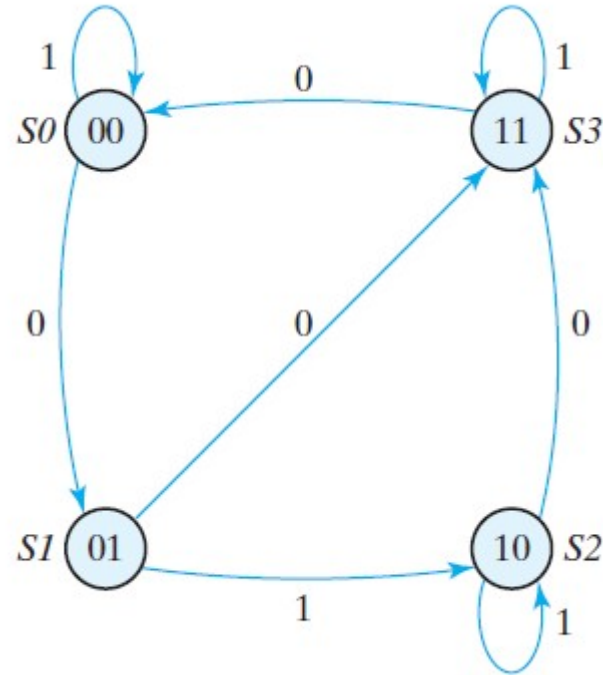
Substituting the values of J_A and K_A from the input equations, we obtain the state equation for A :

$$A(t + 1) = BA' + (Bx')'A = A'B + AB' + Ax$$

The state equation provides the bit values for the column headed “Next State” for A in the state table. Similarly, the state equation for flip-flop B can be derived from the characteristic equation by substituting the values of J_B and K_B :

$$B(t + 1) = x'B' + (A \oplus x)'B = B'x' + ABx + A'Bx'$$

Analysis of JK Flip-Flop



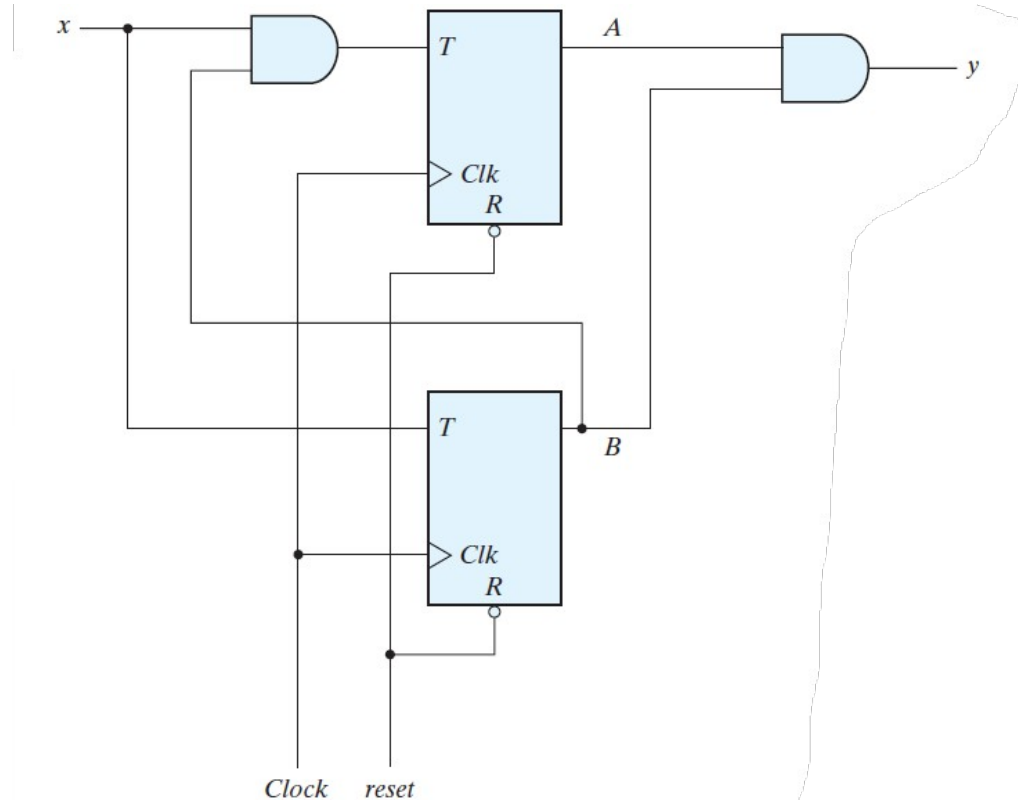
State diagram of the circuit

Analysis with T Flip-Flops

- The analysis of a sequential circuit with T flip-flops follows the same procedure.
- The next-state values in the state table can be obtained by using either using the characteristic table or characteristic equation.

$$Q(t + 1) = T \oplus Q = T'Q + TQ'$$

Analysis with T Flip-Flops



(a) Circuit diagram

Analysis with T Flip-Flops

It has two flip-flops A and B, one input x, and one output y and can be described algebraically by two input equations and an output equation:

$$T_A = Bx$$

$$T_B = x$$

$$y = AB$$

Analysis with T Flip-Flops

State Table for Sequential Circuit with T Flip-Flops

Present State		Input	Next State		Output
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	1

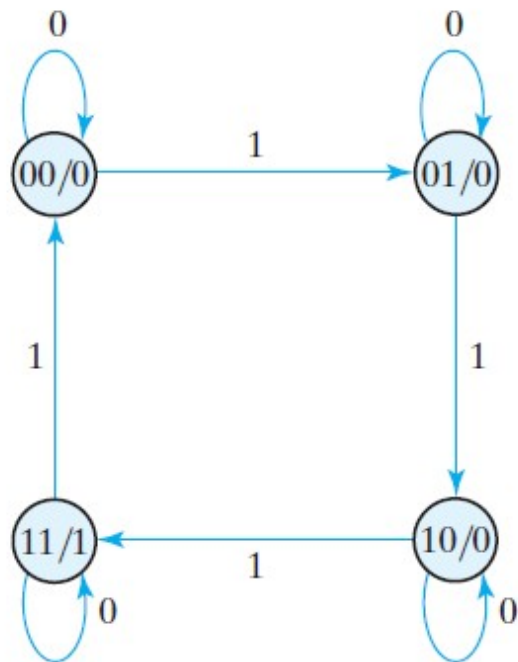
Analysis with T Flip-Flops

The values for y are obtained from the output equation. The values for the next state can be derived from the state equations by substituting T_A and T_B in the characteristic equations, yielding

$$A(t+1) = (Bx)'A + (Bx)A' = AB' + Ax' + A'Bx$$

$$B(t+1) = x \oplus B$$

Analysis with T Flip-Flops



(b) State diagram

As long as input x is equal to 1, the circuit behaves as a binary counter with a sequence of states 00, 01, 10, 11, and back to 00.

When $x = 0$, the circuit remains in the same state. Output y is equal to 1 when the present state is 11.

Here, the output depends on the present state only and is independent of the input.

The two values inside each circle and separated by a slash are for the present state and output.

Applications

Elevator Controller (Multi-floor Building)

- **Description:**

The elevator logic determines movement between floors, door operation, and priority handling.

- **How it relates to your concept:**

- States: Floor positions + Door open/close status.
- Inputs: Floor request buttons, sensors.
- Flip-flops: Store current floor/door state;
- **state equations** decide next movement.

Traffic Light Controller

- **Description:**

A four-way traffic signal is controlled by a **finite state machine (FSM)** implemented using flip-flops.

- **How it relates to your concept:**

- **States:** Red → Green → Yellow → Red.
- **Flip-flops:** Store the current light state.
- **State table & equations:** Determine which light turns on next based on current state and timer pulse.

Applications



Digital Lock System

Description:

Unlocks only when a correct sequence of digits/buttons is entered.

How it relates to your concept:

States: Each entered digit represents a state transition.

Inputs: Keypad inputs.

Flip-flops: Store entered sequence progress.

State equations: Verify input sequence and transition to “Unlock” state if correct.

1. The present state of a D flip-flop is 1. If the input equation is $D=A \oplus x \oplus y$ where A is the current flip-flop output, $x=1$ and $y=0$, what will be the next state after the clock edge?
A) 0
B) 1
C) Cannot be determined
D) Remains same
2. A sequential circuit uses two D flip-flops A and B, and one input x. The next state equations are:
 $A(t+1)=A \cdot x + B \cdot x$
 $B(t+1)=A' \cdot x$
If present state is $A=1, B=0, x=1$, what is the next state?
A) (1,0)
B) (1,1)
C) (0,1)
D) (0,0)

1. The present state of a D flip-flop is 1. If the input equation is $D=A \oplus x \oplus yD$ where A is the current flip-flop output, $x=1$, and $y=0$, what will be the next state after the clock edge?

A) 0
B) 1
C) Cannot be determined
D) Remains same

Answer: A — $D=1 \oplus 1 \oplus 0$, so next state is 0.

2. A sequential circuit uses two D flip-flops A and B, and one input x. The next state equations are:

$$A(t+1)=A \cdot x + B \cdot x$$

$$B(t+1)=A' \cdot x$$

If present state is $A=1, B=0, x=1$, what is the next state?

A) (1,0)
B) (1,1)
C) (0,1)
D) (0,0)

Answer: A — Direct substitution into the state equations.



THANK YOU

Team DDCO

Department of Computer Science and Engineering



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Mealy and Moore Models of FSMs

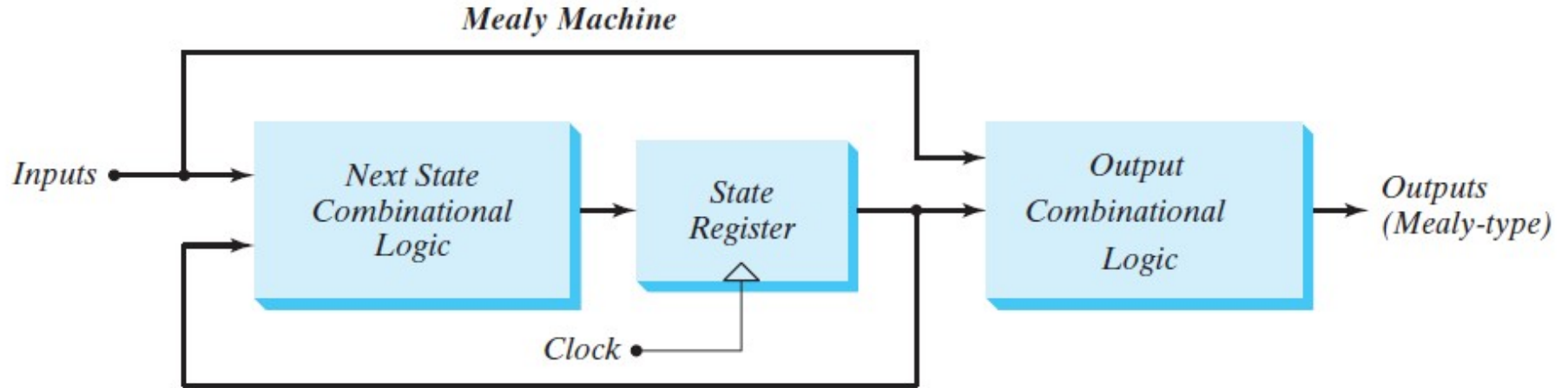
Team DDCO

Department of Computer Science and Engineering

Introduction(T1-section 5.5)

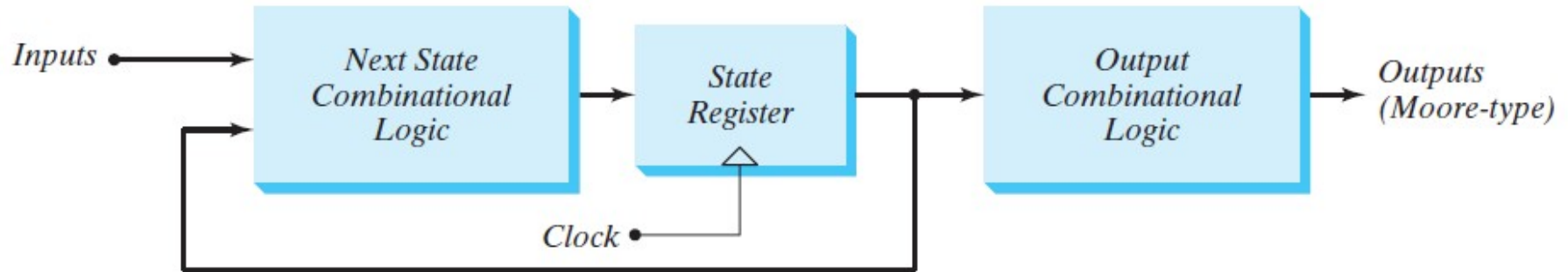
- The most general model of a sequential circuit has inputs, outputs, and internal states.
- It is customary to distinguish between two models of sequential circuits: the Mealy model and the Moore model.
- They differ only in the way the output is generated.

Block Diagrams



Block Diagrams

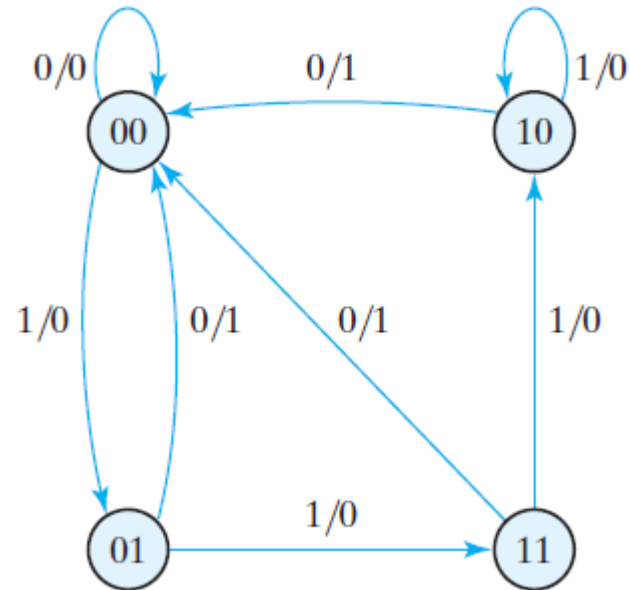
Moore Machine



- In the Mealy model, the output is a function of both the present state and the input.
- In the Moore model, the output is a function of only the present state.
- A circuit may have both types of outputs.
- The two models of a sequential circuit are commonly referred to as a **finite state machine**, abbreviated FSM.
- The Mealy model of a sequential circuit is referred to as a Mealy FSM or Mealy machine.
- The Moore model is referred to as a Moore FSM or Moore machine.

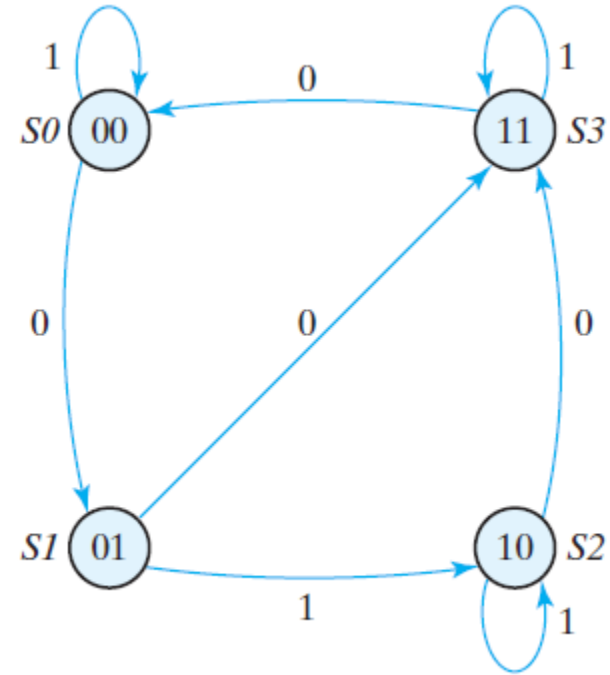
Mealy Machine

- Output y is a function of both input x and the present state of A and B .
- The corresponding state diagram shows both the input and output values, separated by a slash along the directed lines between the states.



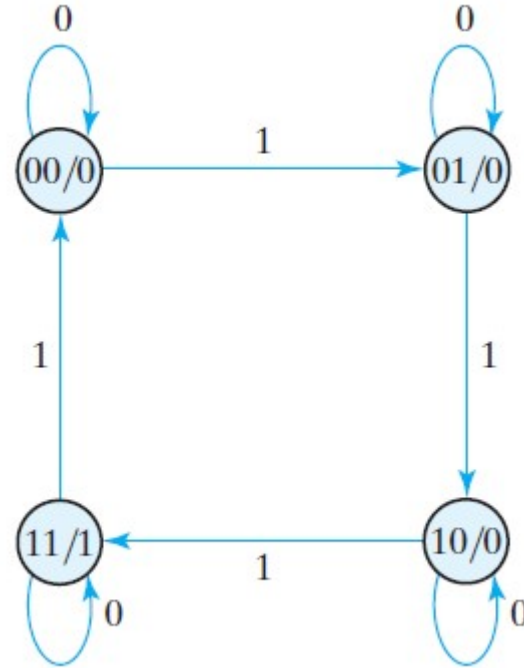
Moore Machine

- Here, the output is a function of the present state only.
- The corresponding state diagram has only inputs marked along the directed lines.
- The outputs are the flip-flop states marked inside the circles.



Moore Machine

- The output depends only on flip-flop values, and that makes it a function of the present state only.
- The input value in the state diagram is labeled along the directed line, but the output value is indicated inside the circle together with the present state.



Points to Remember

- **In a Moore model, the outputs of the sequential circuit are synchronized with the clock, because they depend only on flip-flop outputs that are synchronized with the clock.** In a Mealy model, the outputs may change if the inputs change during the clock cycle.
- In order to synchronize a Mealy-type circuit, the inputs of the sequential circuit must be synchronized with the clock and the outputs must be sampled immediately before the clock edge.
- The inputs are changed at the inactive edge of the clock to ensure that the inputs to the flip-flops stabilize before the active edge of the clock occurs.
- **Thus, the output of the Mealy machine is the value that is present immediately before the active edge of the clock.**

Mealy Model

Vending Machine with Coin Detection

- **How it works:** As soon as the correct coin is inserted (input) while the machine is in the “waiting” state, it immediately unlocks the product selection buttons (output).
- **Why Mealy:** Output depends on both **current state** (“waiting”) and **instant input** (coin detected) → faster interaction.

Moore Model

- **Example: Washing Machine Cycle**
- **How it works:** Once the washer enters the “spin” state, it spins for a fixed duration regardless of what buttons are pressed during that time.
- **Why Moore:** Output (motor spinning) depends only on **current state**, ensuring stability and avoiding mid-cycle changes

Moore:

- Needs separate states for “output=0” and “output=1.”
- More states.
- **Mealy:**
 - Can stay in one state and just make the output depend on x.
 - Fewer states.

Consider a Mealy machine and a Moore machine implemented for the same problem. Which of the following is true?

- A) Moore machine always has fewer states.
- B) Mealy machine may have fewer states.
- C) Both have the same number of states always.
- D) Moore machine always produces output one clock earlier than Mealy.

Consider a Mealy machine and a Moore machine implemented for the same problem. Which of the following is true?

- A) Moore machine always has fewer states.
- B) Mealy machine may have fewer states.
- C) Both have the same number of states always.
- D) Moore machine always produces output one clock earlier than Mealy.

Answer: B — Mealy can have fewer states due to immediate output change.

In a Mealy machine, the output depends on:

- A) Present state only
- B) Present state and next state
- C) Present state and input
- D) Input only

The main difference between a Moore and a Mealy machine is that:

- A) Moore's output depends only on input.
- B) Mealy's output depends only on state.
- C) Moore's output depends only on state, while Mealy's output depends on state and input.
- D) Mealy's output is always faster than Moore's output, regardless of design.

In a Mealy machine, the output depends on:

- A) Present state only
- B) Present state and next state
- C) Present state and input
- D) Input only

Answer: C — Present state and input.

The main difference between a Moore and a Mealy machine is that:

- A) Moore's output depends only on input.
- B) Mealy's output depends only on state.
- C) Moore's output depends only on state, while Mealy's output depends on state and input.
- D) Mealy's output is always faster than Moore's output, regardless of design.

Answer: C — Moore → State only; Mealy → State + Input



THANK YOU

Team DDCO

Department of Computer Science and Engineering



DIGITAL DESIGN AND COMPUTER ORGANIZATION

**State Reduction and Assignment
Design**

Team DDCO

Department of Computer Science and Engineering

State Reduction(T1- section 5.7)



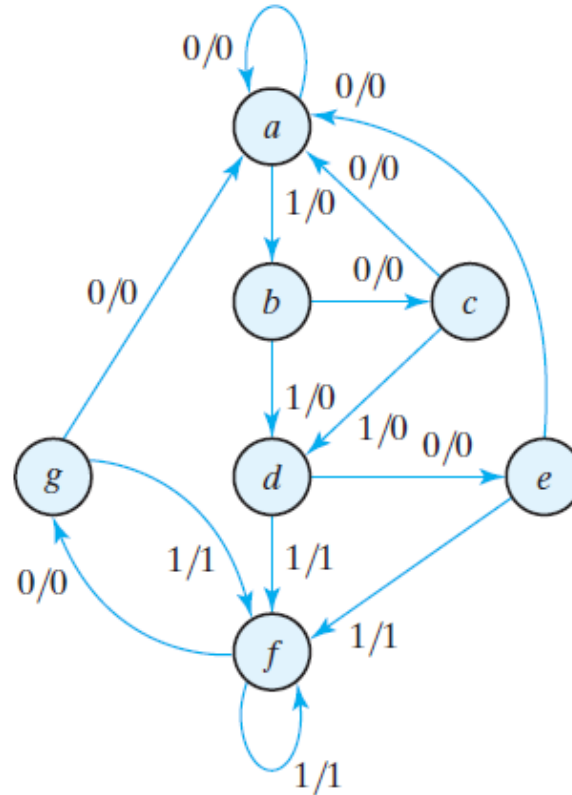
- The **analysis** of sequential circuits starts from a circuit diagram and culminates in a state table or diagram.
- The **design** (synthesis) of a sequential circuit starts from a set of specifications and culminates in a logic diagram. (section 5.8)
- Two sequential circuits may exhibit the same input–output behavior, but have a different number of internal states in their state diagram.

State Reduction(T1- section 5.7)

- Properties of sequential circuits that may **simplify a design by reducing the number of gates and flip-flops it uses**. In general, reducing the number of flip flops reduces the **cost** of a circuit.
- **The reduction in the number of flip-flops in a sequential circuit is referred to as the state-reduction problem**
- Since m flip-flops produce 2^m states, a reduction in the number of states may (or may not) result in a reduction in the number of flip-flops.
- Each state can be encoded with binary numbers for 2 flipflops
 - A=00
 - B=01
 - C=10
 - D=11

State Reduction

State diagram



As an example, consider the input sequence 01010110100 starting from the initial state a .

State Reduction

- As an example, consider the input sequence 01010110100 starting from the initial state *a* .
- Each input of 0 or 1 produces an output of 0 or 1 and causes the circuit to go to the next state.

state	<i>a</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>f</i>	<i>g</i>	<i>f</i>	<i>g</i>	<i>a</i>
input	0	1	0	1	0	1	1	0	1	0	0	
output	0	0	0	0	0	1	1	0	1	0	0	

State Reduction



Two states are said to be equivalent if, for each member of the set of inputs, they give exactly the same output and send the circuit either to the same state or to an equivalent state.

When two states are equivalent, one of them can be removed without altering the input–output relationships.

State Reduction

Table 5.6
State Table

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
<i>a</i>	<i>a</i>	<i>b</i>	0	0
<i>b</i>	<i>c</i>	<i>d</i>	0	0
<i>c</i>	<i>a</i>	<i>d</i>	0	0
<i>d</i>	<i>e</i>	<i>f</i>	0	1
<i>e</i>	<i>a</i>	<i>f</i>	0	1
<i>f</i>	<i>g</i>	<i>f</i>	0	1
<i>g</i>	<i>a</i>	<i>f</i>	0	1

State Reduction

- States e and g are two such states: They both go to states a and f and have outputs of 0 and 1 for $x=0$ and $x=1$, respectively. Therefore, states g and e are equivalent, and one of these states can be removed.
- states f and d are equivalent, and state f can be removed and replaced by d .

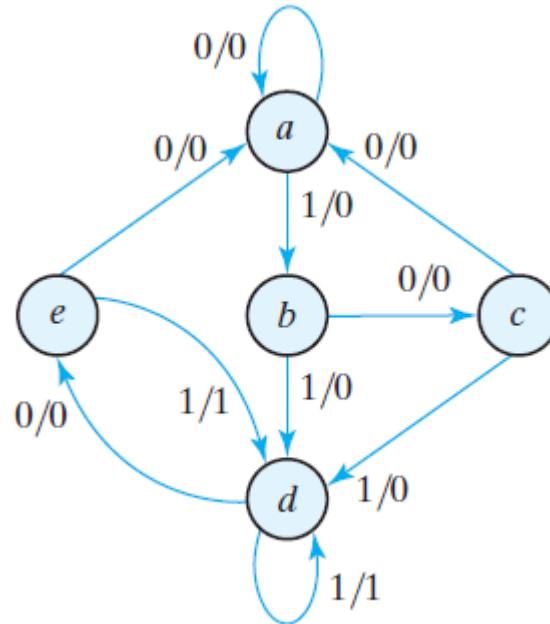
Table 5.7
Reducing the State Table

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
<i>a</i>	<i>a</i>	<i>b</i>	0	0
<i>b</i>	<i>c</i>	<i>d</i>	0	0
<i>c</i>	<i>a</i>	<i>d</i>	0	0
<i>d</i>	<i>e</i>	<i>f</i>	0	1
<i>e</i>	<i>a</i>	<i>f</i>	0	1
<i>f</i>	<i>e</i>	<i>f</i>	0	1

Reduced State Table

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
<i>a</i>	<i>a</i>	<i>b</i>	0	0
<i>b</i>	<i>c</i>	<i>d</i>	0	0
<i>c</i>	<i>a</i>	<i>d</i>	0	0
<i>d</i>	<i>e</i>	<i>d</i>	0	1
<i>e</i>	<i>a</i>	<i>d</i>	0	1

State Reduction



Reduced state diagram

- The sequential circuit of this example was reduced from seven to five states.
- In general, reducing the number of states in a state table may result in a circuit with **less equipment**.
- However, **the fact that a state table has been reduced to fewer states does not guarantee a saving in the number of flip-flops or the number of gates.**
- In actual practice designers may skip this step because target devices are rich in resources.

State Assignment

- For a circuit with m states, the codes must contain n bits, where $2^n \geq m$.
- For example, with three bits, it is possible to assign codes to eight states, denoted by binary numbers 000 through 111.
- Example: 7 states are there
 - 000 to 111
 - one state can be unused.
 - In truth table mentioned as don't care condition

State Assignment

Three Possible Binary State Assignments

State	Assignment 1, Binary	Assignment 2, Gray Code	Assignment 3, One-Hot
<i>a</i>	000	000	00001
<i>b</i>	001	001	00010
<i>c</i>	010	011	00100
<i>d</i>	011	010	01000
<i>e</i>	100	110	10000

- Binary= $2^n = m$ -The simplest way to code five states is to use the first five integers in binary counting order
- Gray code = adjacent bit differ by one bit
- On hot encoding-t. At any given time, only one bit is equal to 1 while all others are kept at 0. This type of assignment uses **one flipflop per state**.

State Assignment

- One-hot encoding usually leads to simpler decoding logic for the next state and output
- One-hot machines can be **faster** than machines with sequential binary encoding, and the silicon area required by the **extra flip-flops** can be offset by the area saved by using simpler decoding logic.
- This **trade-off** is not guaranteed, so it must be evaluated for a given design.

State Assignment

Reduced State Table with Binary Assignment 1

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
000	000	001	0	0
001	010	011	0	0
010	000	011	0	0
011	100	011	0	1
100	000	011	0	1

Reduced State Table

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
<i>a</i>	<i>a</i>	<i>b</i>	0	0
<i>b</i>	<i>c</i>	<i>d</i>	0	0
<i>c</i>	<i>a</i>	<i>d</i>	0	0
<i>d</i>	<i>e</i>	<i>d</i>	0	1
<i>e</i>	<i>a</i>	<i>d</i>	0	1

Also referred as transition table

A different assignment will result in a state table with different binary values for the states.

Design Procedure(T1 –section 5.8)



- Design procedures or methodologies **specify hardware** that will implement a desired behavior
- The sequential building block used by synthesis tools is the D flip-flop. Together with additional logic, it can implement the behavior of JK and T flip-flops.
- In fact, designers **generally do not concern themselves with the type of flip-flop; rather, their focus is on correctly describing the sequential functionality that is to be implemented by the synthesis tool**
- In contrast to a combinational circuit, which is fully specified by a truth table, a sequential circuit requires a state table for its specification.

Design Procedure(T1 –section 5.8)



- A synchronous sequential circuit is made up of flip-flops and combinational gates.
- The number of flip-flops is determined from the number of states needed in the circuit and the choice of state assignment codes.
- The combinational circuit is derived from the state table

Design Procedure

The procedure for designing synchronous sequential circuits can be summarized by a list of recommended steps:

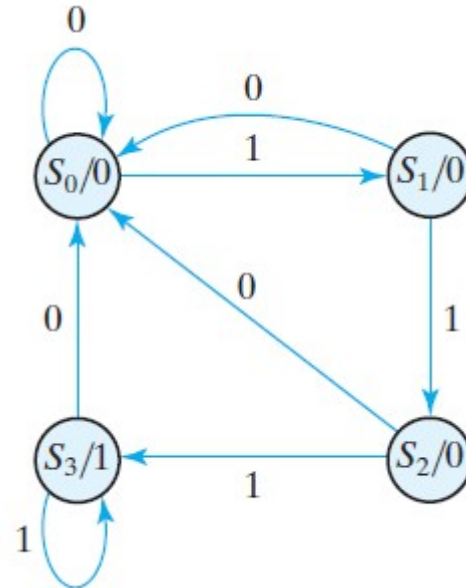
1. From the word description and specifications of the desired operation, derive a state diagram for the circuit.
2. Reduce the number of states if necessary.
3. Assign binary values to the states.
4. Obtain the binary-coded state table.
5. Choose the type of flip-flops to be used.
6. Derive the simplified flip-flop input equations and output equations.
7. Draw the logic diagram.

Design Procedure

Suppose we wish to design a circuit that detects a sequence of **three or more consecutive 1's** in a string of bits coming through an input line (i.e., the input is a serial bit stream).

S0= reset state

Below shown is moore model



Design Procedure



If the input is 0, the circuit stays in S_0 , but if the input is 1, it goes to state S_1 to indicate that a 1 was detected. If the next input is 1, the change is to state S_2 to indicate the arrival of two consecutive 1's, but if the input is 0, the state goes back to S_0 . The third consecutive 1 sends the circuit to state S_3 . If more 1's are detected, the circuit stays in S_3 . Any 0 input sends the circuit back to S_0 . In this way, the circuit stays in S_3 as long as there are three or more consecutive 1's received.

This is a Moore model sequential circuit, since the output is 1 when the circuit is in state S_3 and is 0 otherwise.

Synthesis Using D Flip-Flops



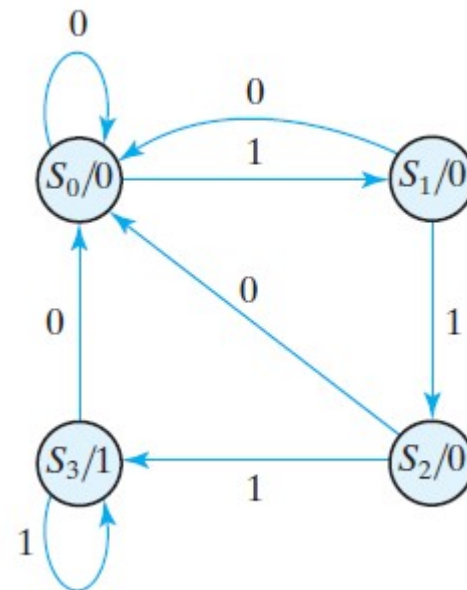
- We choose two D flip-flops to represent the four states, and we label their outputs A and B. There is one input x and one output y.
- The characteristic equation of the D flip-flop is $Q(t+1)=DQ$, which means that the next-state values in the state table specify the D input condition for the flip-flop.

State Table for Sequence Detector. – BINARY ENCODED STATE TABLE
S0=00, S1=01, S2=10 s3=11

Synthesis Using D Flip-Flops

State Table for Sequence Detector

Present State		Input x	Next State		Output y
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1



Synthesis Using D Flip-Flops

K-Maps for sequence detector

$$A(t + 1) = D_A(A, B, x) = \Sigma(3, 5, 7)$$

$$B(t + 1) = D_B(A, B, x) = \Sigma(1, 5, 7)$$

$$y(A, B, x) = \Sigma(6, 7)$$

		Bx			
		B			
A	Bx	00	01	11	10
		m_0	m_1	m_3	m_2
0				1	
1		m_4	m_5	m_7	m_6
			1	1	

$$D_A = Ax + Bx$$

		Bx			
		B			
A	Bx	00	01	11	10
		m_0	m_1	m_3	m_2
0			1		
1		m_4	m_5	m_7	m_6
			1	1	

$$D_B = Ax + B'x$$

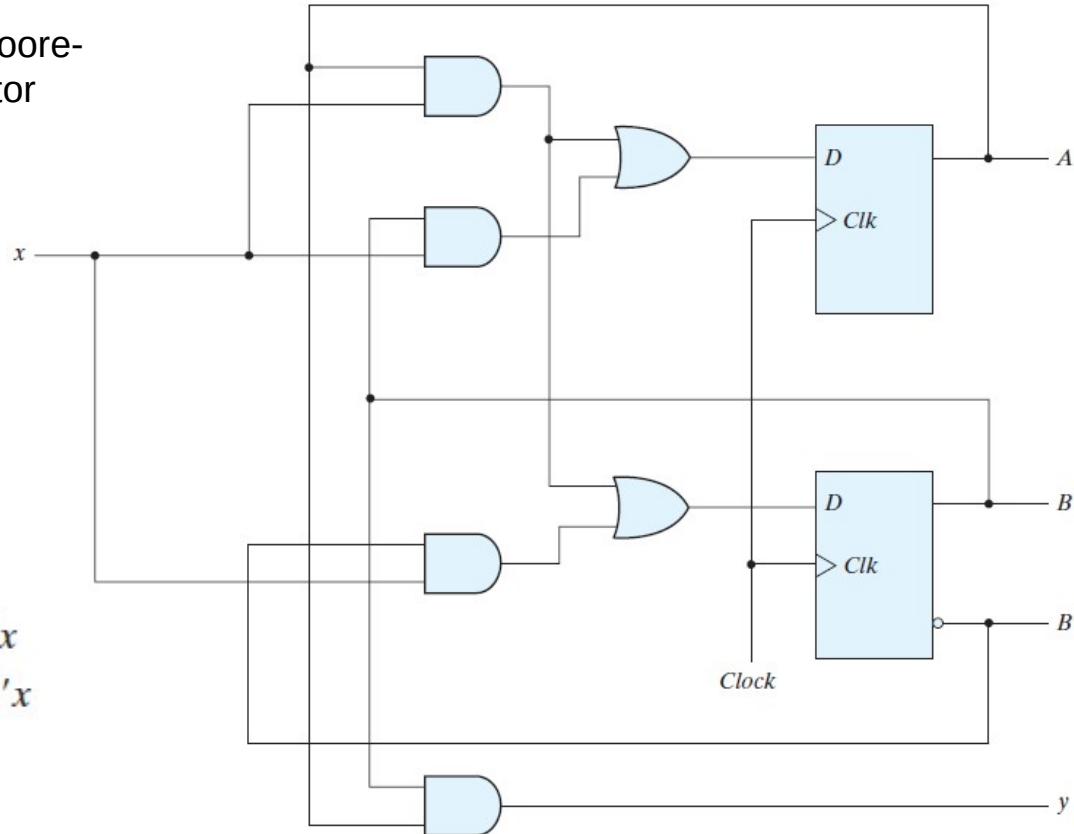
		Bx			
		B			
A	Bx	00	01	11	10
		m_0	m_1	m_3	m_2
0					
1		m_4	m_5	m_7	m_6
				1	1

$$y = AB$$

Synthesis Using D Flip-Flops

Logic diagram of a Moore-type sequence detector

$$\begin{aligned}D_A &= Ax + Bx \\D_B &= Ax + B'x \\y &= AB\end{aligned}$$



Excitation Table

- When D -type flip-flops are employed, the input equations are obtained directly from the next state. This is not the case for the JK and T types of flip-flops.
- In order to determine the input equations for these flip-flops, it is necessary to derive a functional relationship between the state table and the input equations.

Flip-Flop Excitation Tables

$Q(t)$	$Q(t = 1)$	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

(a) JK Flip-Flop

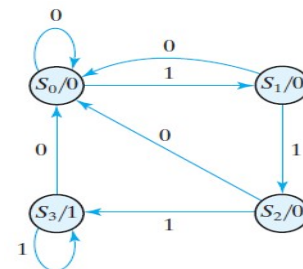
$Q(t)$	$Q(t = 1)$	T
0	0	0
0	1	1
1	0	1
1	1	0

(b) T Flip-Flop

Synthesis Using JK Flip-Flops

State Table and JK Flip-Flop Inputs

Present State		Input	Next State		Flip-Flop Inputs			
A	B		A	B	J_A	K_A	J_B	K_B
0	0	0	0	0	0	X	0	X
0	0	1	0	1	0	X	1	X
0	1	0	1	0	1	X	X	1
0	1	1	0	1	0	X	X	0
1	0	0	1	0	X	0	0	X
1	0	1	1	1	X	0	1	X
1	1	0	1	1	X	0	X	0
1	1	1	0	0	X	1	X	1



$Q(t)$	$Q(t = 1)$	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

(a) JK Flip-Flop

Excitation Table

Synthesis Using JK Flip-Flops

		B			
		Bx	00	01	11 10
A	0	m_0	m_1	m_3	m_2 1
	1	m_4	m_5	m_7	m_6 X
		x			

$J_A = Bx'$

		B			
		Bx	00	01	11 10
A	0	m_0	X	X	m_3 X m_2 X
	1	m_4			m_7 1 m_6
		x			

$K_A = Bx$

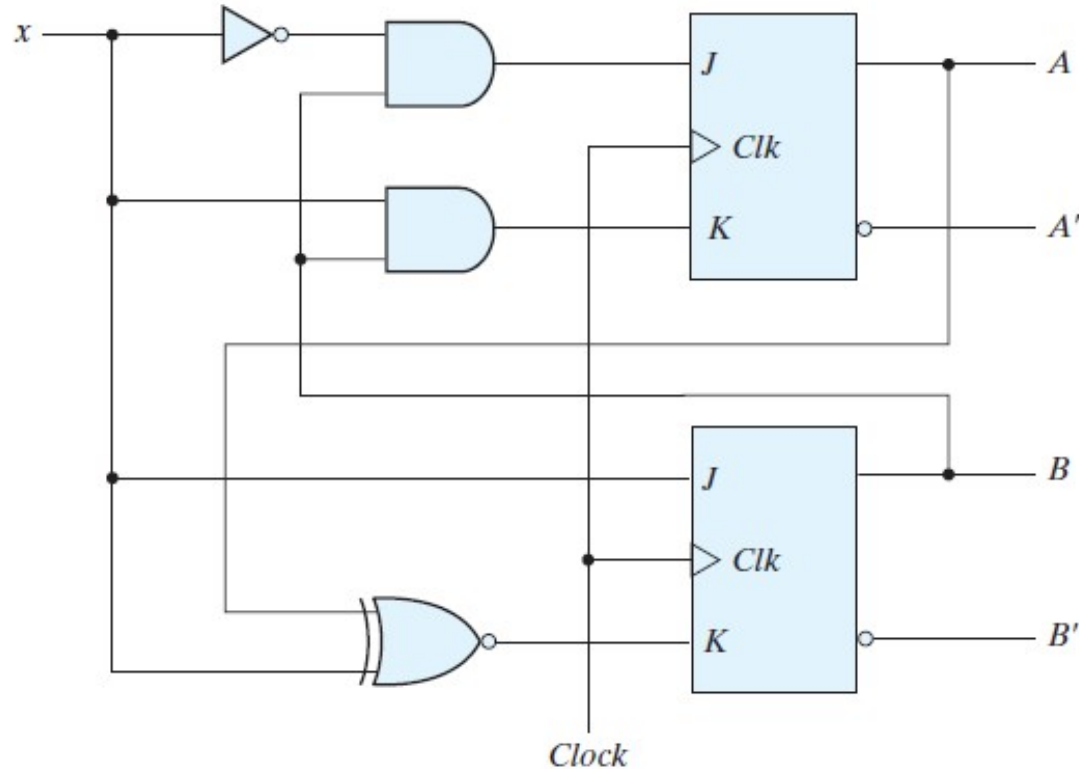
		B			
		Bx	00	01	11 10
A	0	m_0		m_1 1	m_3 X m_2 X
	1	m_4		m_5 1	m_7 X m_6 X
		x			

$J_B = x$

		B			
		Bx	00	01	11
A	0	m_0 X	m_1 X	m_3	m_2 1
	1	m_4 X	m_5 X	m_7 1	m_6
		x			

$K_B = (A \oplus x)'$

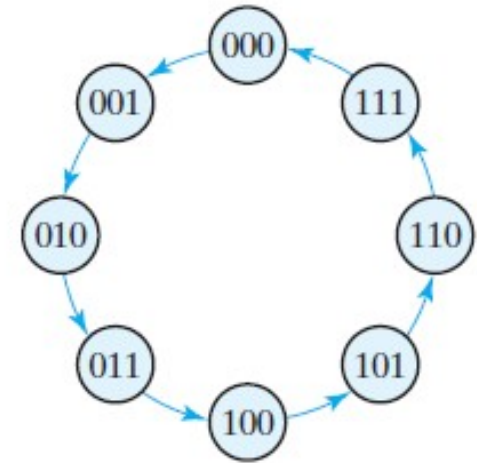
Synthesis Using JK Flip-Flops



Logic diagram for sequential circuit with JK flip-flops

Synthesis Using T Flip-Flops

- The procedure for synthesizing circuits using T flip-flops will be demonstrated by designing a binary counter.
- *An n -bit binary counter consists of n flip-flops that can count in binary from 0 to $2^n - 1$*
- the state diagram of a counter does not have to show input and output values along the directed lines.
- The only input to the circuit is the clock, and the outputs are specified by the present state of the flip-flops.
- The next state of a counter depends entirely on its present state, and the state transition occurs every time the clock goes through a transition.



State diagram of three-bit binary counter.

Synthesis Using T Flip-Flops

State Table for Three-Bit Counter

Present State			Next State			Flip-Flop Inputs		
A_2	A_1	A_0	A_2	A_1	A_0	T_{A2}	T_{A1}	T_{A0}
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	1	1	1	0	1	1
1	1	1	0	0	0	1	1	1

$Q(t)$	$Q(t = 1)$	T
0	0	0
0	1	1
1	0	1
1	1	0

(b) T Flip-Flop

Excitation Table

Synthesis Using T Flip-Flops

		A_1A_0			
		00	01	11	10
A_2	0	m_0	m_1	m_3 1	m_2
	1	m_4	m_5	m_7 1	m_6

$T_{A2} = A_1A_0$

		A_1A_0			
		00	01	11	10
A_2	0	m_0	m_1 1	m_3 1	m_2
	1	m_4	m_5 1	m_7 1	m_6

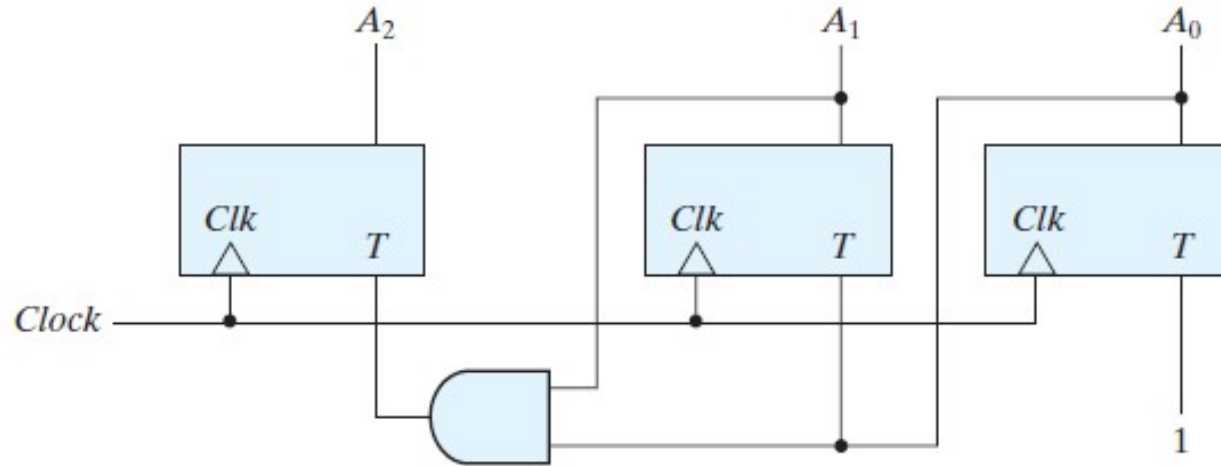
$T_{A1} = A_0$

		A_1A_0			
		00	01	11	10
A_2	0	m_0 1	m_1 1	m_3 1	m_2 1
	1	m_4 1	m_5 1	m_7 1	m_6 1

$T_{A0} = 1$

Maps for three-bit binary counter

Synthesis Using T Flip-Flops



Logic diagram of three-bit binary counter

Additional questions



Design the circuit diagram for mealy state diagram using D flipflop with binary encoding

Steps: write state table, use binary encodings, build transition table , derive equations for next state and output

Think about it: Design the circuit diagram for mealy state diagram using D flipflop with one hot encoding. One-hot encoding for 8 states → needs 8 flip-flops. Binary encoding for 8 states → needs only 3 flip-flops.

Additional questions

Present state	Input X	Next state	output
S0	0	S1	0
S0	1	S0	0
S1	0	S1	0
S1	1	S0	1

AFTER USING BINARY ENCODING- $S_0=0$ AND $S_1=1$

Present state	Input X	Next state	output
0	0	1	0
0	1	0	0
1	0	1	0
1	1	0	1

Additional questions

AFTER USING BINARY ENCODING- $S_0=0$ AND $S_1=1$

Present state A	Input X	Next state A	Output Y
0	0	1	0
0	1	0	0
1	0	1	0
1	1	0	1

Solving using K map next state equation $D_A=x'$ and $y=Ax$

Additional questions



Design the circuit diagram for Moore state diagram using D flipflop with Gray code encoding and one hot encoding

Summary

Analysis of circuit:

Circuit diagram $\rightarrow$ Equations – State table $\rightarrow$ State diagram

The state table can of 2 format: format 1: output depends on present state(Moore model)

format 2: Output depends on present state and input (Mealey Model)

Both cases Next state always depends on present state and input

For JK and T flipflop to build next state in table use characteristic equation

Design of circuit : From state diagram build Circuit

Steps:(Moore and Mealey model):

State diagram– check for equivalent states– use encoding(binary/gray code/one hot encoding) and build table---select type of flip flop--- get equations for next state and output- build circuit (number of flipflops to be decided on type of encoding)

For JK and T flipflop

Use excitation tables inputs

Summary

	Goal	Real-Time Benefit	Example
State Reduction	Minimize number of equivalent states	Lower logic complexity → faster response	Sequence detector in network packet filtering
One-Hot Assignment	One bit per state	Faster decoding logic, suitable for FPGA designs	High-speed motor control
Gray Code	Adjacent states differ by 1 bit	Ensures reliable state transitions (only 1-bit change).	Rotary encoder position tracking-As the shaft rotates, only one bit changes at a time
Binary Assignment	Compact encoding	Since fewer bits are used, fewer flip-flops are needed to store the state, reduces hardware cost	Embedded communication controller

A finite state machine (FSM) designed using **one-hot encoding** for 5 states will require:

- A) 3 flip-flops
- B) 4 flip-flops
- C) 5 flip-flops
- D) 6 flip-flops

A sequential circuit is reduced from 8 states to 5 states after **state reduction**. If implemented with binary state assignment, the change in minimum flip-flops required is:

- A) From 4 to 3
- B) From 3 to 2
- C) From 4 to 2
- D) No change

A finite state machine (FSM) designed using **one-hot encoding** for 5 states will require:

- A) 3 flip-flops
- B) 4 flip-flops
- C) 5 flip-flops
- D) 6 flip-flops

Answer: C

Explanation: In one-hot encoding, **one flip-flop per state** → 5 states → 5 flip-flops.

A sequential circuit is reduced from 8 states to 5 states after **state reduction**. If implemented with binary state assignment, the change in minimum flip-flops required is:

- A) From 4 to 3
- B) From 3 to 2
- C) From 4 to 2
- D) No change

Answer: D

Explanation:

$$N = 8 \lceil \log_2(8) \rceil = \lceil 3 \rceil = 3 \text{ flip-flops}$$

$$\lceil \log_2(5) \rceil = \lceil 2.32 \rceil = 3 \text{ flip-flops}$$



THANK YOU

Team DDCO

Department of Computer Science and Engineering



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Registers

Team DDCO

Department of Computer Science and Engineering

Registers(T1- section 6.1 and 6.2)

Introduction:

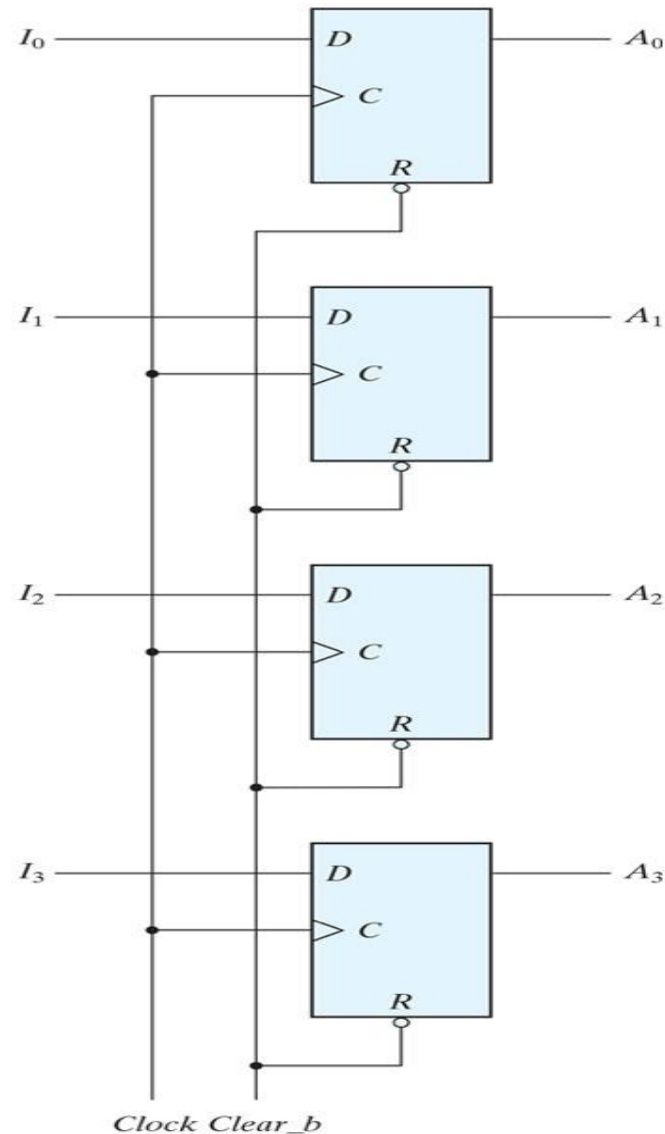
- A register is a **group of flip-flops**, each one of which shares a common clock and is capable of storing one bit of information.
- An n -bit register consists of a group of n flip-flops capable of **storing n bits of binary information**.
- In addition to the flip-flops, **a register may have combinational gates** that perform certain data-processing tasks.
- In its broadest definition, a register consists of a group of flip-flops together with gates that affect their operation.
- The **flip-flops hold the binary information, and the gates determine how the information is transferred into the register**.

Registers

- A counter is essentially a register that goes through a predetermined sequence of binary states.
- The gates in the counter are connected in such a way as to produce the prescribed sequence of states.
- The simplest register is one that consists of only flip-flops, without any gates
- Figure 6.1 shows such a register constructed with four D-type flip-flops to form a four-bit data storage register. **The common clock input triggers all flip-flops on the positive edge of each pulse**, and the binary data available at the four inputs are transferred into the register. The value of (I3 , I2 , I1 , I0) immediately before the clock edge determines the value of (A3 , A2 , A1 , A0) after the clock edge.
- The input **Clear_b** goes to the active-low R (reset) input of all four flip-flops. When this input goes to 0, all flip-flops are reset asynchronously. The Clear_b input is useful for clearing the register to all 0's prior to its clocked operation. **The R inputs must be maintained at 1 for normal operation .**

Registers

Figure 6.1
Four-bit register.



Registers with Parallel Load:

- The pulses are applied to all flip-flops and registers in the system. The master clock acts like a drum that supplies a constant beat to all parts of the system.
- The transfer of new information into a register is referred to as loading or updating the register. If all the bits of the register are loaded simultaneously with a common clock pulse, we say that the loading is done in parallel .
- A clock edge applied to the C inputs of the register of Fig. 6.1 will load all four inputs in parallel. In this configuration, if the contents of the register must be left unchanged, the inputs must be held constant or the clock must be inhibited from the circuit

Registers with Parallel Load:

- **Parallel Load Mechanism:**
 - Registers with parallel load update all bits simultaneously using a common clock edge.
 - A control signal (Load) decides whether new data is written or existing data is retained.
 - This ensures fast and synchronized data transfer across all bits.
- **Clock Control and Synchronization:**
 - Instead of gating the clock, data flow is controlled at the register inputs using multiplexers.
 - This avoids timing issues caused by variable delays in clock paths.
 - Flip-flops retain or load new data based on the Load signal during each clock pulse.

Registers

Registers with Parallel Load:

Clock Gating (Old Method)

In **clock gating**, you literally **stop or pass** the clock to flip-flops depending on an **Enable** signal.

Example: If Enable=0, the clock pulse is blocked; if Enable=1, the clock pulse reaches the flip-flops.

Problem:

The clock has to travel through extra logic (AND gates, buffers, etc.).

These extra delays can differ slightly for different flip-flops.

Result: Not all flip-flops see the clock edge at exactly the same time → **timing skew**.

Skew can cause some flip-flops to update while others don't, leading to corrupted register values.

Input Control with Multiplexers (Better Method)

Instead of touching the clock, keep the **clock clean and uniform** for all flip-flops.

Use a **multiplexer (MUX)** in front of each flip-flop's **D input**:

One input of the MUX is the **new data (D_{in})**.

The other input is the **old output (Q)** fed back.

A **Load (Enable)** signal selects which one to send to the flip-flop's D input.

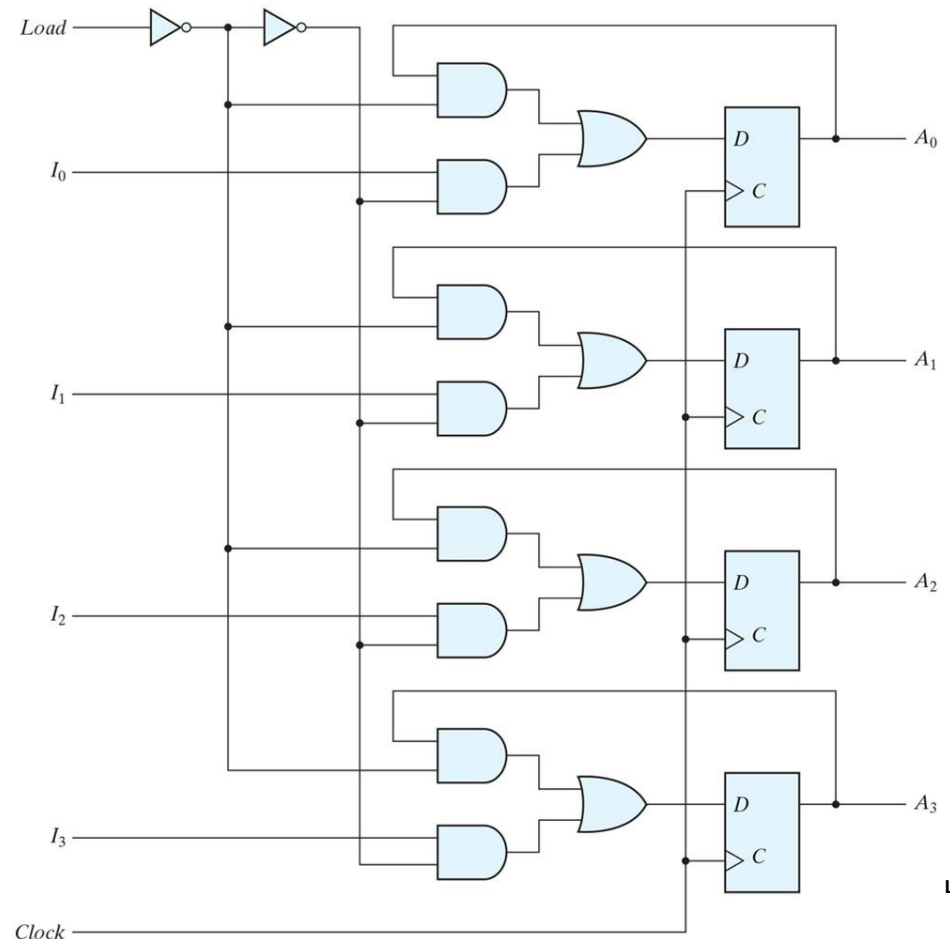
Registers with Parallel Load:

The transfer of information from the data inputs or the outputs of the register is done simultaneously with all four bits in response to a clock edge.

Figure 6.2
Four-bit register
with parallel load.

Load=0 retains data

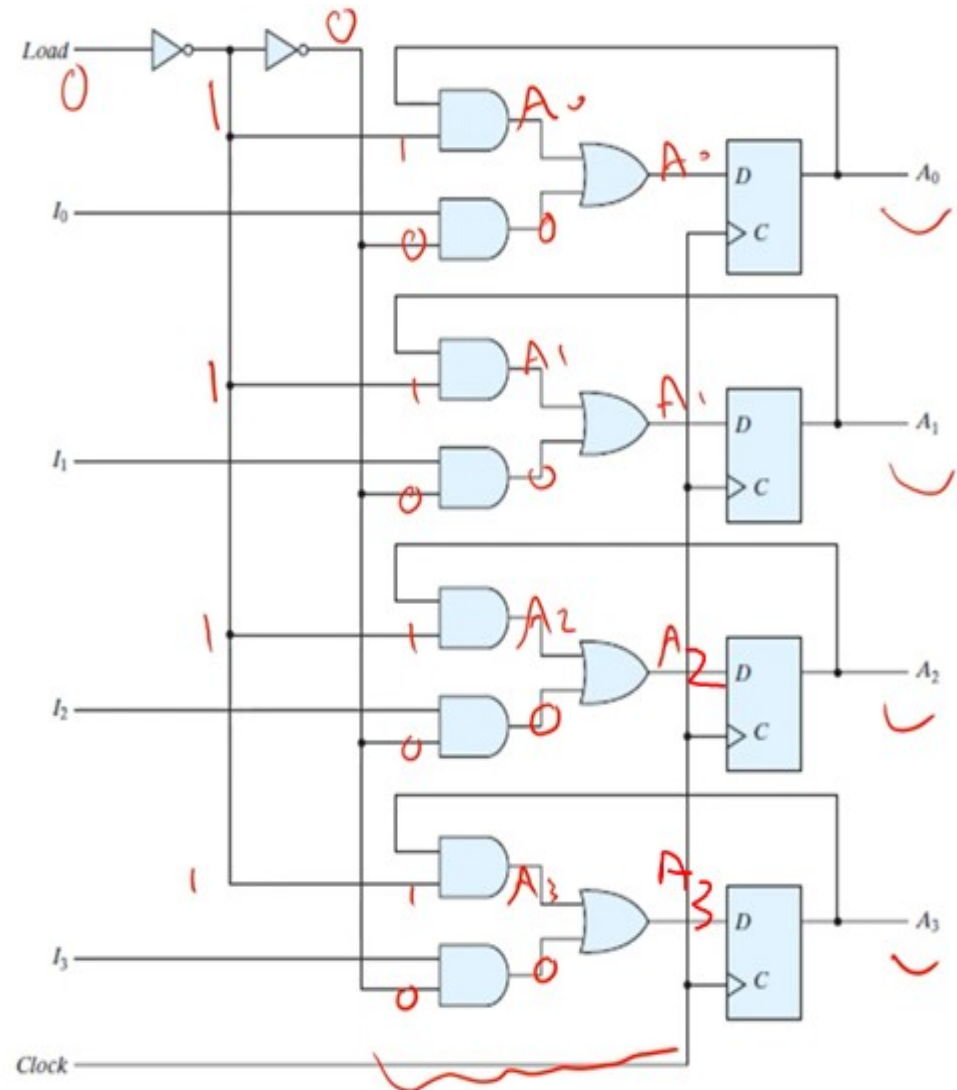
Load =1
Register accepts new data



Registers with Parallel Load:

Figure 6.2
Four-bit register
with parallel load.

Load=0 retains data

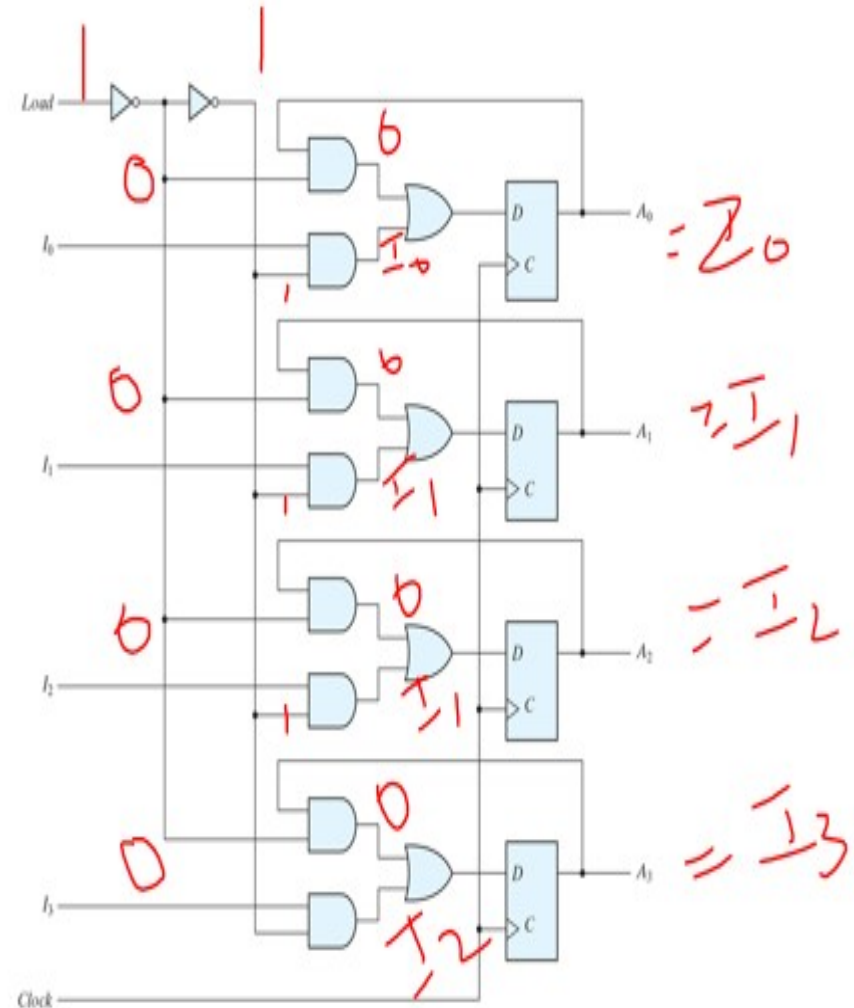


Registers with Parallel Load:

Figure 6.2
Four-bit register
with parallel load.

Load = 1

Register accepts new data



Serial Shift Registers:

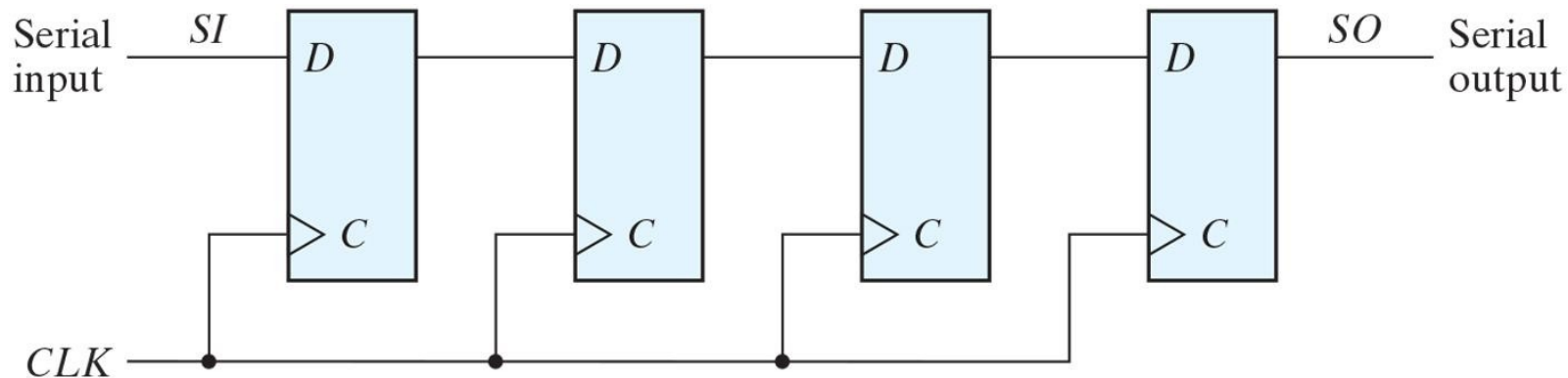
A register capable of shifting the binary information held in each cell to its neighboring cell, in a selected direction, is called a shift register.

The logical configuration of a shift register consists of a chain of flip-flops in cascade, **with the output of one flip-flop connected to the input of the next flip-flop.**

All flip-flops receive common clock pulses, which activate the shift of data from one stage to the next.

Shift Registers

Figure 6.3
Four-bit shift register.



The output of a given flip-flop is connected to the D input of the flip-flop at its right.

This shift register is **unidirectional** (left-to-right).

The serial input determines what goes into the leftmost flip-flop during the shift.

The serial output is taken from the output of the rightmost flip-flop.

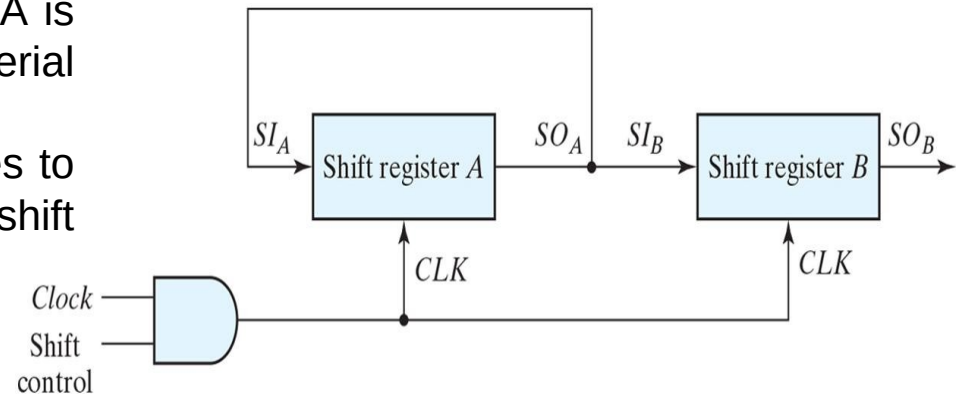
Serial Transfer:

- Serial transfer occurs **one bit at a time**, unlike parallel transfer where **all bits move together**.
- Data is shifted from **register A to register B**, using **shift registers** connected in series.
- To prevent data loss, register A is made to **circulate its own contents**, while register B shifts in the data.
- A **control signal** (Shift Control) enables shifting for **a fixed number of clock cycles**.
- When active, the signal allows **4 clock pulses (T1–T4)** through AND gates to both registers.
- Each **rising clock edge** shifts data one step, syncing both registers with each clock pulse.

Serial Transfer

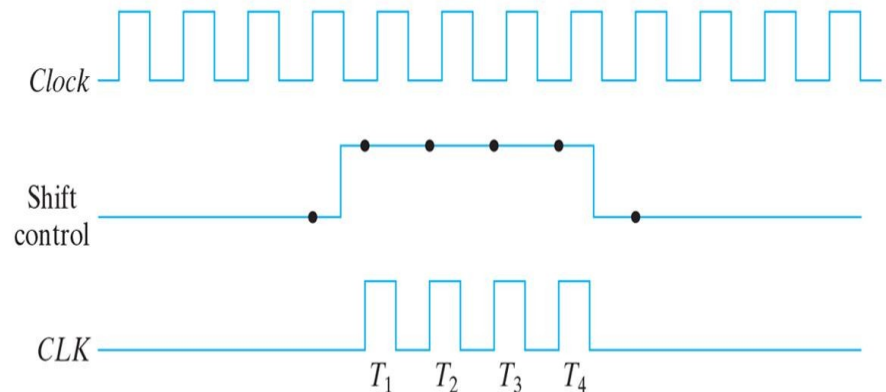
The serial output (SO) of register A is connected to the serial input (SI) of register B. To prevent the loss of information stored in the source register, the information in register A is made to circulate by connecting the serial output to its serial input.

AND gate that allows clock pulses to pass into the CLK terminals only when the shift control is active.



(a) Block diagram

Figure 6.4
Serial transfer from
register A to register B.



(b) Timing diagram

Summary:

Two shift registers: **A** (source) and **B** (destination).

Data is transferred **serially** bit by bit from register A → register B.

A clock signal drives both registers.

But the transfer should **not happen all the time** → only when we want.

That's where **Shift Control** comes in.

The **Shift Control** signal is ANDed with the clock before reaching the registers.

If **Shift Control = 1**:

The AND gate passes the clock pulses.

Registers A and B **shift on each clock edge**.

Data moves serially from A → B.

If **Shift Control = 0**:

Clock pulses are blocked.

Registers **hold their data** (no shifting).

So **Shift Control acts like an enable signal for shifting**.

Serial Transfer

With the first pulse, T_1 , the rightmost bit of A is shifted into the leftmost bit of B and is also circulated into the leftmost position of A. At the same time, all bits of A and B are shifted one position to the right. The previous serial output from B in the rightmost position is lost, and its value changes from 0 to 1.

Timing Pulse	Shift Register A				Shift Register B			
Initial value	1	0	1	1	0	0	1	0
After T_1	1	1	0	1	1	0	0	1
After T_2	1	1	1	0	1	1	0	0
After T_3	0	1	1	1	0	1	1	0
After T_4	1	0	1	1	1	0	1	1

Table 6.1 Serial-Transfer Example.

Thus, the contents of A are copied into B, so that the contents of A remain unchanged i.e., the contents of A are restored to their original value.

Serial Transfer

- In the parallel mode, information is available from all bits of a register and all bits can be transferred simultaneously during one clock pulse.
- In the serial mode mode, the registers have a single serial input and a single serial output. The information is transferred one bit at a time while the registers are shifted in the same direction.

Serial Addition:

- Operations in digital computers are usually done in parallel because that is a faster mode of operation. Serial operations are slower because a datapath operation takes several clock cycles, but serial operations have the advantage of requiring fewer hardware components. In VLSI circuits, they require less silicon area on a chip
- The two binary numbers to be added serially are stored in two shift registers. Two binary numbers are stored in shift registers A (augend) and B (addend). A single full adder adds one bit pair at a time, starting from the least significant bit (LSB).
- The carry-out is stored in a D flip-flop, which feeds into the carry-in of the next bit pair in the next clock cycle.
- With each clock pulse, both registers shift right, a new sum is generated, and carry is updated.
- This continues for n cycles (n = number of bits) until the shift control is disabled, completing the addition process.

Serial Addition

- It is used to perform bit by bit addition in serial form
- It is done using flipflop and adder
- D flipflop- stores carry output after addition
- Right shift register A-A
- Right shift register-B-B
- Add A+B bit by bit using Full adder

Initially, register A holds the augend, register B holds the addend, and the carry flip-flop is cleared to 0. The outputs (SO) of A and B provide a pair of significant bits for the full adder at x and y. Output Q of the flip-flop provides the input carry at z. The shift control enables both registers and the carry flip-flop, so at the next clock pulse, both registers are shifted once to the right, the sum bit from S enters the leftmost flip-flop of A, and the output carry is transferred into flip-flop Q. The shift control enables the registers for a number of clock pulses equal to the number of bits in the registers. For each succeeding clock pulse, a new sum bit is transferred to A, a new carry is transferred to Q, and both registers are shifted once to the right

Serial Addition

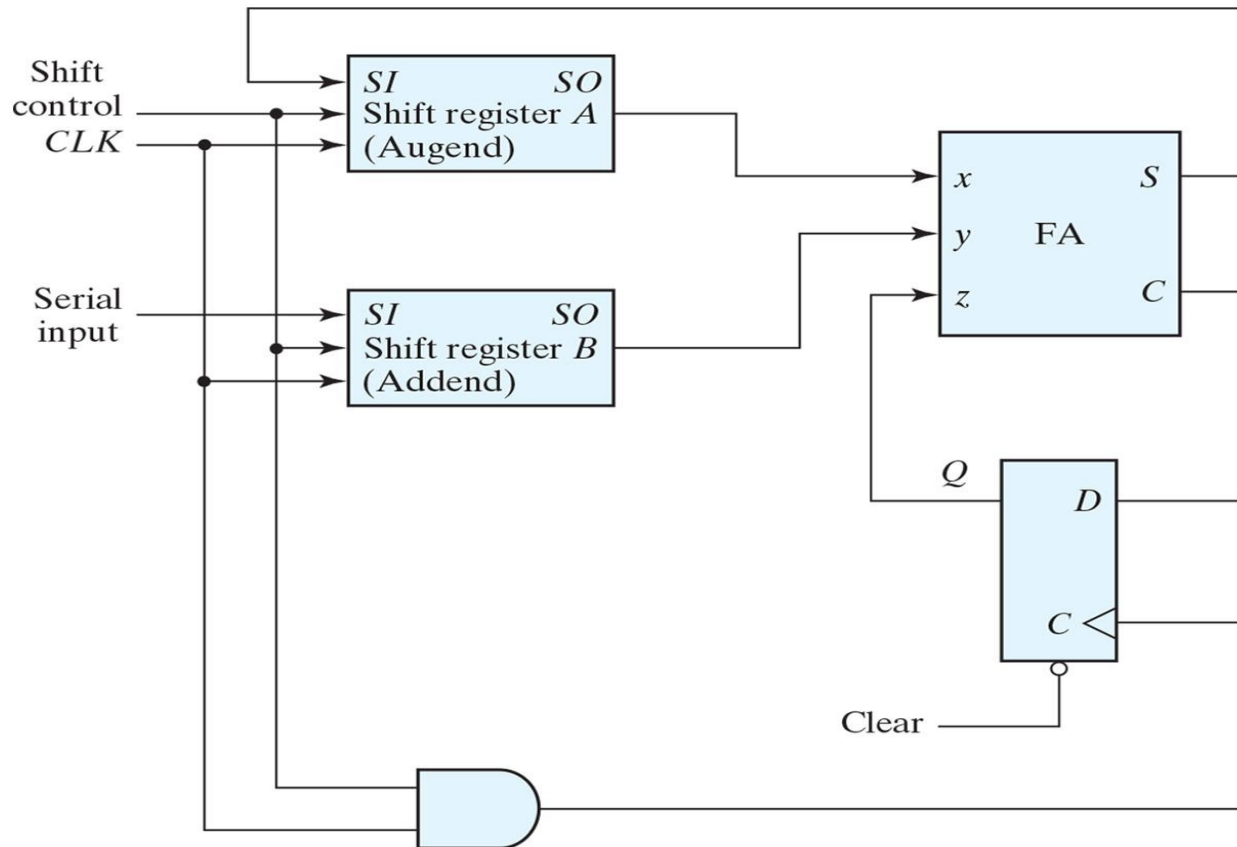
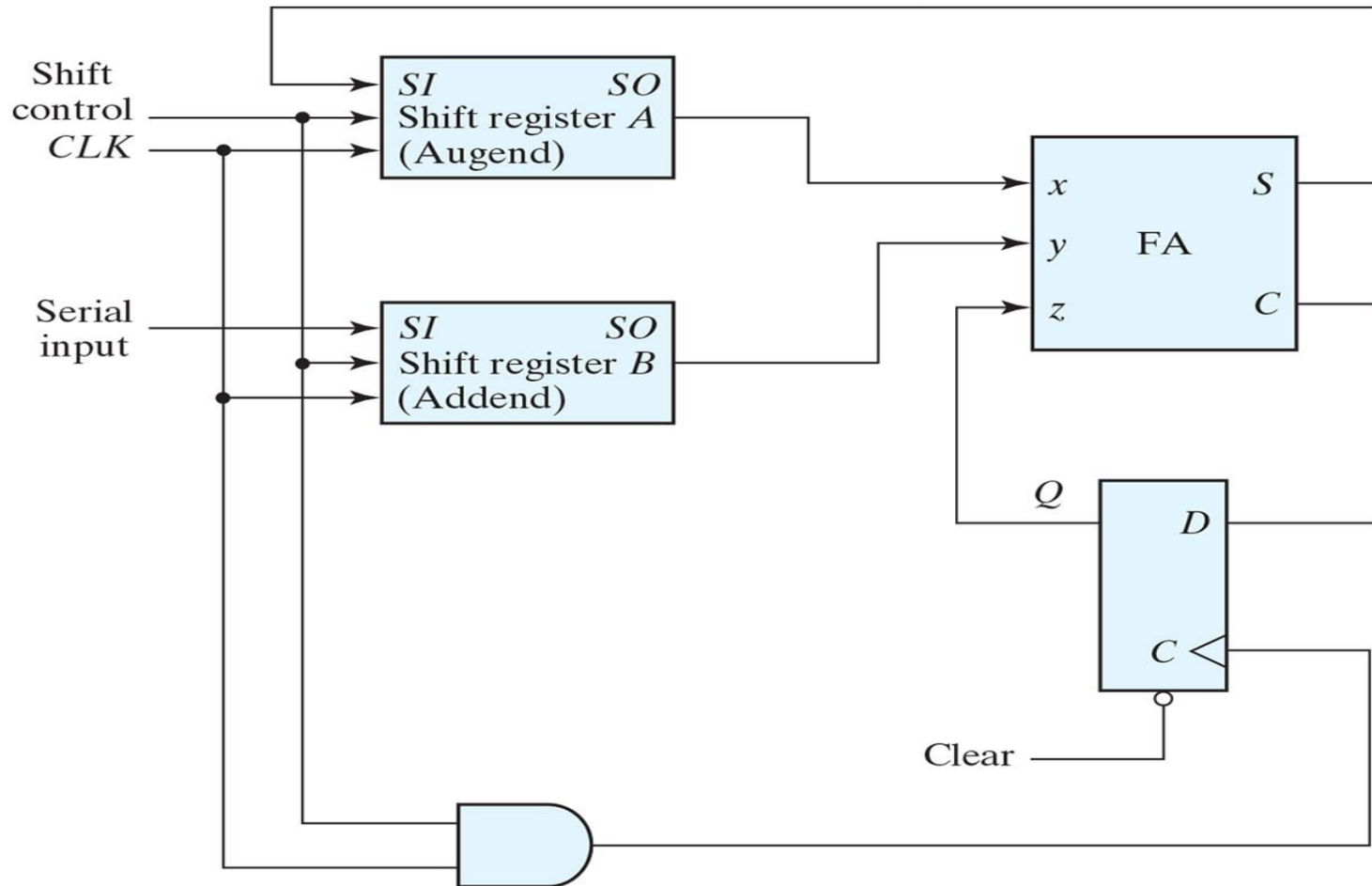
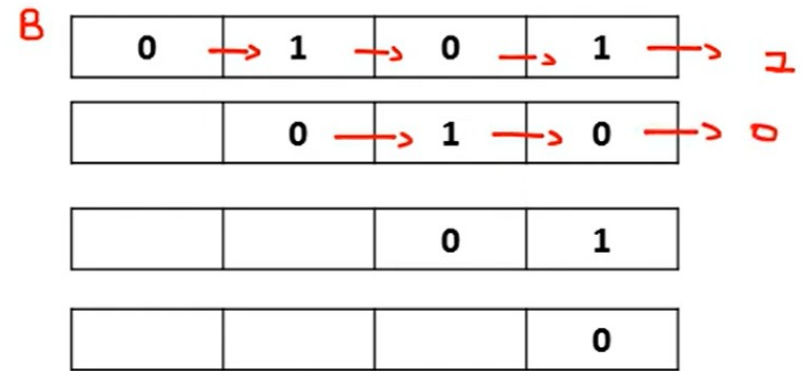
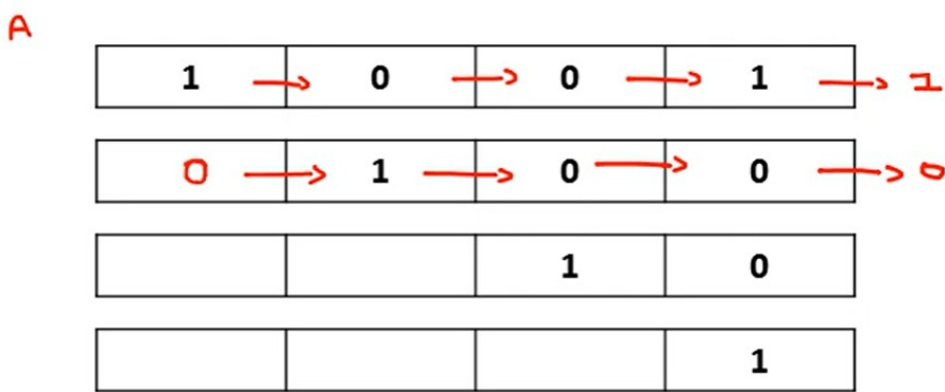


Figure 6.5: Serial adder.

Z is Cin
C is C out
Q=0 initially



Serial Addition

To show that serial operations can be designed by means of **sequential circuit procedure**, we will **redesign** the serial adder with the use of a state table. First, we assume that two shift registers are available to store the binary numbers to be added serially. The serial outputs from the registers are designated by x and y .

Table 6.2 The next state of Q is equal to the output carry.
State Table for Serial Adder.

Present State		Inputs		Next State	Output	Flip-Flop Inputs	
Q		x	y	Q	S	J _Q	K _Q
0		0	0	0	0	0	X
0		0	1	0	1	0	X
0		1	0	0	1	0	X
0		1	1	1	0	1	X
1		0	0	0	1	X	1
1		0	1	1	0	X	0
1		1	0	1	0	X	0
1		1	1	1	1	X	0

Serial Addition

$$J_Q = xy$$

$$K_Q = x'y' = (x + y)'$$

$$S = x \oplus y \oplus Q$$

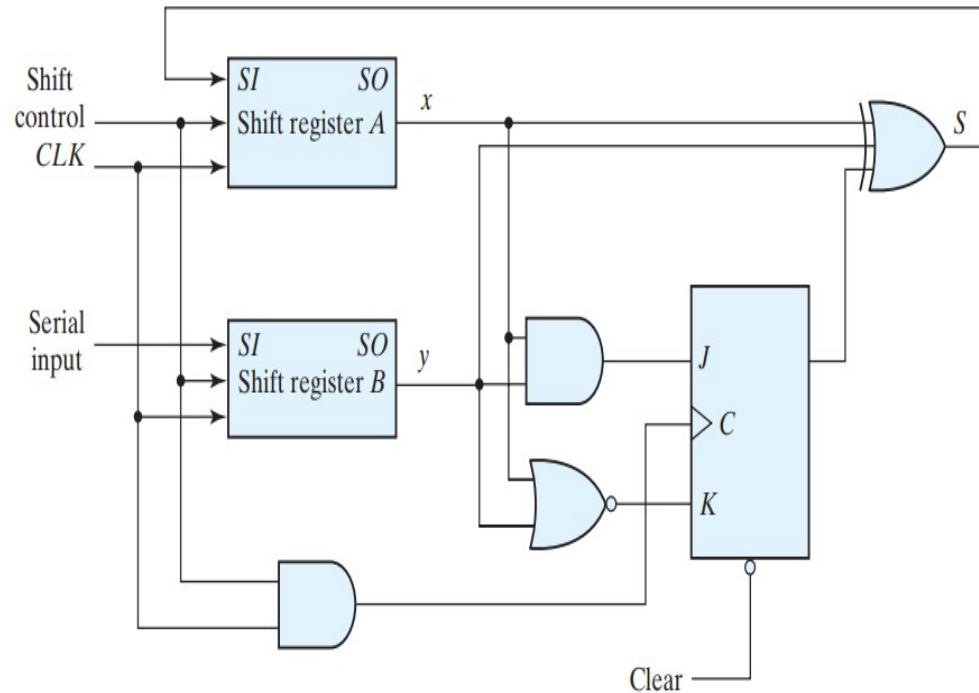
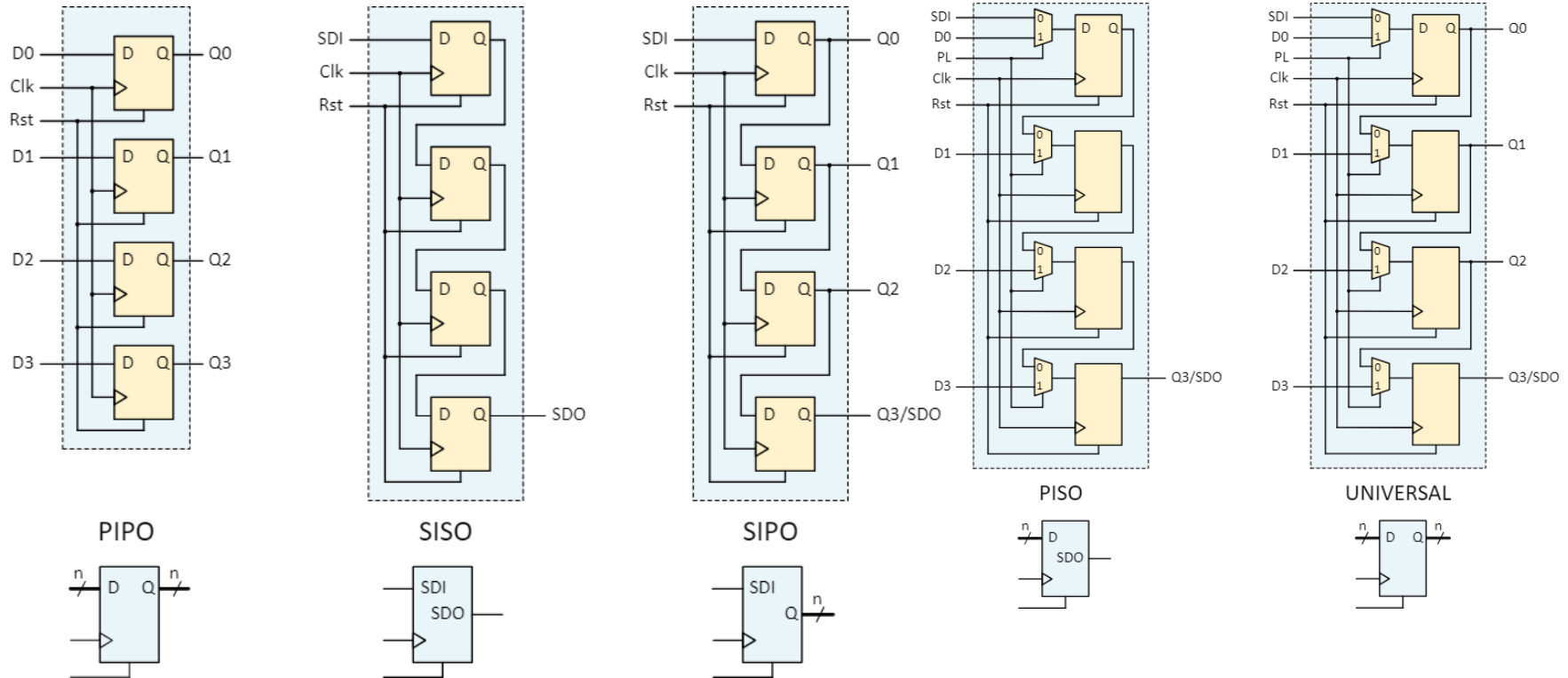


FIGURE 6.6
Second form of serial adder

Types of Representation of Registers

Universal register



The universal register, as the name implies, can be used as a PIPO, SISO, SIPO, or PISO register.

Parallel Data Transfer

Where used: Communication between processor and peripherals.

Example: In a keyboard interface, parallel registers capture all key signals at once for quick transfer to the processor.

Why important: Transfers multiple bits simultaneously, ideal for high-speed operations.

Serial Data transfer:

USB as Serial Communication

Every time you connect a pen drive to your computer, the data is transferred serially (one bit at a time)

uses high-speed serial communication over just a few wires instead of large parallel buses.

Universal Shift Registers

Where used: Versatile data handling.

Example: In data encryption hardware, universal shift registers can load, shift, and output data in multiple formats.

Why important: Provides flexibility for complex digital operations.

In a 4-bit parallel load register, what happens when the Load signal is 0 during a clock pulse?

- A) All bits are shifted to the right
- B) The register is cleared to 0000
- C) The existing data is retained
- D) The register loads new data from inputs

Which of following is true about serial Transfer?

- A. In the serial mode, the registers have a single serial input and a single serial output
- B. If Shift Control = 0: The AND gate passes the clock pulses. Registers A and B shift on each clock edge.
- C. In the serial mode, information is available from all bits of a register and all bits can be transferred simultaneously during one clock pulse.
- D. all statements are true

In a 4-bit parallel load register, what happens when the Load signal is 0 during a clock pulse?

- A) All bits are shifted to the right
- B) The register is cleared to 0000
- C) The existing data is retained
- D) The register loads new data from inputs

Answer: C – The existing data is retained when Load = 0, since no new parallel input is latched.

Which of following is true about serial Transfer?

- A. In the serial mode, the registers have a single serial input and a single serial output
- B. If Shift Control = 0: The AND gate passes the clock pulses. Registers A and B shift on each clock edge.
- C. In the serial mode, information is available from all bits of a register and all bits can be transferred simultaneously during one clock pulse.
- D. all statements are true

ans: A



THANK YOU

Team DDCO

Department of Computer Science and Engineering



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Ripple counters

Team DDCO

Department of Computer Science and Engineering

Ripple Counters(T1- section 6.3)

In digital logic and computing, a counter is a device which stores (and sometimes displays) the **number of times a particular event or process** has occurred, often in relationship to a clock

A register that goes through a **prescribed sequence of states upon the application of input pulses is called a counter**. The input pulses may be clock pulses, or they may originate from some external source and may occur at a fixed interval of time or at random.

The sequence of states may follow the binary number sequence or any other sequence of states. A counter that follows the binary number sequence is called a **binary counter** . **An n -bit binary counter consists of n flip-flops and can count in binary from 0 through $2^n - 1$.**

EX: 3 bit counter- consists of 3 flipflops, counts from 0-7 (000-111)

Ripple Counters

Counters are available in two categories: **ripple counters** and **synchronous counters**.

In a ripple counter, a flip-flop output transition serves as a source for triggering other flip-flops. In other words, the C input of some or all flip-flops are triggered, not by the common clock pulses, but rather by the transition that occurs in other flip-flop outputs.

In a synchronous counter, the C inputs of all flip-flops receive the common clock

Counters (Ripple/synchronous)- **upcounter, down counter, updown counter**

Ripple Counters

Asynchronous / Ripple Counter	Synchronous Counter
Output of one flip-flop drives the clock of the next flip-flop.	No connection between output of one flip-flop and clock of next flip-flop.
Flip-flops are not clocked simultaneously.	Flip-flops are clocked simultaneously.
Circuit is simple even for more number of states.	Circuit becomes complicated as number of states increases.
Speed is slow since clock is propagated through multiple stages.	Speed is high as clock is given to all flip-flops at the same time.

Binary Ripple Counter:

A binary ripple counter consists of a series connection of complementing flip-flops, with the output of each flip-flop connected to the C input of the next higher order flip-flop.

Can be obtained by:

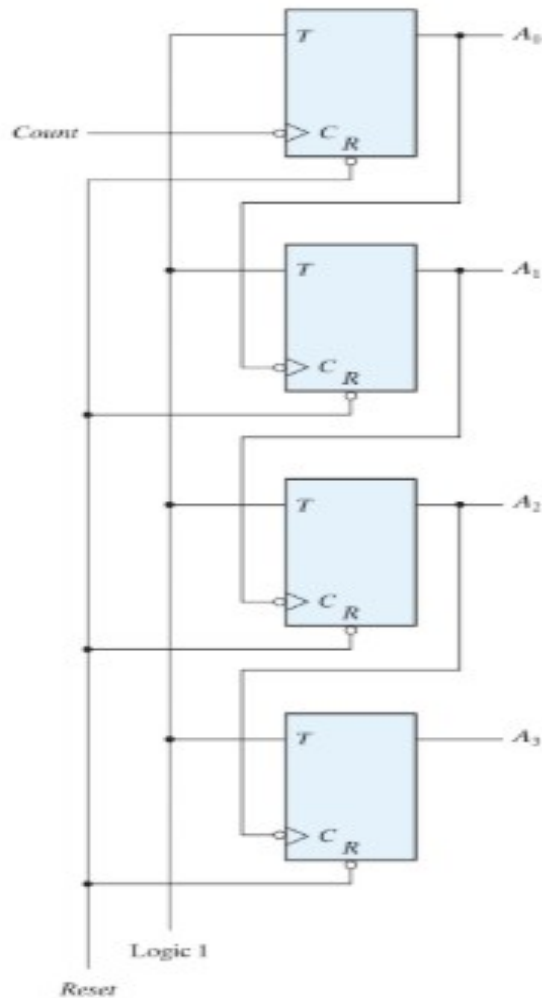
JK connected to same input and converting it into toggle (T flipflop)

D flipflop- D input is always the complement of the present state, and the next clock pulse will cause the flip-flop to complement.

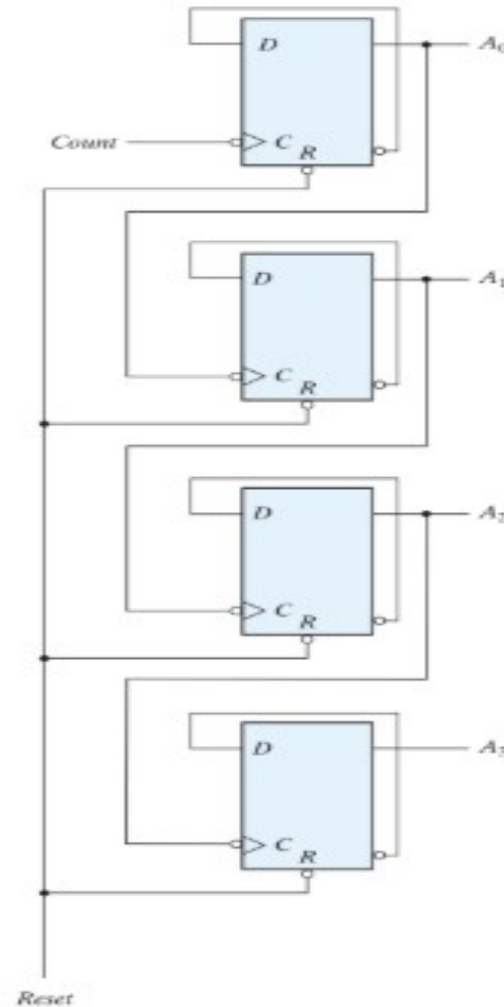
The bubble in front of the dynamic indicator symbol next to C indicates that the flip-flops respond to the negative-edge transition of the input. (transition from 1 to 0)

Binary Ripple Counters-

Figure 6.8
Four-bit binary ripple counter.



(a) With T flip-flops



(b) With D flip-flops

Binary Ripple Counters

Binary Ripple counter:

The flip-flop holding the least significant bit receives the incoming count pulses.

The T inputs of all the flip-flops in

(a) are connected to a permanent logic 1, making each flip-flop complement if the signal in its C input goes through a negative transition. The bubble in front of the dynamic indicator symbol next to C indicates that the flip-flops respond to the negative-edge transition of the input. The negative transition occurs when the output of the previous flip-flop to which C is connected goes from 1 to 0.

Binary Ripple Counters

Binary Ripple Counter:

Table 6.4
Binary Count Sequence

A₃	A₂	A₁	A₀
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0

Binary Ripple Counters

Four-bit Binary Ripple Counter.

Binary Ripple counter : four-bit binary ripple counter

4 flipflop

$2^4=16$ states

0-15 sequence(0000-1111)

The count starts with binary 0 and increments by 1 with each count pulse input. After the count of 15, the counter goes back to 0 to repeat the count

Working-

The least significant bit, A0, is complemented with each count pulse input.

Every time that A0 goes from 1 to 0, it complements A1

Every time that A1 goes from 1 to 0, it complements A2.

Every time that A2 goes from 1 to 0, it complements A3

Binary Ripple Counters

- A binary counter with a reverse count is called a **binary countdown counter** . In a countdown counter, the binary count is decremented by 1 with every input count pulse.
- The count of a four-bit countdown counter starts from binary 15 and continues to binary counts 14, 13, 12, . . . , 0 and then back to 15
- A list of the count sequence of a binary countdown counter shows that the least significant bit is complemented with every count pulse. Any other bit in the sequence is complemented if its previous least significant bit goes from **0 to 1**.
- **all flip-flops trigger on the positive edge of the clock**
- If negative-edge-triggered flip-flops are used, then the C input of each flip-flop must be connected to the complemented output of the previous flip-flop. Then, when the true output goes from 0 to 1, the complement will go from 1 to 0 and complement the next flip-flop as required.

BCD Ripple Counters

A decimal counter follows a sequence of 10 states and returns to 0 after the count of 9.

Such a counter must have at least four flip-flops to represent each decimal digit, since a decimal digit is represented by a binary code with at least four bits

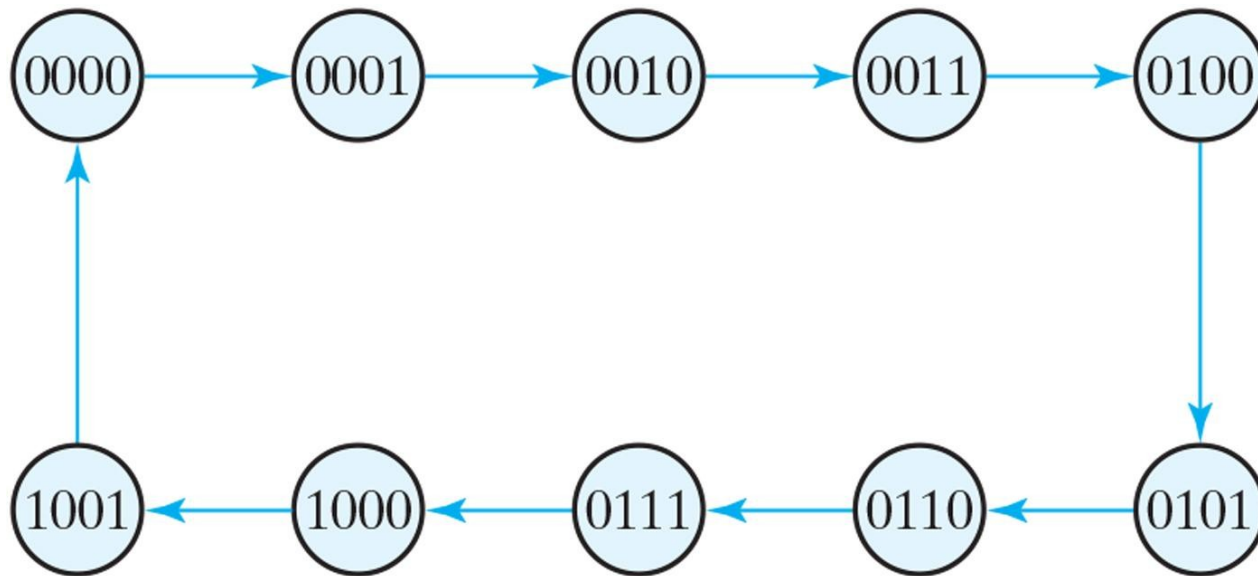
Also called Mod 10 counter.

$$2^n=10$$

$$N=4$$

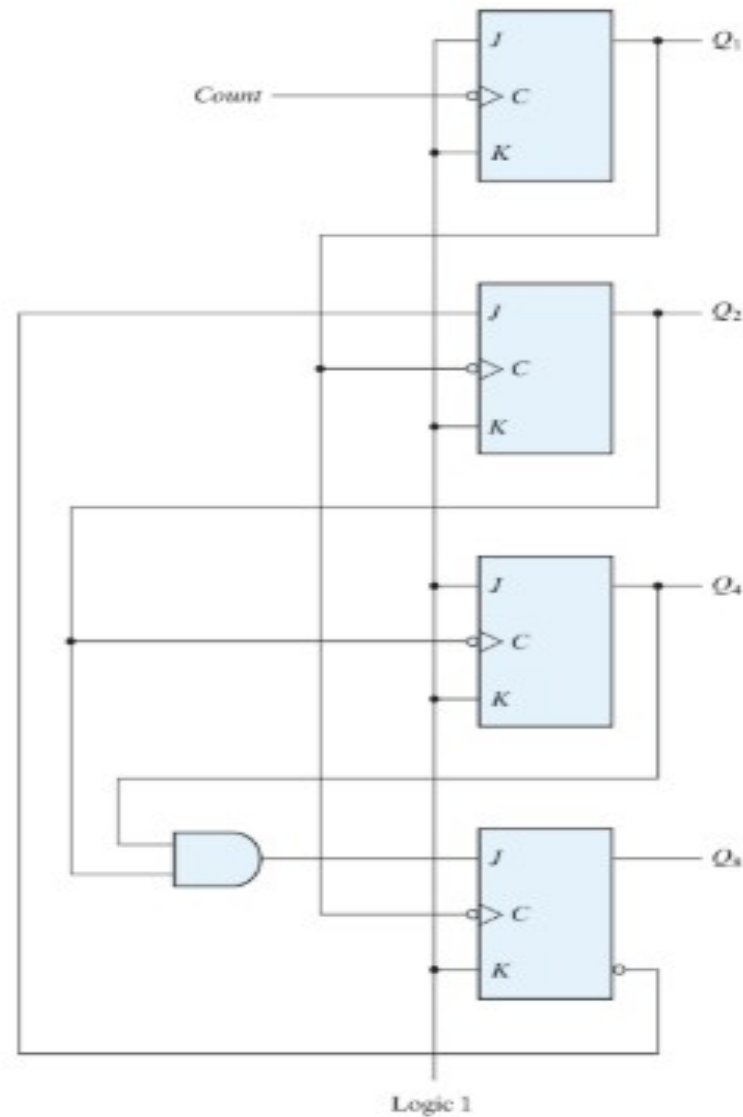
BCD Ripple Counters

Figure 6.9
State diagram of a decimal BCD counter.



BCD Ripple Counters

Figure 6.10
BCD ripple counter.



BCD Ripple Counters

- A ripple counter is an asynchronous sequential circuit. Signals that affect the flip-flop transition depend on the way they change from 1 to 0
- when the C input goes from 1 to 0, the flip-flop is set if $J = 1$, is cleared if $K = 1$, is complemented if $J = K = 1$, and is left unchanged if $J = K = 0$.
- Q1 changes state after each clock pulse. Q2 complements every time Q1 goes from 1 to 0, as long as Q8 = 0. When Q8 becomes 1, Q2 remains at 0. Q4 complements every time Q2 goes from 1 to 0. Q8 remains at 0 as long as Q2 or Q4 is 0. When both Q2 and Q4 become 1, Q8 complements when Q1 goes from 1 to 0. Q8 is cleared on the next transition of Q1.

BCD Ripple Counters

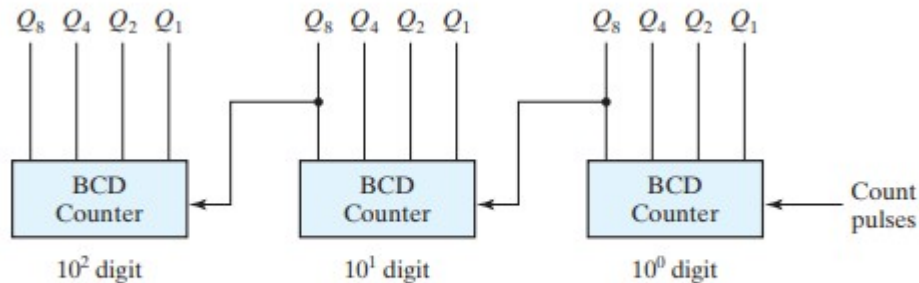


FIGURE 6.11
Block diagram of a three-decade decimal BCD counter

The BCD counter of Fig. 6.10 is a decade counter, since it counts from 0 to 9. To count in decimal from 0 to 99, we need a two-decade counter. To count from 0 to 999, we need a three-decade counter. Multiple decade counters can be constructed by connecting BCD counters in cascade, one for each decade. A three-decade counter is shown in Fig. 6.11. The inputs to the second and third decades come from Q8 of the previous decade. When Q8 in one decade goes from 1 to 0, it triggers the count for the next higher order decade while its own decade goes from 9 to 0.

In automated industries like **bottling plants, packaging units, and textile mills**, machines need to **count repetitive events** — e.g., bottles filled, packets sealed, or cloth rolls measured. A **counter** circuit is ideal because it can automatically keep track of such pulses and trigger control actions.

Ripple Counter Stage : The sensor pulses are fed into a binary ripple counter.

Example: a 4-bit ripple counter can count from 0000 (0) to 1111 (15).

Each flip-flop toggles on the falling edge of the previous flip-flop's output → “ripple effect.”

Count Expansion (Cascading)

If we need to count higher numbers (e.g., 1000 bottles), multiple counters are cascaded (seconds → minutes analogy in clocks).

For example, 3 cascaded BCD ripple counters give **000–999** counts.

Step-by-Step Ripple Counter Operation

Bottle 1 passes → Counter = 0001

Bottle 2 passes → Counter = 0010...

Bottle 15 passes → Counter = 1111 (overflow → resets back to 0)

Cascading ensures higher count capacity (e.g., up to 9999).

In a 4-bit down ripple counter, the initial state is 1111. After 5 clock pulses, the state will be:

- A) 1010
- B) 1011
- C) 1101
- D) 1001

The modulus of a BCD counter is:

- A) 8
- B) 10
- C) 12
- D) 16

In a 4-bit down ripple counter, the initial state is 1111. After 5 clock pulses, the state will be:

- A) 1010
- B) 1011
- C) 1101
- D) 1001

Answer: B) 1011

Explanation: Sequence goes 1111 (15) → 1110 (14) → 1101 (13) → 1100 (12) → 1011 (11).

The modulus of a BCD counter is:

- A) 8
- B) 10
- C) 12
- D) 16

Answer: B) 10

Explanation: BCD counter counts 10 states (0–9). After 1001, it resets to 0000.

If a BCD counter is implemented with 4 flip-flops, how many states are unused?

- A) 2
- B) 4
- C) 6
- D) 10

In a 2-digit decimal counter, how many flip-flops are required?

- A) 4
- B) 6
- C) 7
- D) 8

If a BCD counter is implemented with 4 flip-flops, how many states are unused?

- A) 2
- B) 4
- C) 6
- D) 10

Answer: C) 6

Explanation: 4 flip-flops = 16 possible states (0000–1111). Only 10 (0000–1001) are used. Remaining 6 (1010–1111) are **unused**.

In a 2-digit decimal counter, how many flip-flops are required?

- A) 4
- B) 6
- C) 7
- D) 8

Answer: D) 8

Explanation: One BCD counter needs 4 flip-flops. For 2-digit (00–99), need 2 counters → total 8 flip-flops.



THANK YOU

Team DDCO

Department of Computer Science and Engineering



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Synchronous Counters

Team DDCO

Department of Computer Science and Engineering

Synchronous Counter(T1-section 6.4)

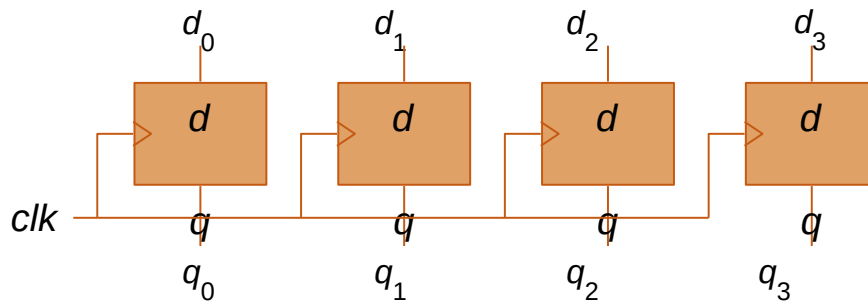
Synchronous counters are different from ripple counters in that clock pulses are applied to the inputs of all flip-flops. **A common clock triggers all flip-flops simultaneously, rather than one at a time in succession as in a ripple counter.**

The decision whether a flip-flop is to be complemented is determined from the values of the data inputs, such as T or J and K at the time of the clock edge. If $T = 0$ or $J = K = 0$, the flip-flop does not change state. If $T = 1$ or $J = K = 1$, the flip-flop complements

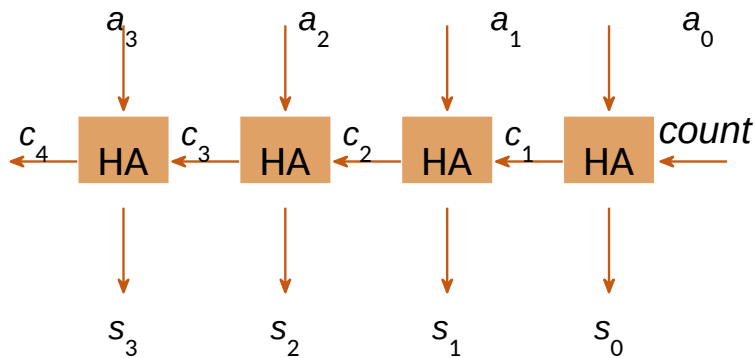
Counters -Recap

Incrementing Counters- Example- count number of people entering a mall

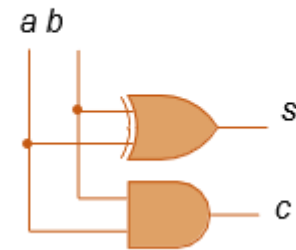
- How to store n -bits? n -bit register



- How to increment an n -bit number? n -bit incrementer



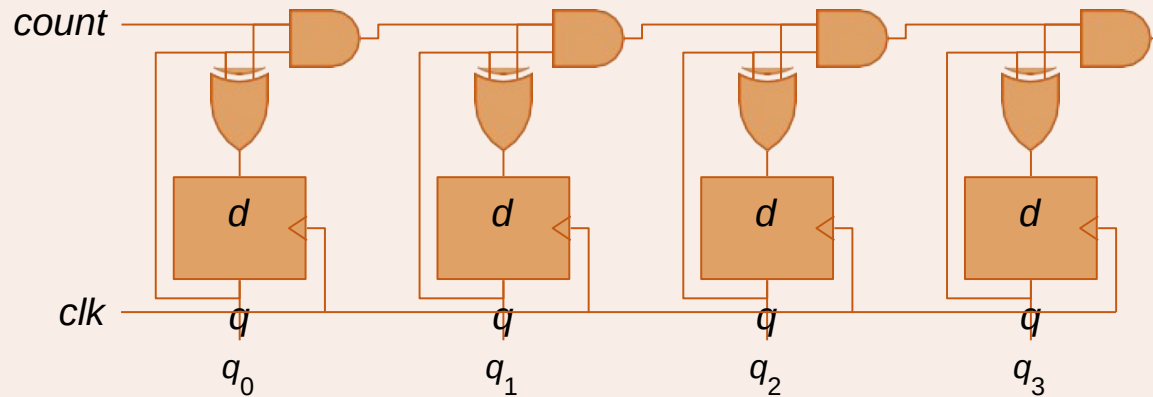
- Half adder logic circuit:



COUNTERS

Incrementing Counters

4-Bit Incrementing Counter

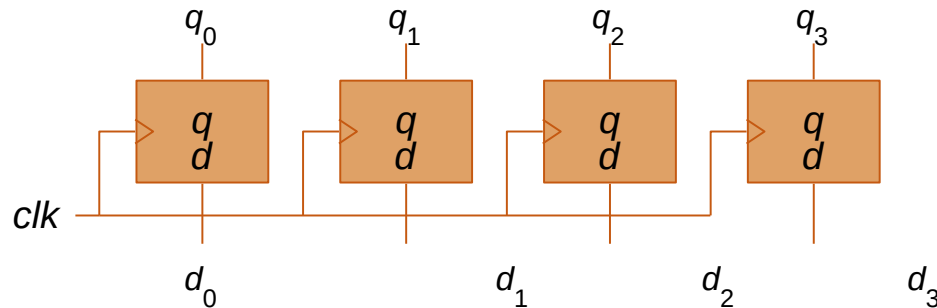


Counters Recap –Decrementing counters

- An n -bit counter that counts back from $2^n - 1$ to 0 (and back to $2^n - 1$)

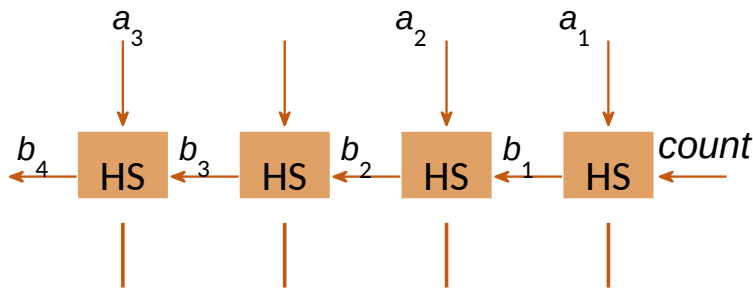
Ex: a 2-bit counter, counts 11, 10, 01, 00, 11, 10,

- How to store n -bits? n -bit register

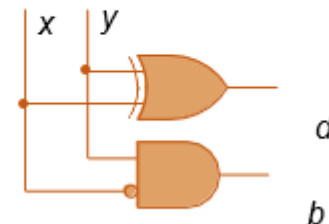


Parking lot counter: Starts at 20 and decrements by 1 for each car entering, displaying remaining slots until it reaches 0 → “Parking Full.”

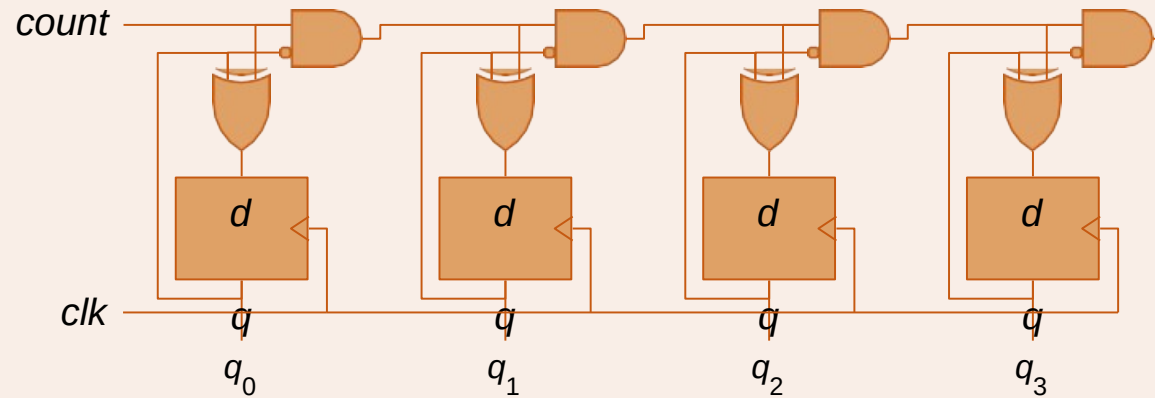
- How to decrement an n -bit number? n -bit decrementer



- Half Subtractor Logic Circuit:



4-Bit Decrementing Counter



Synchronous Counters

Binary Counters

In a synchronous binary counter, **the flip-flop in the least significant position is complemented with every pulse. A flip-flop in any other position is complemented when all the bits in the lower significant positions are equal to 1** . For example, if the present state of a four-bit counter is $A_3A_2A_1A_0 = 0011$, the next count is 0100. A_0 is always complemented. A_1 is complemented because the present state of $A_0 = 1$. A_2 is complemented because the present state of $A_1A_0 = 11$. However, A_3 is not complemented, because the present state of $A_2A_1A_0 = 011$, which does not give an all-1's condition.

The synchronous counter can be triggered with either the positive or the negative clock edge.

Synchronous Counters

The C inputs of all flip-flops are connected to a common clock. The counter is enabled by Count_enable. **If the enable input is 0, all J and K inputs are equal to 0 and the clock does not change the state of the counter.** The first stage, A0, has its J and K equal to 1 if the counter is enabled. The other J and K inputs are equal to 1 if all previous least significant stages are equal to 1 and the count is enabled. The chain of AND gates generates the required logic for the J and K inputs in each stage. **The counter can be extended to any number of stages, with each stage having an additional flip-flop and an AND gate that gives an output of 1 if all previous flip-flop outputs are 1.**

Synchronous Counters

Steps to design the counter using flipflops:

1. Decide the number of flipflop
2. Use excitation of the flipflop
3. State table and ckt excitation table
4. Obtain simplified equation using K map
5. Draw the sequential ckt

<https://www.tutorialspoint.com/digital-electronics/design-of-synchronous-counter.htm>

Synchronous Counters

Design 2 bit up synchronous counter using JK flipflop:
Also called Mod 4 counter

Synchronous Counters

Present (Q1 Q0)	Next (Q1 ⁺ Q0 ⁺)
00	01
01	10
10	11
11	00

Excitation table

Q(n)	Q(n+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Synchronous Counters

Present (Q1 Q0)	Next (Q1 ⁺ Q0 ⁺)	inputs			
		J1	K1	J0	k0
00	01	0	X	1	X
01	10	1	x	x	1
10	11	x	0	1	X
11	00	1	1	x	1

Excitation table

Q(n)	Q(n+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Solving using 2 variable K map of present state

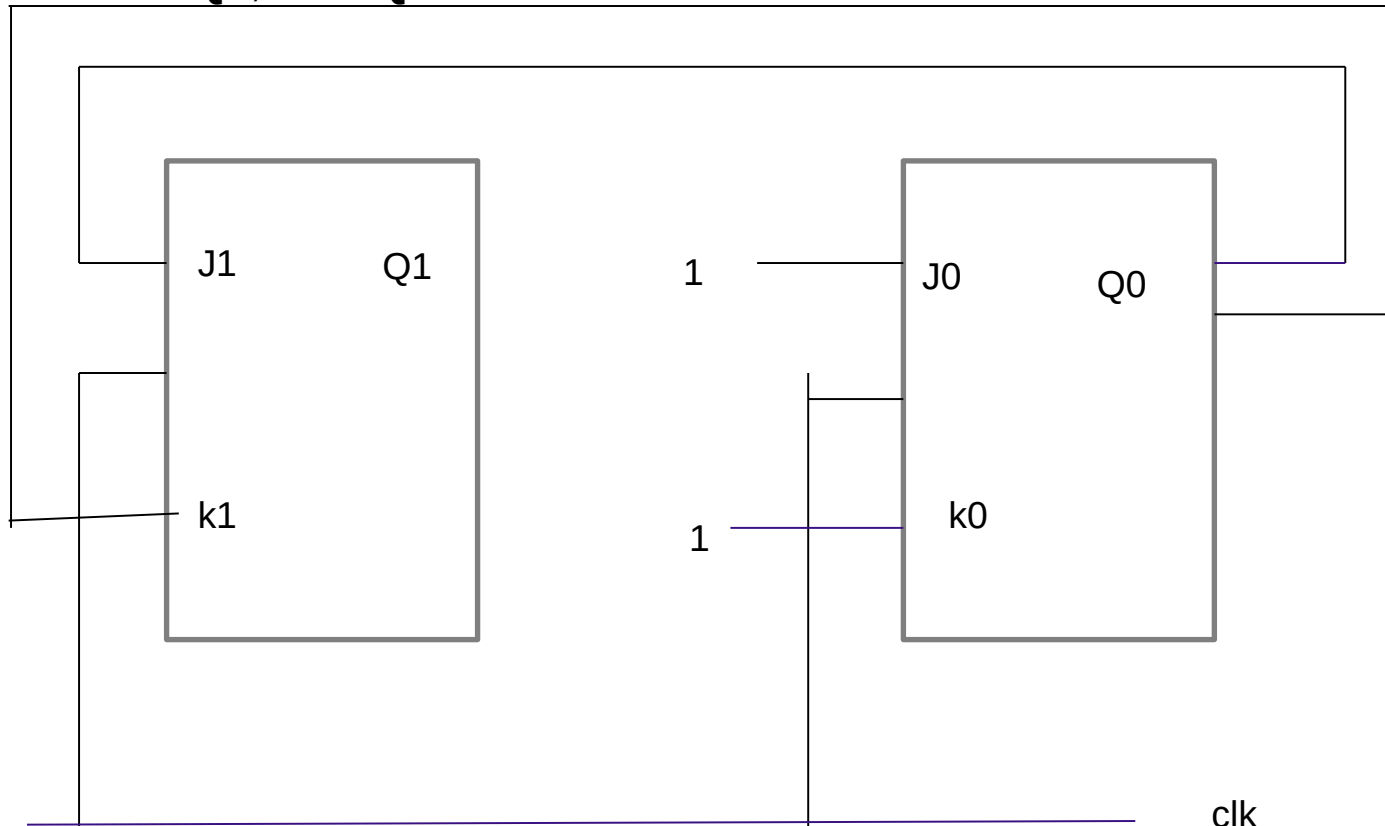
$$J0 = 1, K0 = 1$$

$$J1 = Q0, K1 = Q0$$

Synchronous Counters

$J0 = 1, K0 = 1$

$J1 = Q0, K1 = Q0$



Synchronous Counters

Additional questions/assignment:

1. Design 2 bit synchronous down counter using JK flipflop
2. Design 3 bit synchronous down counter using T flipflop

BCD Counter.

A BCD counter counts in binary-coded decimal from 0000 to 1001 and back to 0000. Because of the return to 0 after a count of 9, a BCD counter does not have a regular pattern, unlike a straight binary count. To derive the circuit of a BCD synchronous counter, it is necessary to go through a sequential circuit design procedure.

BCD Counters o decade counter

Present State				Next State				Output	Flip-Flop Inputs			
Q_8	Q_4	Q_2	Q_1	Q_8	Q_4	Q_2	Q_1	y	T_{Q8}	T_{Q4}	T_{Q2}	T_{Q1}
0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	1	0	0	1	0	0	0	0	1	1
0	0	1	0	0	0	1	1	0	0	0	0	1
0	0	1	1	0	1	0	0	0	0	1	1	1
0	1	0	0	0	1	0	1	0	0	0	0	1
0	1	0	1	0	1	1	0	0	0	0	1	1
0	1	1	0	0	1	1	1	0	0	0	0	1
0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	0	0	1	0	0	0	0	1
1	0	0	1	0	0	0	0	1	1	0	0	1

Table 6.5
State Table for BCD Counter.

$$T_{Q1} = 1$$

$$T_{Q2} = Q_8'Q_1$$

$$T_{Q4} = Q_2Q_1$$

$$T_{Q8} = Q_8Q_1 + Q_4Q_2Q_1$$

$$y = Q_8Q_1$$



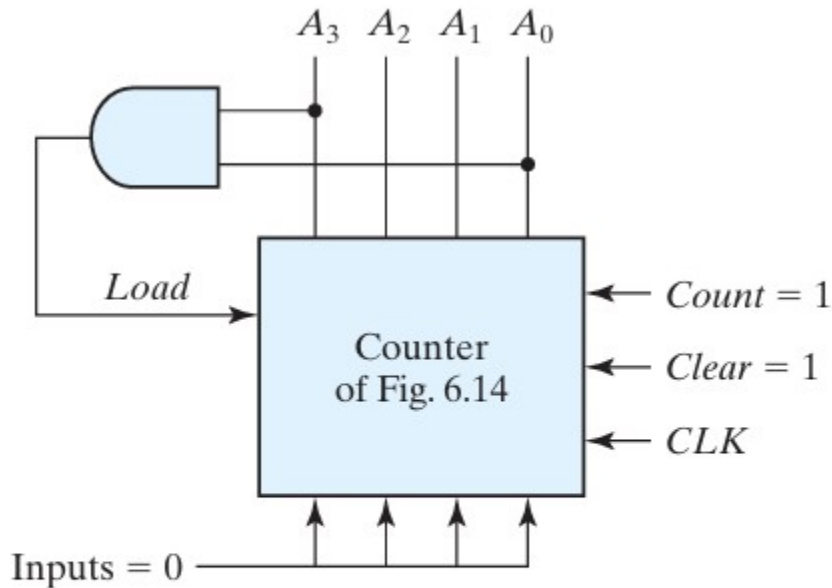
C_out: A carry-out signal that goes high when the counter overflows (from 15 back to 0).

Binary Counter with Parallel Load

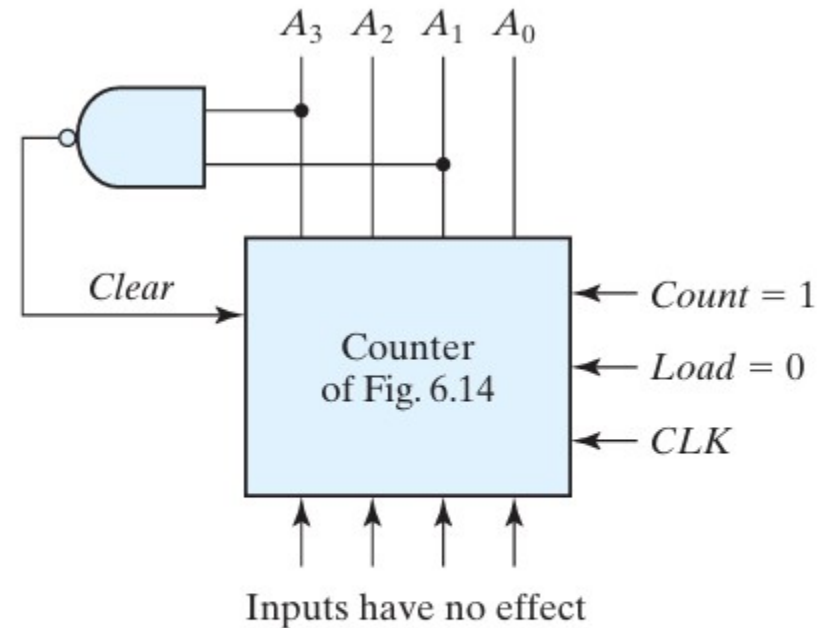
Table 6.6
Function Table for the Counter of Fig. 6.14.

Clear_b	CLK	Load	Count	Function
0	X	X	X	Clear to 0
1	↑	1	X	Load inputs
1	↑	0	1	Count next binary state
1	↑	0	0	No change

Binary Counter with Parallel Load



(a) Using the load input



(b) Using the clear input

FIGURE 6.15

Two ways to achieve a BCD counter using a counter with parallel load

Synchronous Counters application

Example: Image Processing with Synchronous Counters

◆ Scenario

Suppose we have a grayscale image of 256×256 pixels stored in memory.

Each pixel = 1 byte, and all pixels are stored sequentially in RAM.

Pixel(0,0) → Address 0000h

Pixel(0,1) → Address 0001h...

Pixel(255,255) → Address FFFFh

How Synchronous Counter Helps?

A 16-bit synchronous counter acts as the address generator.

On each clock pulse, the counter increments → next memory address is produced instantly.

The memory outputs the pixel value at that address.

Why Synchronous Counter?

All flip-flops update together → no glitching on the address bus.

Essential for high-speed video/image systems (e.g., cameras, medical imaging, graphics cards).

A MOD-10 synchronous counter is implemented. What will be the count sequence of the counter?

- (A) 0000 to 1111
- (B) 0000 to 1001
- (C) 0001 to 1110
- (D) 0000 to 1010

A synchronous counter is designed to count in the sequence: 000 → 010 → 011 → 101 → 110 → 000. What is the modulus of this counter?

- (A) 3
- (B) 5
- (C) 6
- (D) 7

A MOD-10 synchronous counter is implemented. What will be the count sequence of the counter?

- (A) 0000 to 1111
- (B) 0000 to 1001
- (C) 0001 to 1110
- (D) 0000 to 1010

Answer: (B) 0000 to 1001

A synchronous counter is designed to count in the sequence: 000 → 010 → 011 → 101 → 110 → 000. What is the modulus of this counter?

- (A) 3
- (B) 5
- (C) 6
- (D) 7

Answer: (B) 5, The modulus (MOD-N) of a counter = number of unique states it cycles through before repeating.



THANK YOU

Team DDCO

Department of Computer Science and Engineering



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Other counters

Team DDCO

Department of Computer Science and Engineering

Other Counters(T1 section 6.5)

Counters can be designed to generate any desired sequence of states. A divide-by- N counter (also known as a modulo- N counter) is a counter that goes through a repeated sequence of N states.

A circuit with n flip-flops has 2^n binary states. There are occasions when a sequential circuit uses fewer than this maximum possible number of states. **States that are not used in specifying the sequential circuit are not listed in the state table. In simplifying the input equations, the unused states may be treated as don't-care conditions or may be assigned specific next states.**

it is necessary to ensure that the circuit eventually goes into one of the valid states so that it can resume normal operation. Otherwise, if the sequential circuit circulates among unused states, there will be no way to bring it back to its intended sequence of state transitions

Counter with Unused States

A circuit with n flip-flops has 2^n binary states. There are occasions when a sequential circuit uses fewer than this maximum possible number of states. States that are not used in sequential circuit.

The count has a repeated sequence of six states, with flip-flops B and C repeating the binary count 00, 01, 10, and flip-flop A alternating between 0 and 1 every three counts.

Other Counters with Unused States

Table 6.7
State Table for Counter.

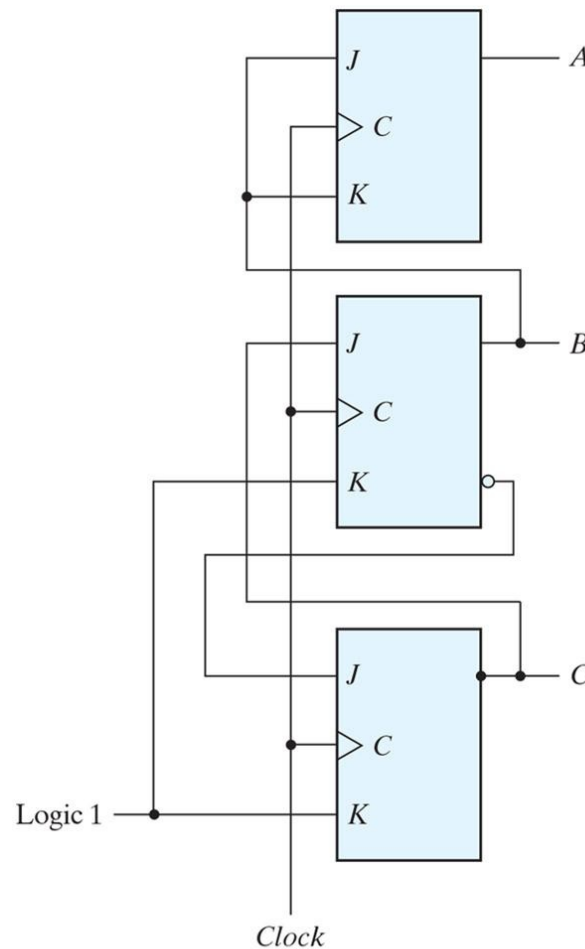
$$\begin{aligned} J_A &= B & K_A &= B \\ J_B &= C & K_B &= 1 \\ J_C &= B' & K_C &= 1 \end{aligned}$$

Present State			Next State			Flip-Flop Inputs					
A	B	C	A	B	C	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	1	0	0	1	X	X	1	0	X
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	0	0	0	X	1	X	1	0	X

Unused states= 011 and 111

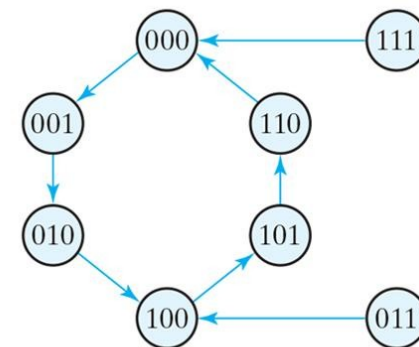
Other Counters with Unused States

Figure 6.16
Counter with
unused states.



(a) Logic circuit diagram

Thus, the counter is **self-correcting**. In a self-correcting counter, if the counter happens to be in one of the unused states, it eventually reaches the normal count sequence after one or more clock pulses



(b) State transition diagram

Ring Counters

A Ring Counter is a circular shift register with only one flip-flop being set at any particular time; all others are cleared. The single bit is shifted from one flip-flop to the next to produce the sequence of timing signals

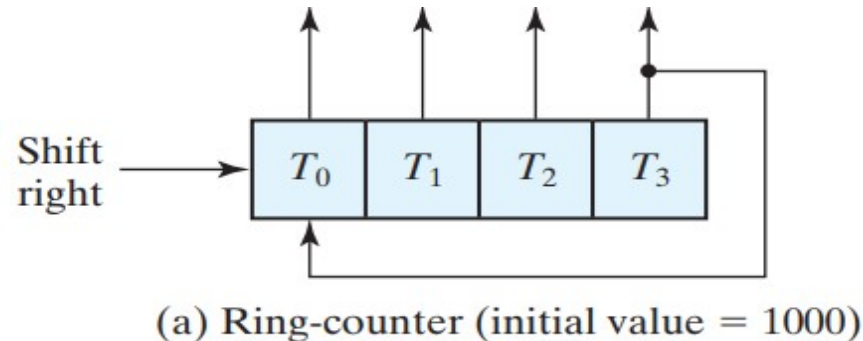
The initial value of the register is 1000 and requires Preset/Clear flip-flops.

Each flip-flop is in the 1 state once every four clock cycles and produces one of the four timing signals

Each output becomes a 1 after the negative-edge transition of a clock pulse and remains 1 during the next clock cycle.

Ring Counters

- Single bit shifted right with every clock pulse and circulates back from T_3 to T_0 . Each flip-flop is in the 1 state once every four clock cycles



- It is negative edge triggered flipflop state changes on negative edge
- Initial value is always 1000

Ring Counters

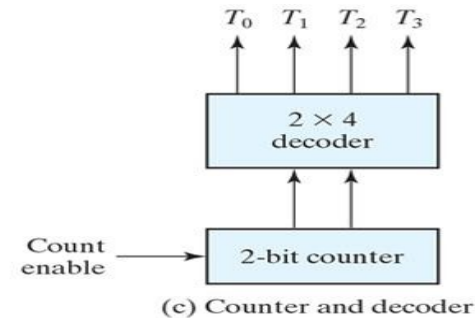
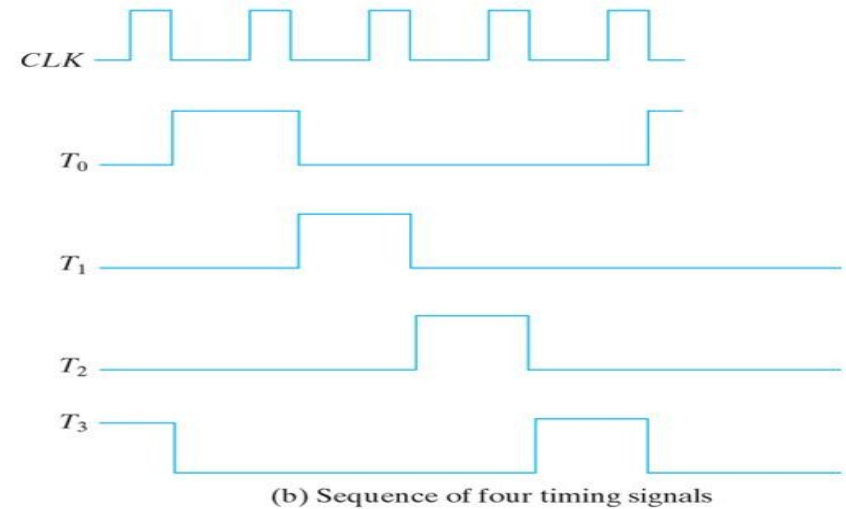
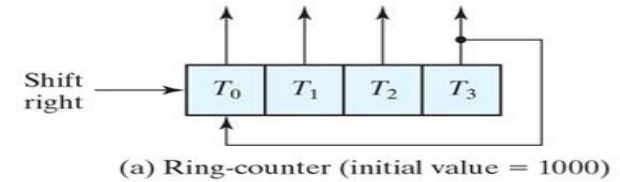


Figure 6.17
Generation of timing signals.

Ring Counters

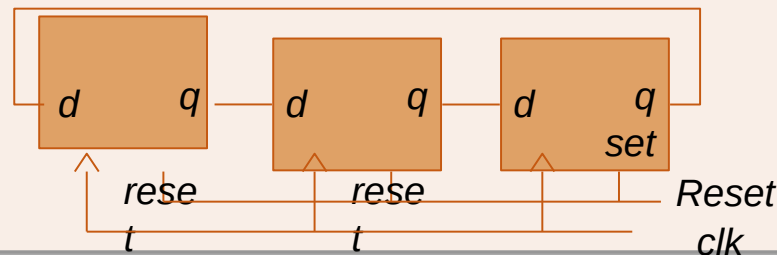
To generate 2^n timing signals, we need either a shift register with 2^n flip-flops or an n -bit binary counter together with an n -to- 2^n -line decoder.

For example, **16 timing signals can be generated with a 16-bit shift register connected as a ring counter or with a 4-bit binary counter and a 4-to-16-line decoder.** In the first case, we need 16 flip-flops. In the second, we need 4 flip-flops and 16 four-input AND gates for the decoder. It is also possible to generate the timing signals with a combination of a shift register and a decoder. That way, the number of flip-flops is less than that in a ring counter, and the decoder requires only two-input gates. This combination is called a Johnson counter .

Ring Counters –Summary

- Counters constructed by connecting together flip-flops in closed loop, typically with a single 1 bit circulating among them, are called **ring counters**
- Modulus of an n -bit ring counter is n

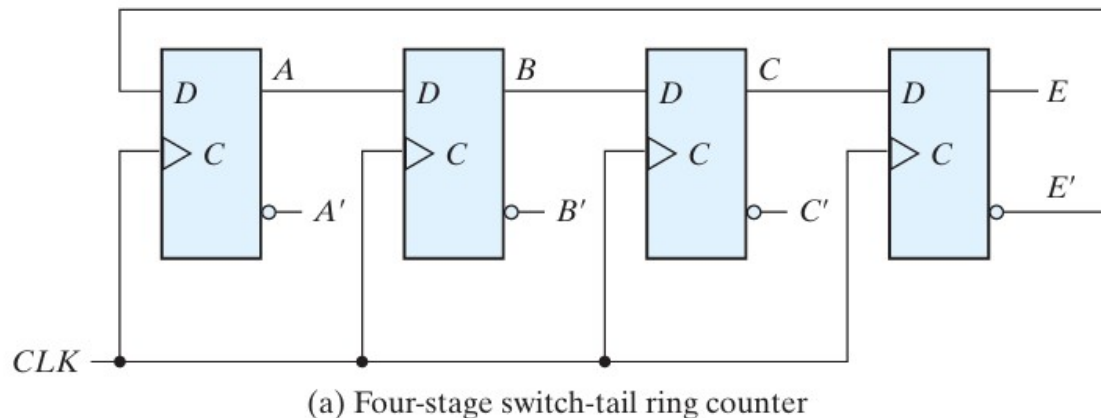
3-bit Ring Counter



- Also called “one-hot” counters (as in one-hot encoding)
Number of states= number of flipflop, mod-8 ring counter requires eight flip-flops and a mod-16 ring counter would require sixteen flip-flops. (number of states=number of flipflops) but Other flipflop number of states is $=2^n$

Johnson Counter

- A k -bit ring counter circulates a single bit among the flip-flops to provide k distinguishable states. The number of states can be doubled if the shift register is connected as a **switch-tail ring counter**.
- A switch-tail ring counter is a circular shift register with the complemented output of the last flip-flop connected to the input of the first flip-flop.



Johnson counter

Sequence number	Flip-flop outputs				AND gate required for output
	<i>A</i>	<i>B</i>	<i>C</i>	<i>E</i>	
1	0	0	0	0	$A'E'$
2	1	0	0	0	AB'
3	1	1	0	0	BC'
4	1	1	1	0	CE'
5	1	1	1	1	AE
6	0	1	1	1	$A'B$
7	0	0	1	1	$B'C$
8	0	0	0	1	$C'E$

(b) Count sequence and required decoding

Johnson counter

The circular connection is made from the complemented output of the rightmost flip-flop to the input of the leftmost flip-flop. The register shifts its contents once to the right with every clock pulse, and at the same time, the complemented value of the E flip-flop is transferred into the A flip-flop.

In general, a k -bit switch-tail ring counter will go through a sequence of **$2k$ states**. Starting from all 0's, each shift operation inserts 1's from the left until the register is filled with all 1's. In the next sequences, 0's are inserted from the left until the register is again filled with all 0's.

A Johnson counter is a k -bit switch-tail ring counter with $2k$ decoding gates to provide outputs for $2k$ timing signals.

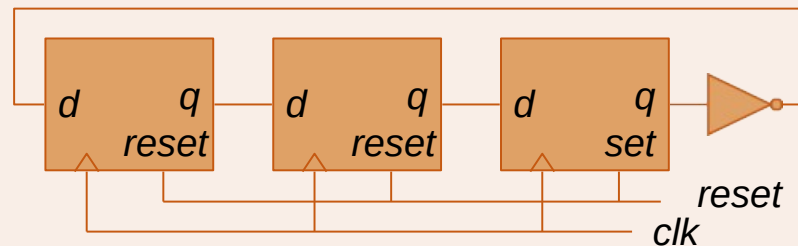
The eight AND gates listed in the table, when connected to the circuit, will complete the construction of the Johnson counter.

Johnson Counter

- One disadvantage of the circuit in Fig. 6.18 (a) is that if it finds itself in an unused state, it will persist in moving from one invalid state to another and never find its way to a valid state. The difficulty can be corrected by modifying the circuit to avoid this undesirable condition
- One correcting procedure is to disconnect the output from flip-flop B that goes to the D input of flip-flop C and instead enable the input of flip-flop C by the function
- $D_C = (A + C)B$ where D_C is the flip-flop input equation for the D input of flip-flop C. Johnson counters can be constructed for any number of timing sequences. The number of flip-flops needed is one-half the number of timing signals.
- The number of decoding gates is equal to the number of timing signals, and only two-input gates are needed.

Johnson counter using D flipflop(negative edge triggered)-summary

3-bit Johnson (Twisted Ring) Counter (switch tail counter)



- What is the modulus of an n -bit Johnson counter?
The MOD of the Johnson counter is $2n$ if n flip-flops are used. The main advantage of the Johnson counter is that it only needs half the number of flip-flops compared to the standard **ring counter** for the same MOD. (4ff=8 states($2n$))

Ring Counter applications

- **Application: Instruction Pipeline Controller in Microprocessors**
- In certain simple microcontrollers and DSP cores, a ring counter is used as the control unit for the pipeline stages (Fetch → Decode → Execute → Memory → Write-back).
- Why? A ring counter naturally cycles one “hot” bit through its outputs, so each stage gets activated in perfect round-robin order.
- This avoids the need for complex decoders and guarantees non-overlapping control signals, which is critical for synchronous CPU timing.
- *Practical use:* Intel’s early pipeline controllers and some ARM/DSP cores have used ring counters in control sequencing because they are simple, reliable, and glitch-free.

Johnson Counter applications

Johnson Counter in LED Chaser / Marquee Display

- A Johnson counter (e.g., CD4017 IC) generates 10 sequential outputs (Q0–Q9).
- Each clock pulse activates one output at a time
- Connect 10 LEDs to Q0–Q9. On each clock tick:
- Q0 HIGH → LED1
- ONNext tick → Q1 HIGH → LED2 ON... continues until Q9, then loops back to Q0.
- LEDs light up one after another in a running sequence → “chaser” or “marquee” effect.

Applications:

- Advertising signboards

Why Johnson Counter?

- Only 5 flip-flops → 10 outputs
- No extra decoder needed

- A **4-bit synchronous counter with unused states** is designed to count only 6 states. If the counter accidentally enters an unused state, it eventually returns to the valid sequence. Such a counter is called:

- (A) Non-deterministic counter
- (B) Self-correcting counter
- (C) Johnson counter
- (D) Binary coded decimal counter

A 4-bit **switch-tail ring counter** (Johnson counter) starts from all 0's. Which of the following will be the **fifth state**?

- (A) 1110
- (B) 1111
- (C) 0111
- (D) 1000

- A **4-bit synchronous counter with unused states** is designed to count only 6 states. If the counter accidentally enters an unused state, it eventually returns to the valid sequence. Such a counter is called:

- (A) Non-deterministic counter
- (B) Self-correcting counter
- (C) Johnson counter
- (D) Binary coded decimal counter

Answer: (B) Self-correcting counter

Explanation: Self-correcting counters ensure that invalid states eventually lead back into the normal cycle.

- A 4-bit **switch-tail ring counter** (Johnson counter) starts from all 0's. Which of the following will be the **fifth state**?

- (A) 1110
- (B) 1111
- (C) 0111
- (D) 1000

Answer: (B) 1111

Explanation: Sequence: 0000 → 1000 → 1100 → 1110 → 1111 → ... The 5th state = **1111**.

- A 3-bit synchronous counter is implemented using T-flip-flops and its present state is 101. What will be the next state?
- A) 110
B) 011
C) 010
D) 001

- A 3-bit synchronous counter is implemented using T-flip-flops and its present state is 101. What will be the next state?
- A) 110
B) 011
C) 010
D) 001
- **Answer:** (A) 110
Explanation: In a synchronous up counter using T-FFs, each output toggles if its T input is 1. For a typical design, all T inputs = 1, so the next state from 101 (5) is 110 (6).

$T_0 = 1$ (LSB toggles on every clock).

$T_1 = Q_0$ (next bit toggles when $Q_0 = 1$).

$T_2 = Q_0 \cdot Q_1$ (next bit toggles when Q_0 and $Q_1 = 1$).

In general, $T_k = Q_0 \cdot Q_1 \cdot \dots \cdot Q_{k-1}$



THANK YOU

Team DDCO

Department of Computer Science and Engineering